

自动驾驶与VLA大模型课程(二)

VLA模块知识

主 讲：咖喱

单 位：清华大学

公众号：自动驾驶之心

自动驾驶之心 全栈知识矩阵

自动驾驶之心，专注自动驾驶与AI



公众号
干货每日放送



B站
不定期直播+最新视频



知识星球
海量图文教程和Paper分享



视频号
技术视频一站式分享

主要内容

① 视觉感知相关

② 语言相关

③ 动作相关基础

④ 大模型技术与智驾

⑤ VLA数据集制作



视觉感知相关

视觉感知的对象

➤ What: 自驾系统需要感知哪些内容?

➤ 知道我们驾驶环境周围的物体、地图元素、运动流等

目标检测



➤ 包围框的坐标:
(x, y, z, w, l, h, yaw)

轨迹预测



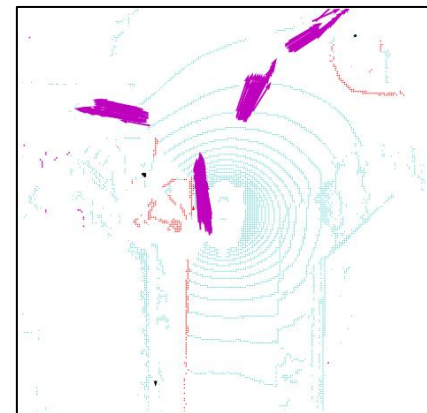
➤ 轨迹点 (z一般是0) : (x1, y1, x2, y2, ...)

地图检测



➤ 几何形状点 (多边形, 直线等)

占据预测



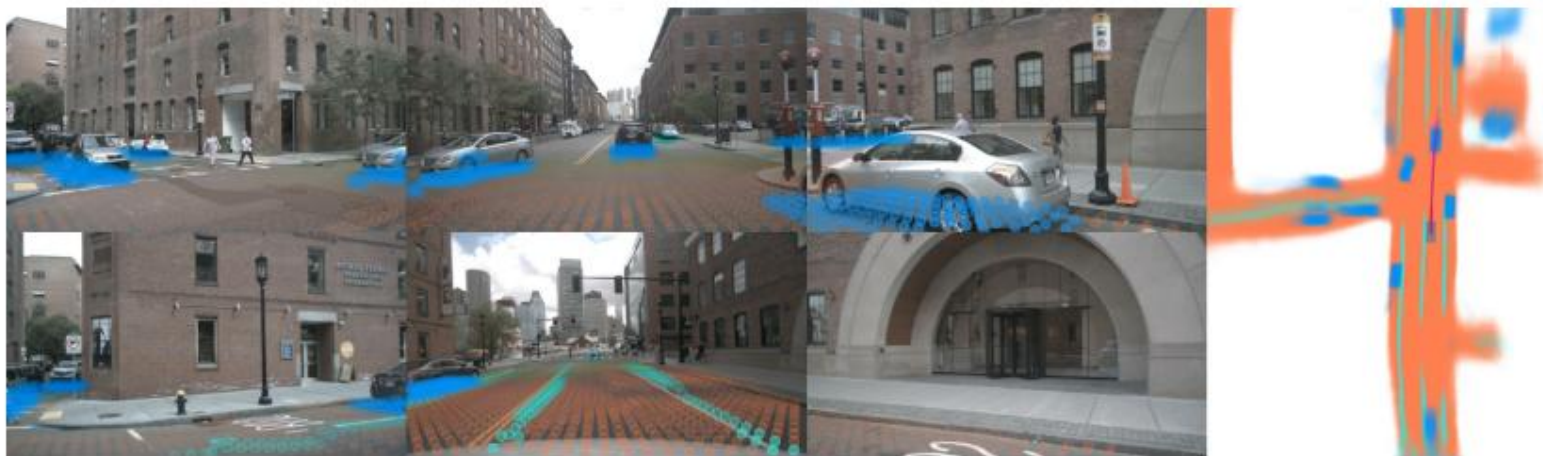
➤ 网格属性图

自动驾驶之心



BEV编码模式感知

- **What:** BEV感知是什么？本质是什么？
- 人类驾驶员是如何感知周围的环境的？
- BEV感知是一种高效的空间表征范式，能够将来自多种传感器模态（如相机、激光雷达（LiDAR）、毫米波雷达）的异构数据投影到统一的BEV坐标系中（上帝视角）。



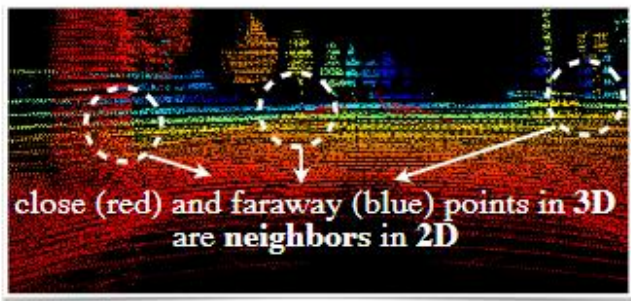
- ✓ 通过这种投影，系统可构建周围环境的一致性结构化空间语义地图。
- ✓ 由于消除了传感器特有的视角差异，这种自上而下的视角能够帮助系统准确感知和理解物体间的空间关系，大幅降低多视角与多模态数据融合的复杂度。



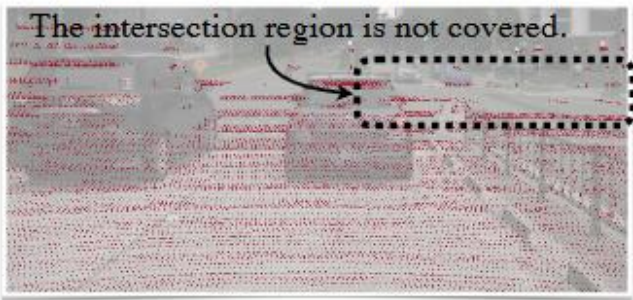
BEV编码模式感知

➤ Why: 为什么BEV感知很重要?

- ❑ 多传感器融合能够有效提供模型的鲁棒性，但是不同模态的异构信息难以直接融合
- ❑ 凭借统一且可解释的俯视空间表征，BEV感知成为自动驾驶中多模态融合感知的理想基础。



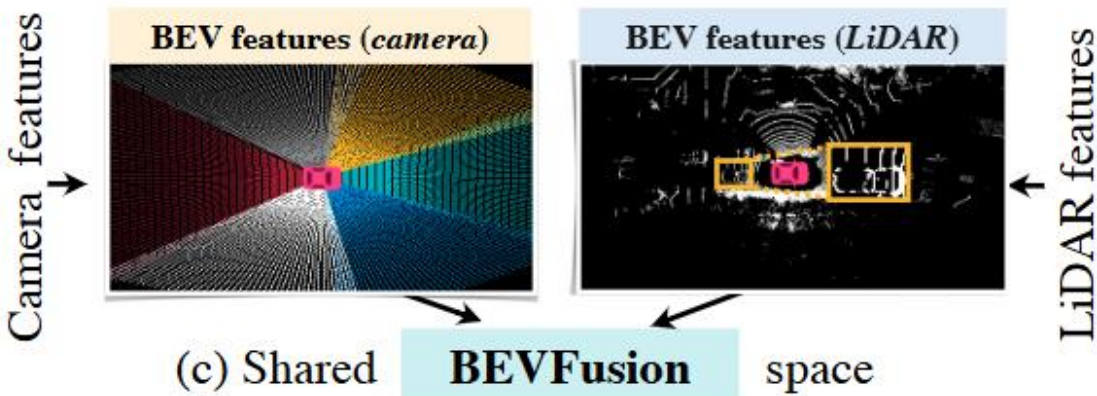
(a) To Camera: *geometric-lossy*



(b) To LiDAR: *semantic-lossy*

- 对于图像：缺失几何信息
- 对于LiDAR：缺失语义信息

BEV结构能够有效保留模态信息



BEV编码模式感知

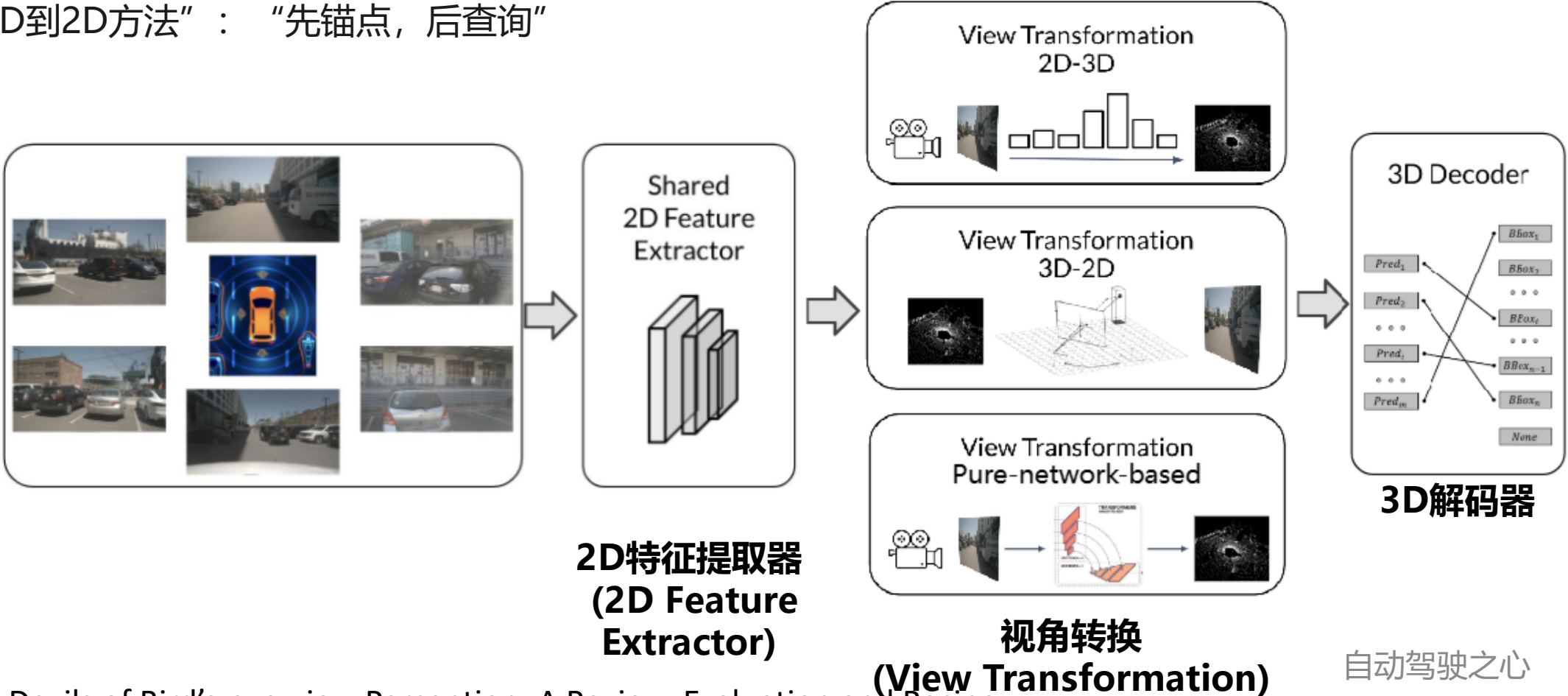
➤ BEV感知可以分为哪几类？



BEV编码模式感知

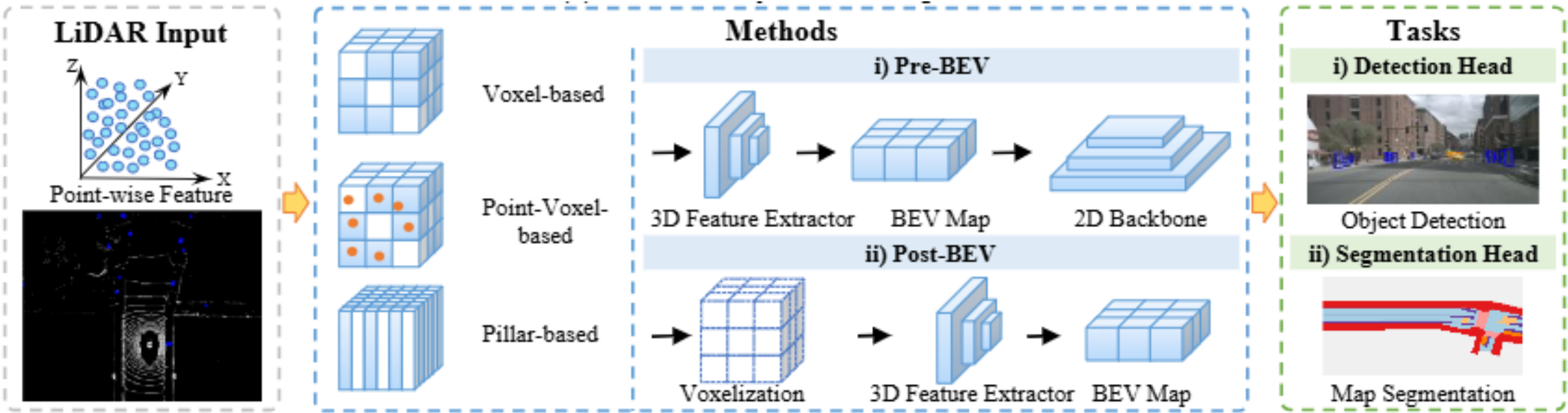
➤以环视图像为中心的BEV感知

- ❑ “2D到3D方法”：“先提升，再展开”
- ❑ “3D到2D方法”：“先锚点，后查询”



BEV编码模式感知

➤以LiDAR为中心的BEV感知



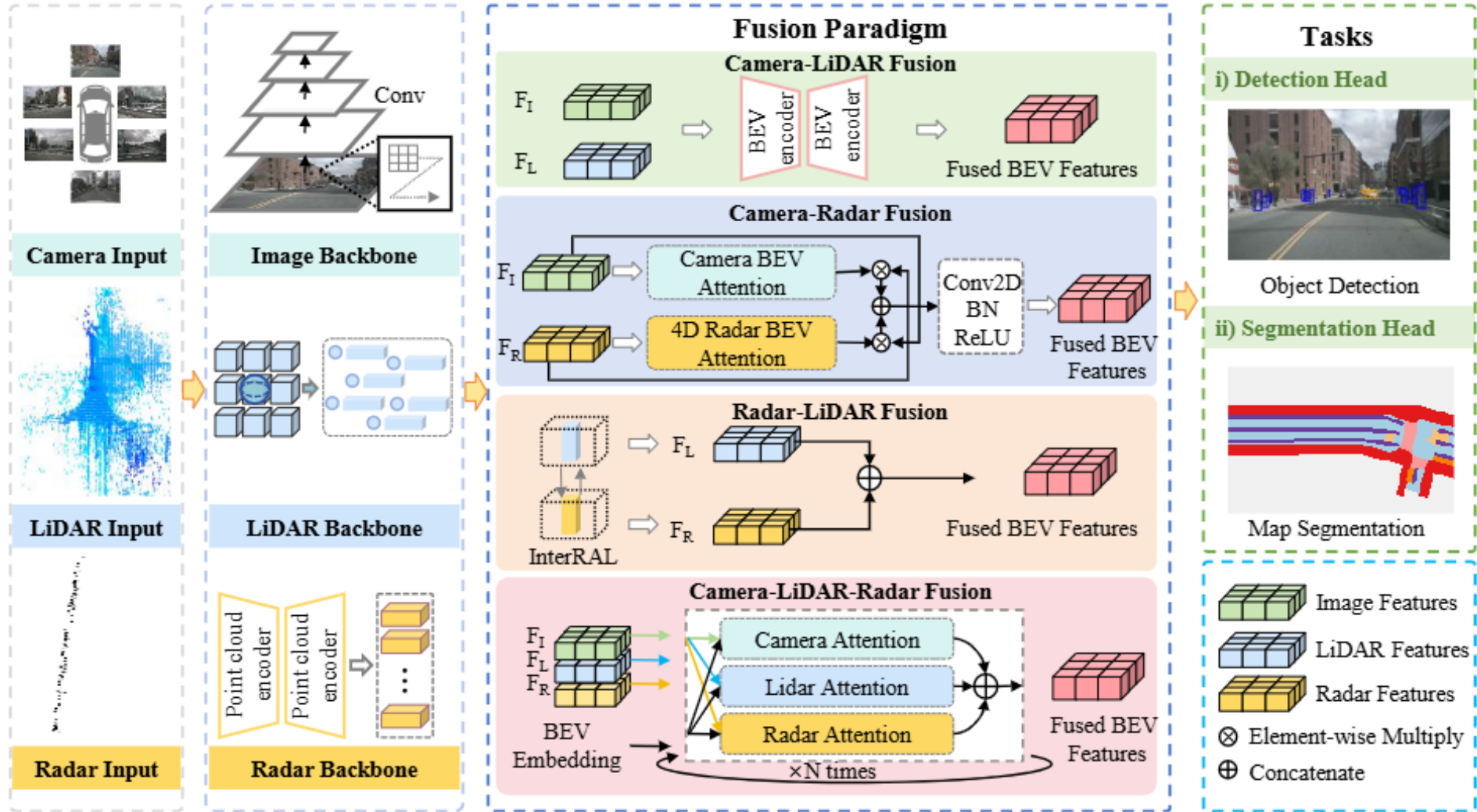
(b) LiDAR-only BEV Perception

- ❑ Pre-BEV方法：投影前先进行特征提取——基于体素/点-体素混合方法，提取点云特征
- ❑ Post-BEV方法：投影后特征提取——将3D点云先投影到2D网格或类图像表示中



BEV编码模式感知

➤ 基于车辆多模态融合的BEV感知



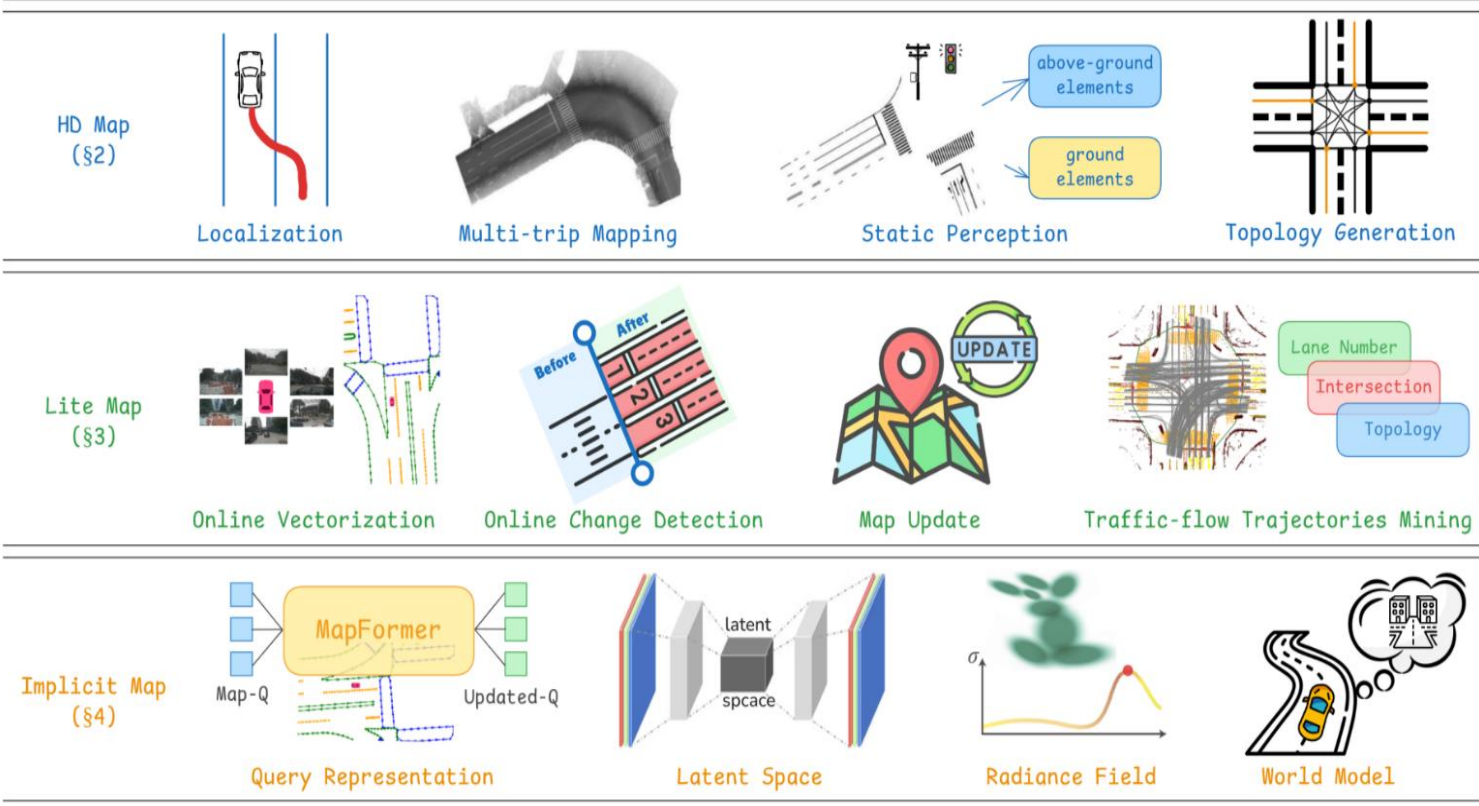
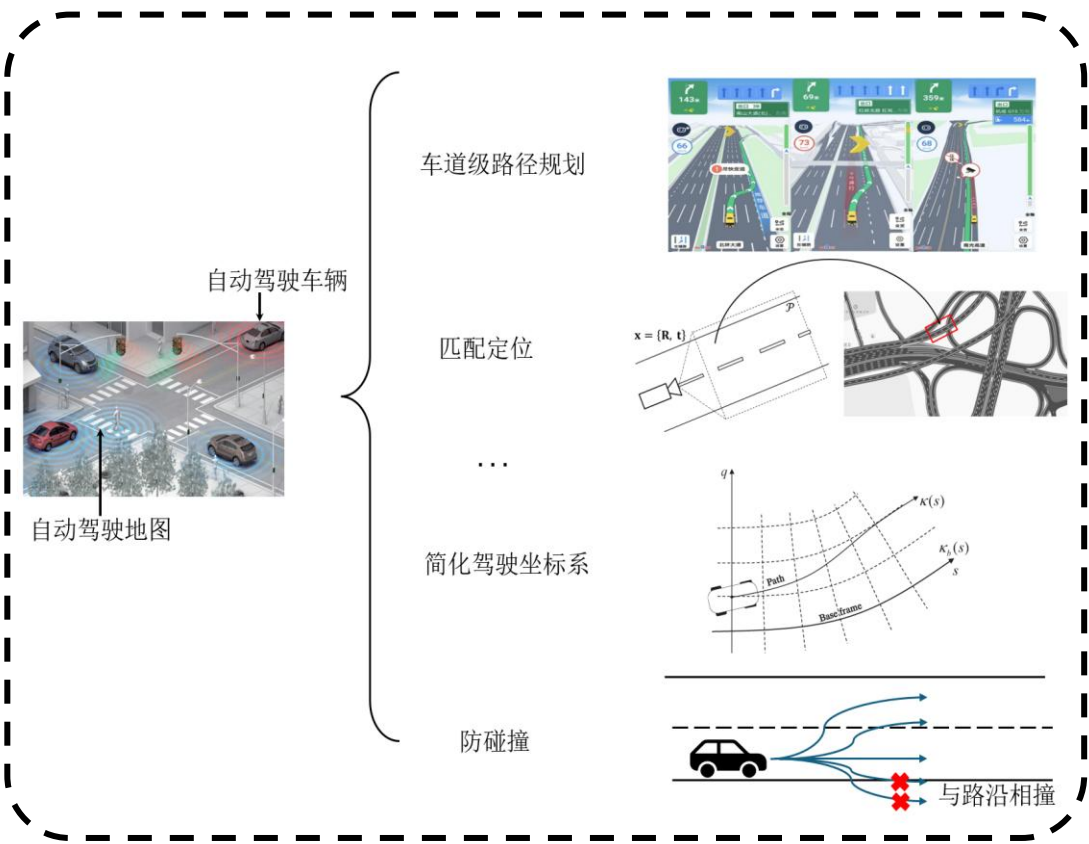
可分为5类:

- ❑ 相机-雷达融合
- ❑ 相机-激光雷达融合
- ❑ 雷达-激光雷达融合
- ❑ 相机-激光雷达-雷达融合
- ❑ 时序融合



地图要素检测

➤ 自动驾驶地图是什么？



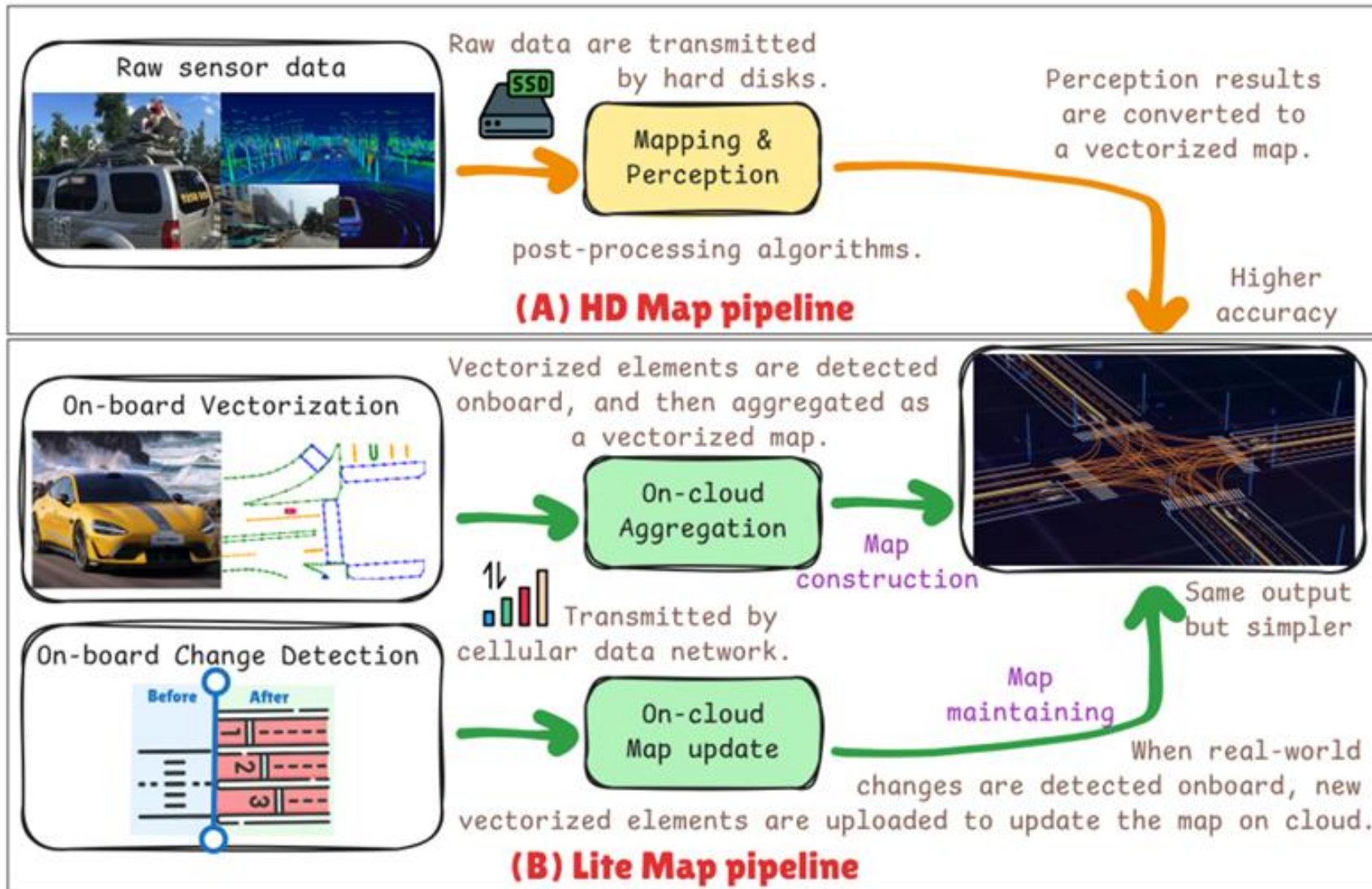
❑ 自动驾驶对地图的精度和丰富度提出了更高要求，需要厘米级车道信息（如道路几何、车道边界、交通信号、语义地标等），以支持高精度定位、辅助感知信息、辅助预测模型和实现规划。

自动驾驶之心



地图要素检测

➤传统高精地图&在线建图



HD Map:

测绘 → 感知 → 地图编译

- ❑ 高生产成本
- ❑ 低更新频率
- ❑ 难以快速响应现实变化

Lite Map:

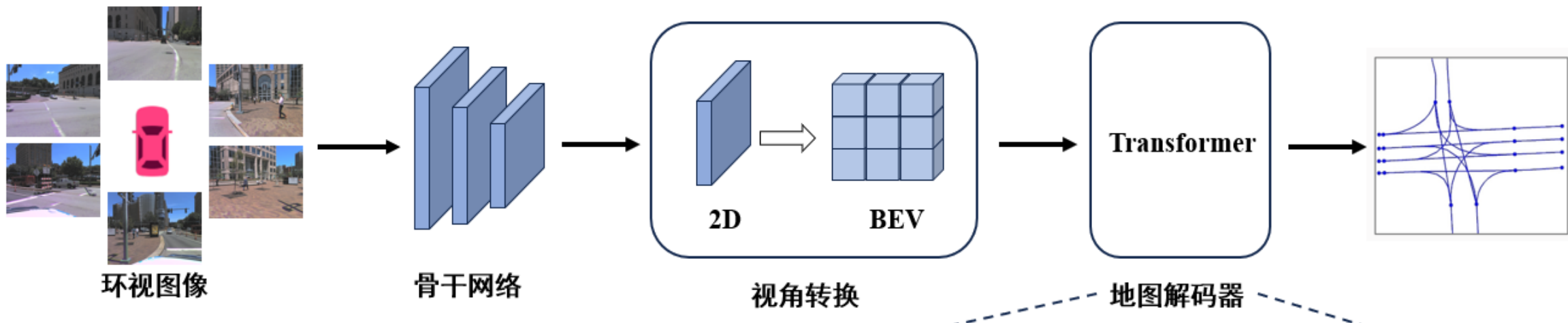
- ❑ 使用量产车采集数据
- ❑ 通过众包方式收集数据
- ❑ Lite地图是HD地图的简化版，元素种类更少，精度较低但足以支持自动驾驶需求

自动驾驶之心



地图要素检测

➤ 在线建图流程

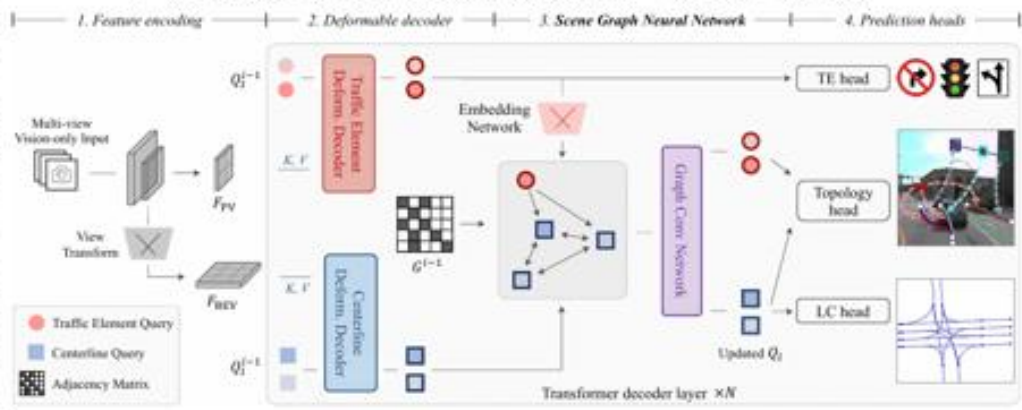


➤ 在线建图范式分类

✓ 地图要素检测（可视物理要素层）



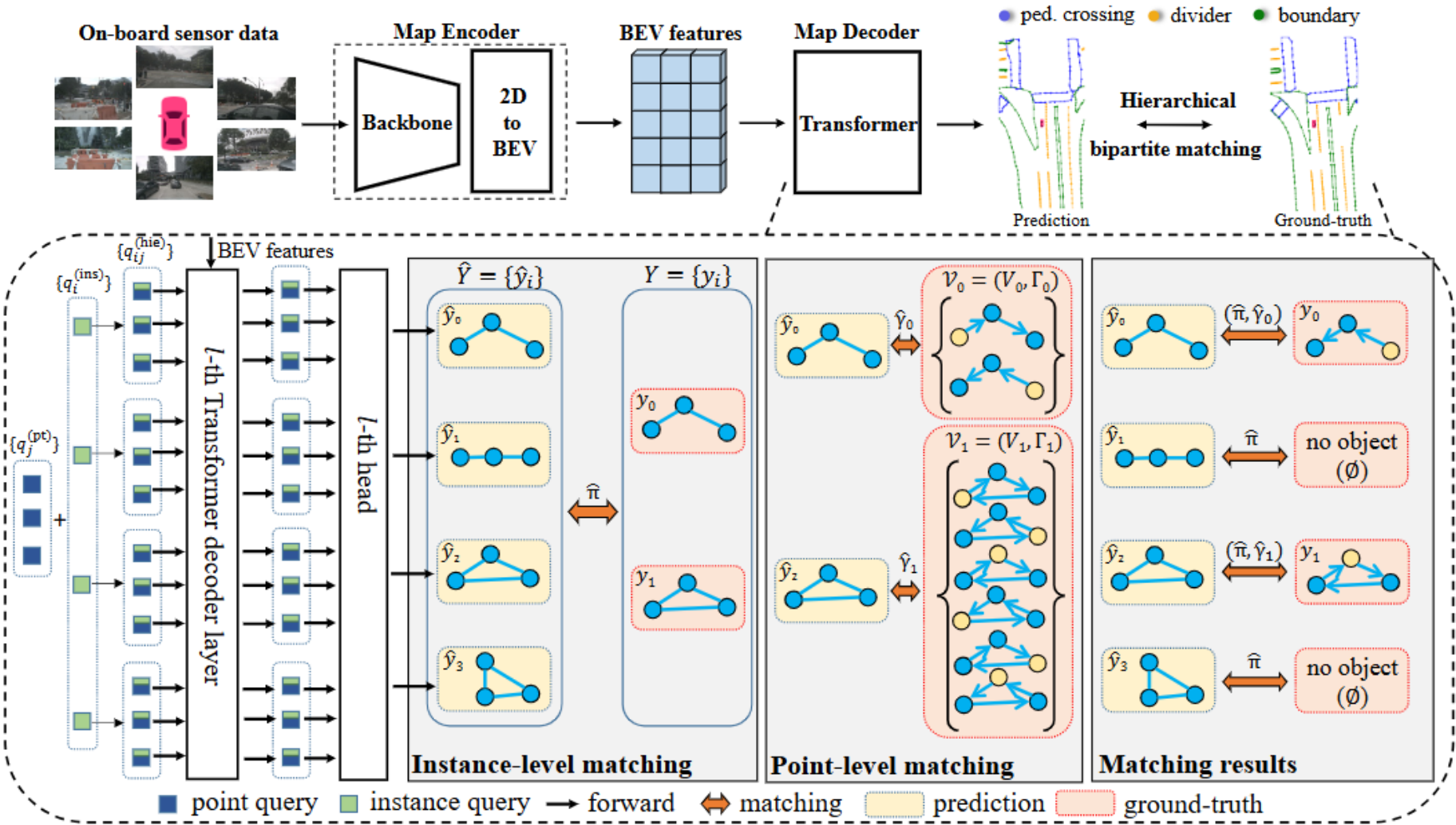
✓ 中心线及拓扑关系推理（逻辑层）



地图要素检测

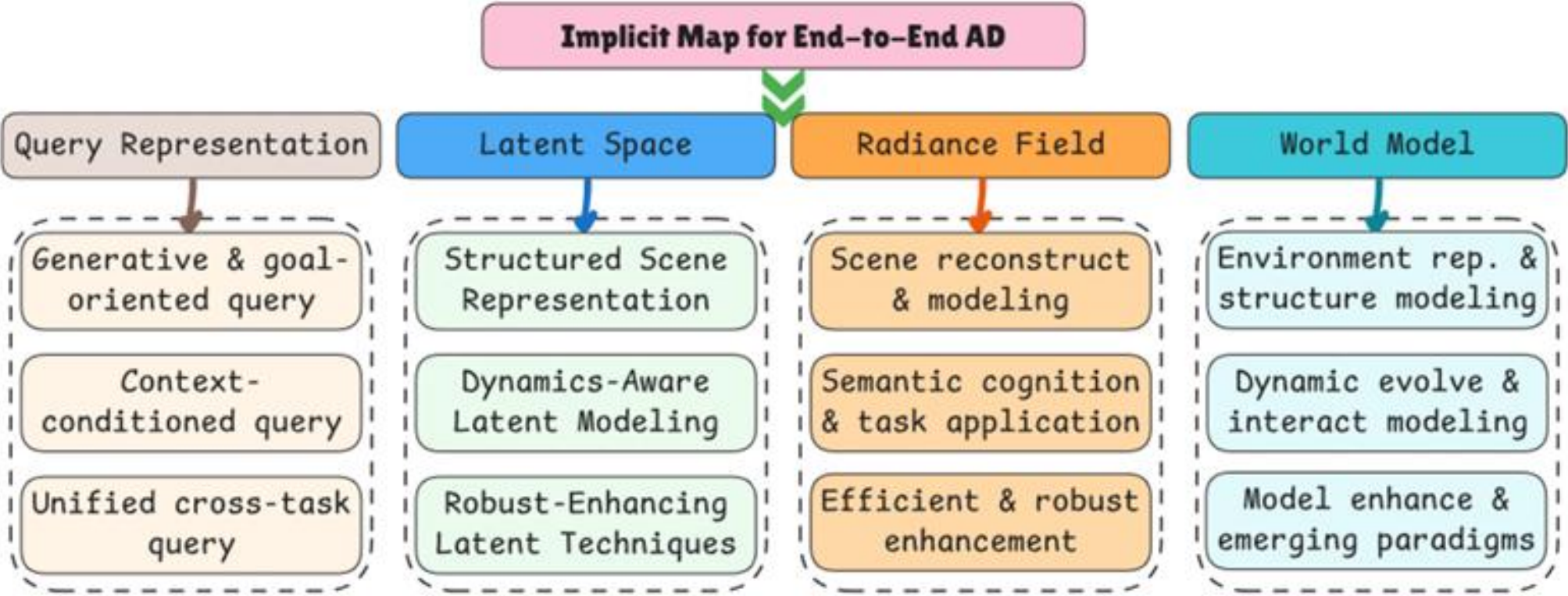
➤ 在线建图典型范式:

▣ MapTR: 将地图元素建模为一个具有一组等价置换的点集



➤ Implicit Map: 隐式地图

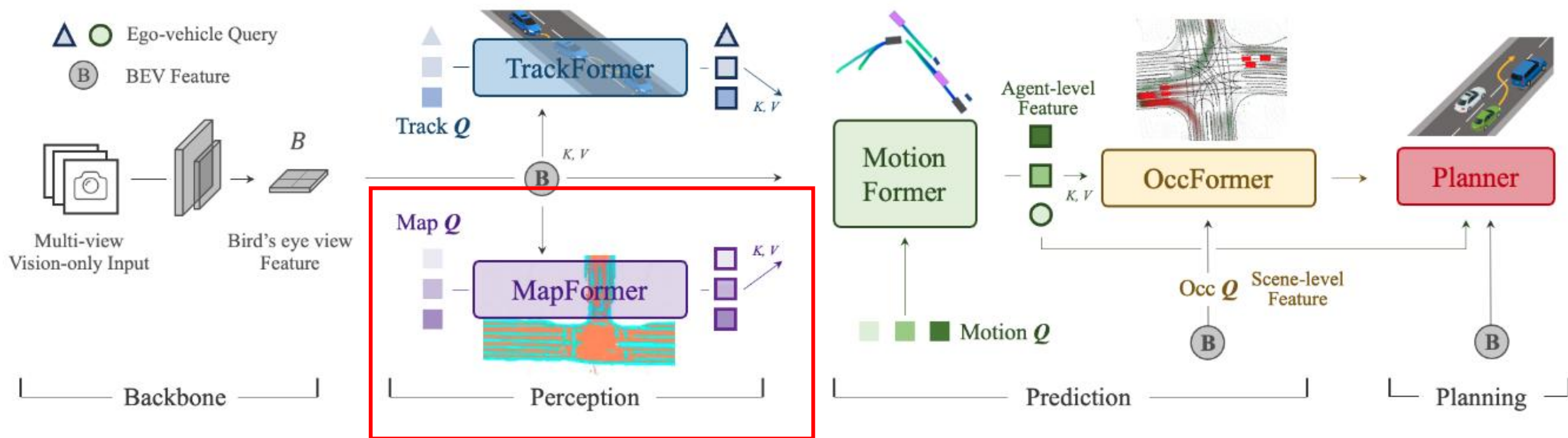
▣ 隐式地图旨在将环境知识直接编码到神经网络参数中，以支持更类人化的端到端自动驾驶系统。



地图要素检测

➤ Implicit Map: 隐式地图

▣ UniAD: Planning-oriented Autonomous Driving



占据网格感知 (Occupancy)

- 占据网格是什么？怎么描述环境空间？
- 打个比方：把我们周围的空间划分为2mx2mx2m的网格，**物质的存在会占据一定的物理空间**，如果这个网格被**某个物质**占据了，那么表达“**被占据**”的状态；

某个物质 -> 占据网格的语义/类别信息
被占据的状态 -> 是否为被占据网格
考虑时间，则还有**运动**信息 -> 占据网格的运动信息（速度）

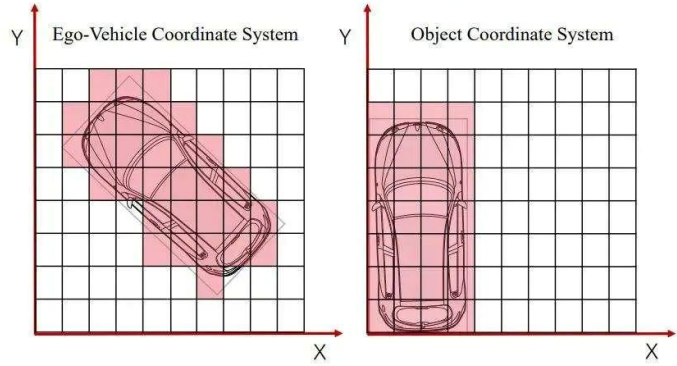
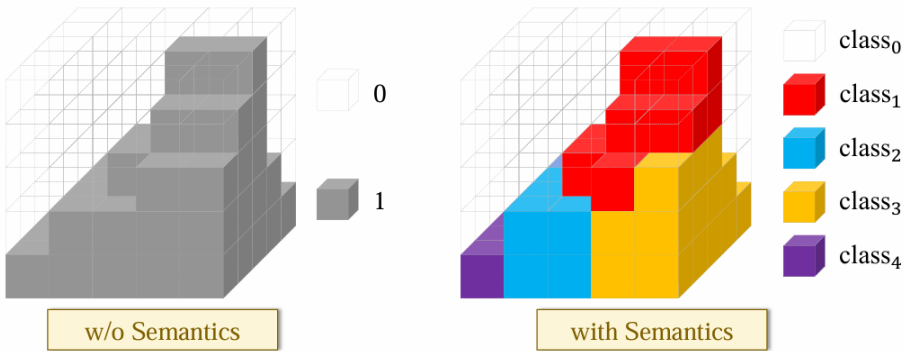


Figure 3: Occupancy grids defined in the ego-vehicle (left) and object-centric (right) coordinate systems. The object shape is jagged in the ego-vehicle occupancy grid due to coordinate misalignment.

Fig. 6. Local-DIF [72]: Local-DIF generates continuous representation for 3D semantic occupancy, which exhibits quantization artefacts on slanted surfaces (e.g. road plane) or edges between objects resulting from a discretization into voxels.



占据网格感知 (Occupancy)

- 网格中心感知指的是给定机载传感器的多模态输入，算法需要将原始信息转换为 BEV或体素网格，并在每个网格单元上**预成型各种感知任务**
- 如何感知建模得到这样的网格？
- $G = f(V_{fusion})$, G denotes Grid.

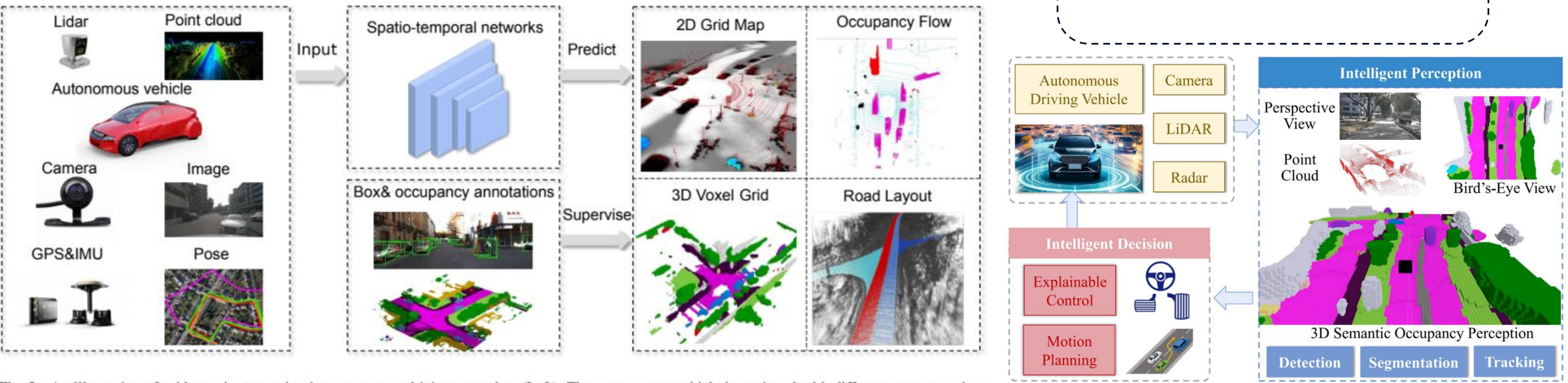


Fig. 2. An illustration of grid-centric perception in autonomous driving scenarios. (Left): The autonomous vehicle is equipped with different sensors such as LiDARs, radars and cameras, as well as GPS and IMU for localization. (Middle): The raw data, point clouds and images are processed in spatio-temporal networks with the supervision from annotations of bounding boxes or occupancy. (Right): Different perception outputs for grid-centric perception. Images source from [9]–[12].

占据网格感知 (Occupancy)

- 为什么要网格建模？相比于3D目标检测有什么区别？
- 放松了对**障碍物形状**的限制，可以描述具有可变形形状的铰接对象；
- 无需**依赖障碍物类别**。它可以准确地描述新类和实例的占用和运动线索，从而增强系统的鲁棒性；

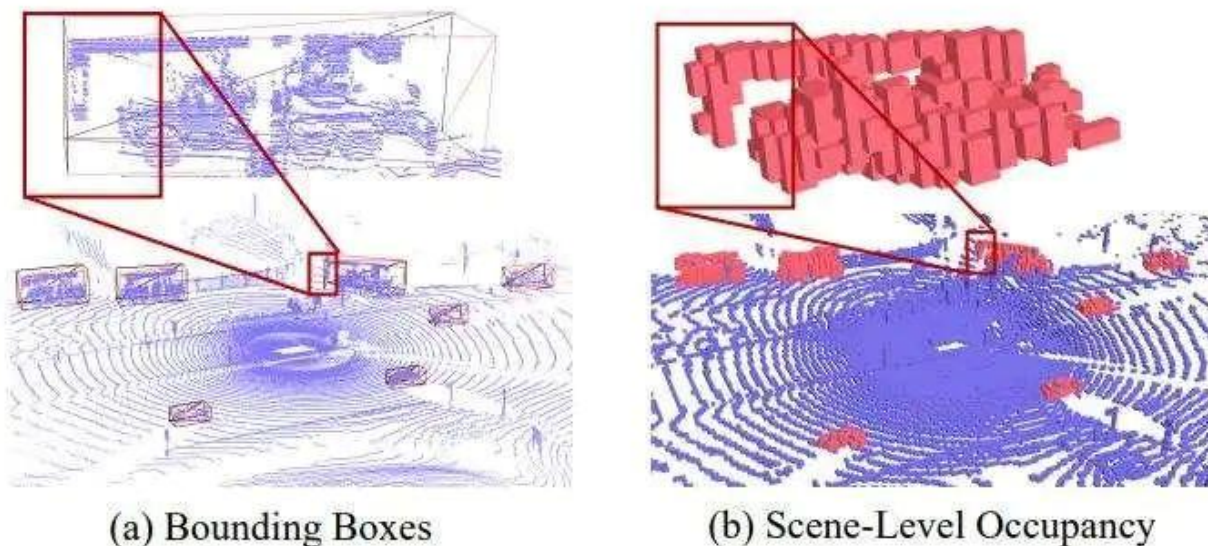
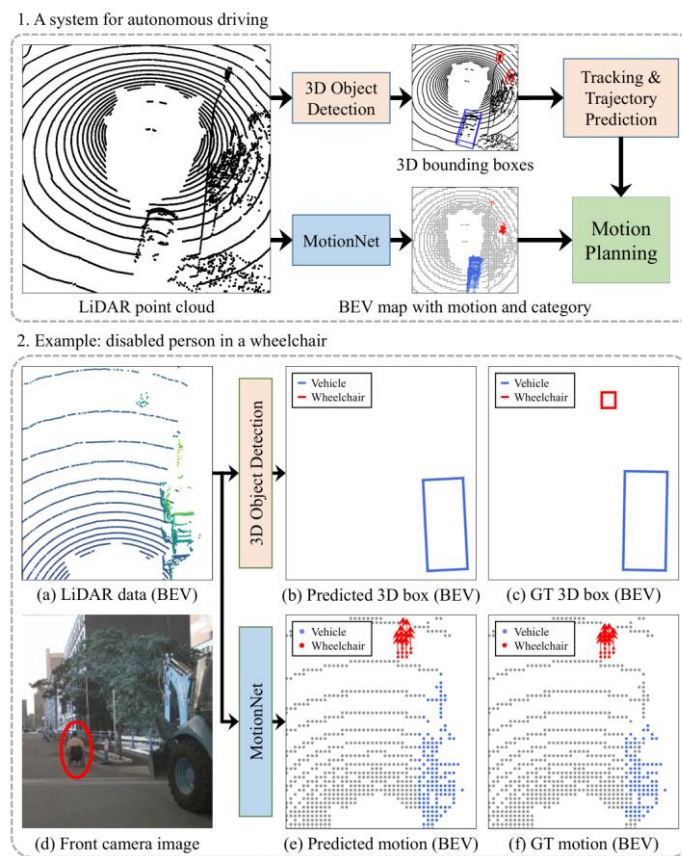


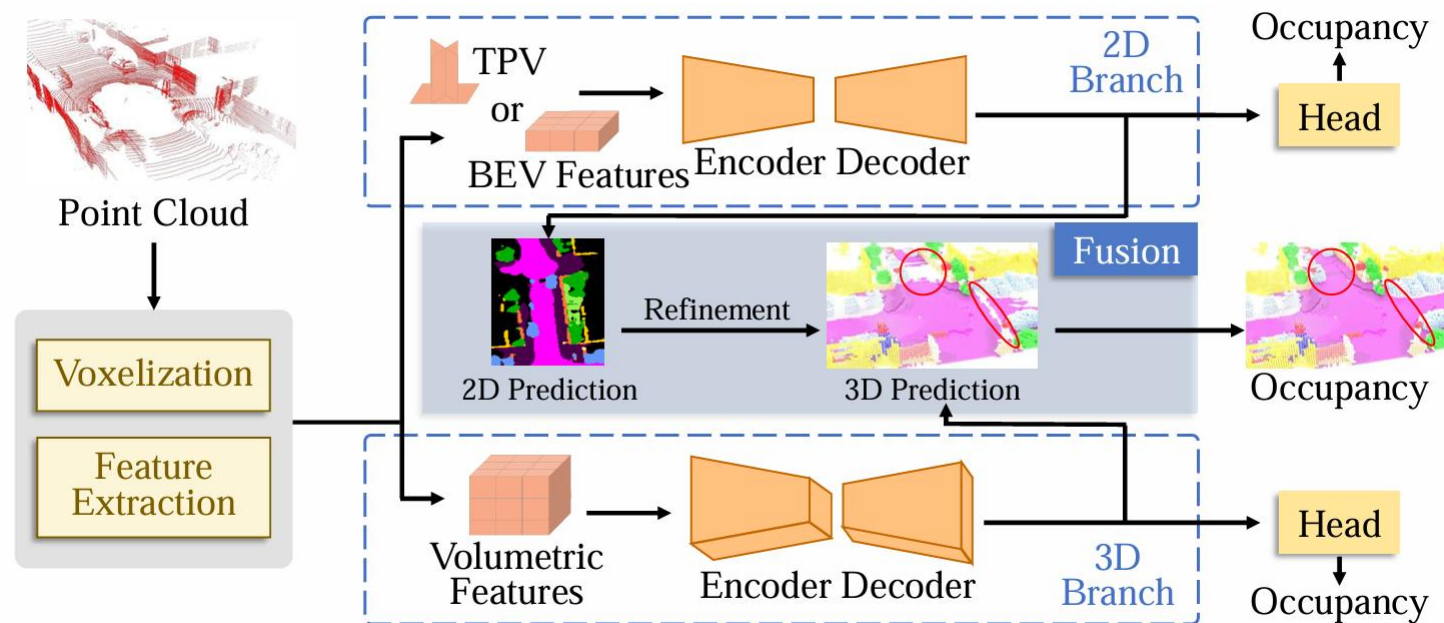
Figure 1: Bounding Box vs. Occupancy. Occupancy can better represent the crane's shape than the bounding box.



占据网格感知 (Occupancy)

➤ 以激光雷达为中心的占据感知：

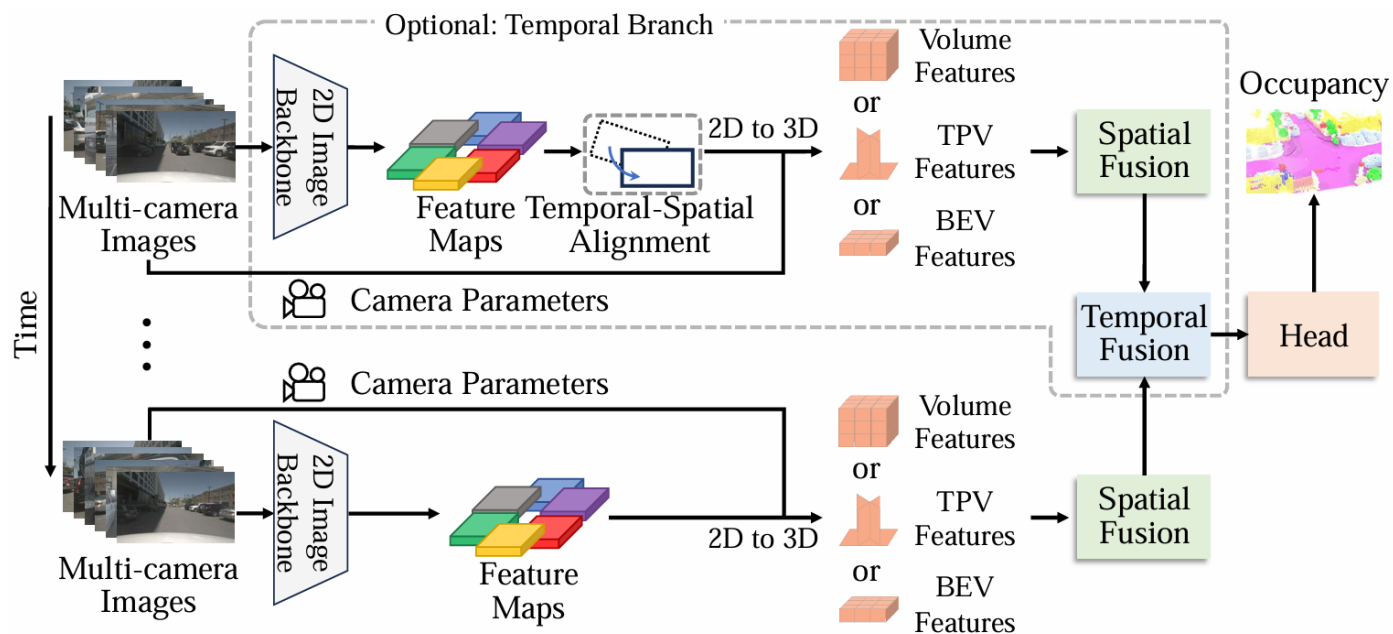
- ❑ 输入点云首先进行体素化和特征提取，然后通过**编解码器**模块增强表示，最终推断出场景的完整密集占据。具体来说，给定一个点云，我们生成一系列初始体素并提取其特征。
- ❑ 编码器和解码器可以是各种特征增强模块。最终的 3D 占据推理涉及对增强后的特征应用卷积或 MLP。
- ❑ 问题：采集的点云本质上具有稀疏性且存在遮挡问题。



占据网格感知 (Occupancy)

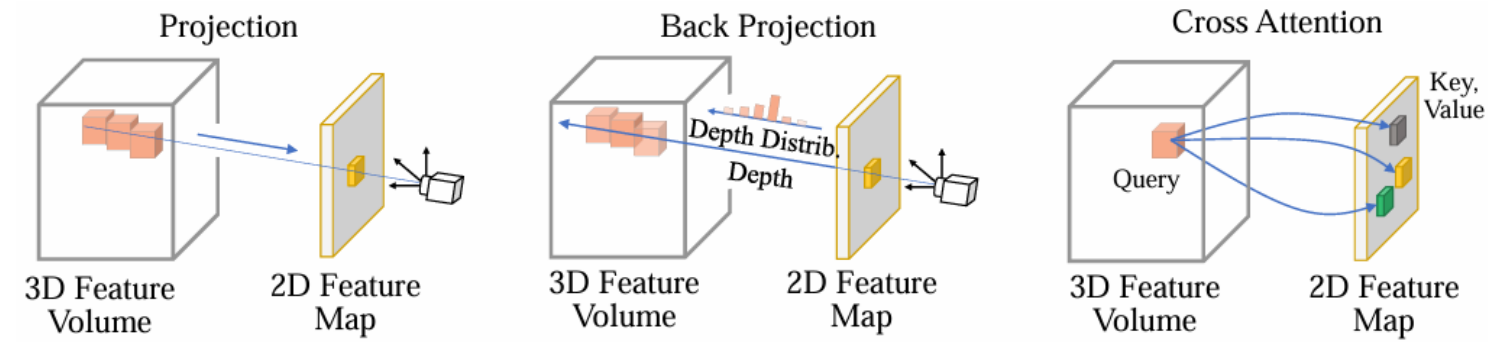
➤ 以视觉为中心的占据感知:

- ❑ 首先从多摄像头图像中提取前视图特征图，然后进行 2D 到 3D 变换、空间信息融合和可选的时间信息融合，最终通过占据头推断环境的 3D 占据。
- ❑ RGB图像的2D特征图是视觉为主的Occupancy流程的基础，一般通过预训练的主干网络（如ResNet, ViT）提取。由于前视图仅提供 2D 视角，因此必须进行 2D 到 3D 变换以推断前视图缺失的深度维度，从而实现 3D 场景感知。



占据网格感知 (Occupancy)

- 2D to 3D: 变换旨在将前视图特征转换为 BEV 特征、TPV 特征或体素特征，以获取前视图缺失的深度维度。值得注意的是，尽管 BEV 特征位于顶视 2D 平面，但其可将高度信息编码到特征的通道维度中，从而表示 3D 场景。三透视视图 (TPV) 将 3D 空间投影到三个正交的 2D 平面，使 3D 空间中的每个特征可表示为三个 TPV 特征的组合。
- 投影：建立从特征体到特征图的几何映射。通过透视投影模型将 3D 空间中的体素质心投影到 2D 前视图特征图上；
- 反投影：反投影是投影的逆过程，同样利用透视投影建立 2D 与 3D 的对应关系。但与投影不同的是，反投影使用每个像素的估计深度计算 2D 到 3D 的精确——映射；
- 交叉注意力



(a) 2D-to-3D transformation. This serves as the fundamental unit for constructing 3D data from 2D observations, typically by projection [26, 31, 38, 53, 91, 119], back projection [55, 80, 81, 82, 90], and cross attention [4, 36, 76, 113, 118].



占据网格感知 (Occupancy)

➤ 时空信息如何提取？

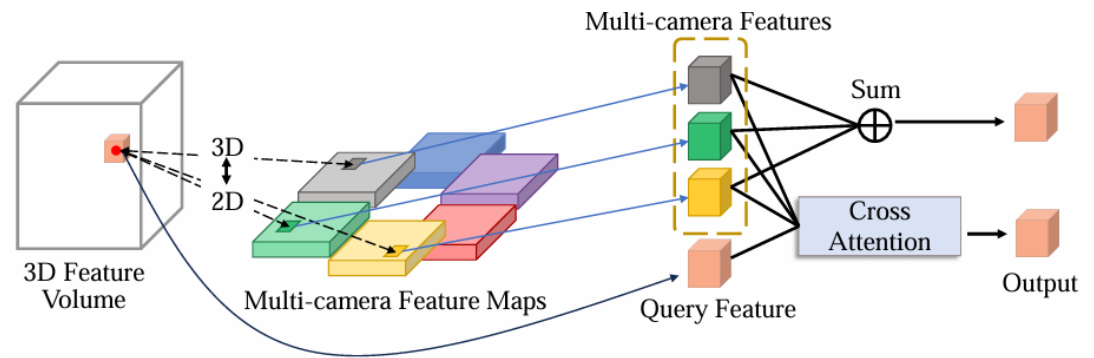
➤ 视觉为中心的占据感知中的信息融合

❑ 多视角聚合：特征体中的一个 3D 体素投影后会对应多个 2D 前视图特征图。融合这些投影命中的 2D 特征有两种方式：平均和交叉注意力

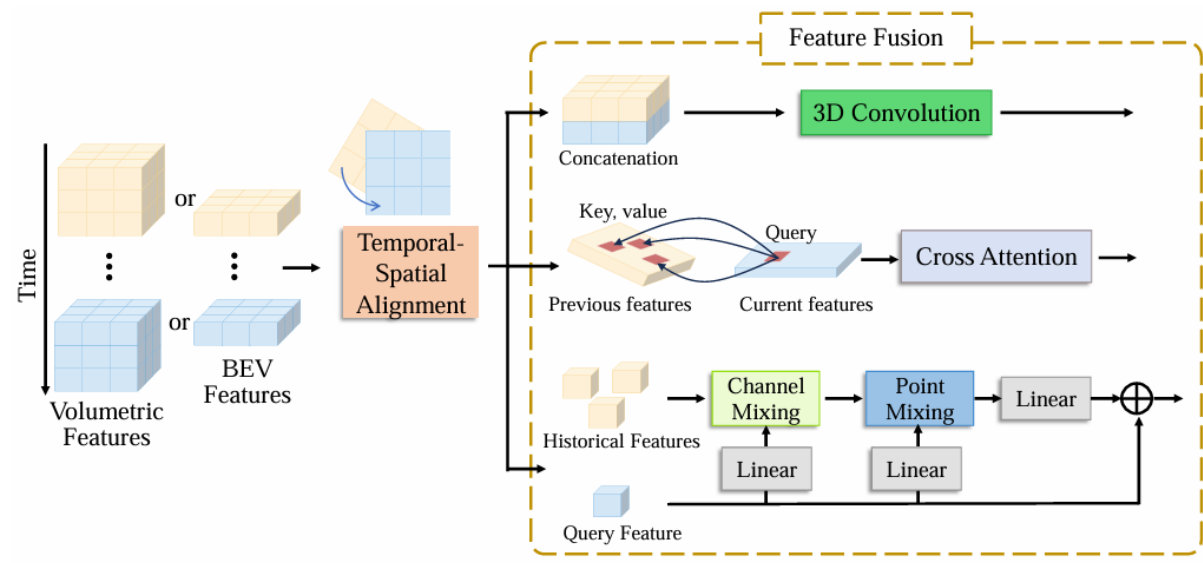
❑ 时序融合：时空对齐利用自车的位姿信息将历史特征与当前特征进行空间对齐。对齐完成后，历史特征和当前特征被输入特征融合模块以增强表示，尤其是增强对遮挡的推理能力和对运动目标的识别能力。

❑ 特征融合主要有三种方式：卷积、交叉注意力和自适应混合。

❑ 解码器：通过空间和时间信息融合得到的特征会由各种类型的头部处理，以确定 3D 占据情况。这些头部包括卷积头、掩码解码器头、线性投影头以及带阈值的线性投影头。



(b) Spatial information fusion. In areas where multiple cameras have overlapping fields of view, features from these cameras are fused through average [38 53 82] or cross attention [4 32 76 113 120].



(c) Temporal information fusion. Historical and current features undergo the temporal-spatial alignment, and then are fused via convolution [4] (see row 1 in the feature fusion block), cross attention [8 33 56 120] (row 2), and adaptive mixing [118] (row 3).



占据网格感知 (Occupancy)

- 相机捕捉的 RGB 图像提供了丰富密集的语义信息，但对天气变化敏感且缺乏精确的几何细节。相比之下，激光雷达或毫米波雷达的点云对天气变化鲁棒，擅长捕捉具有精确深度测量的场景几何，但仅产生稀疏特征。多模态占据感知可以结合多种模态的优势，弥补单模态感知的局限性。
- 大多数多模态方法将 2D 图像特征映射到 3D 空间，然后与点云特征融合。此外，在融合过程中加入 2D 透视视图特征可以进一步优化表示。融合后的表示经过可选的优化模块和占据头（如 3D 卷积或 MLP）处理，生成最终的 3D 占据预测。

三种主要的多模态信息融合技术用于集成不同模态分支：拼接（Concatenation）、求和（Summation）和交叉注意力（Cross Attention）

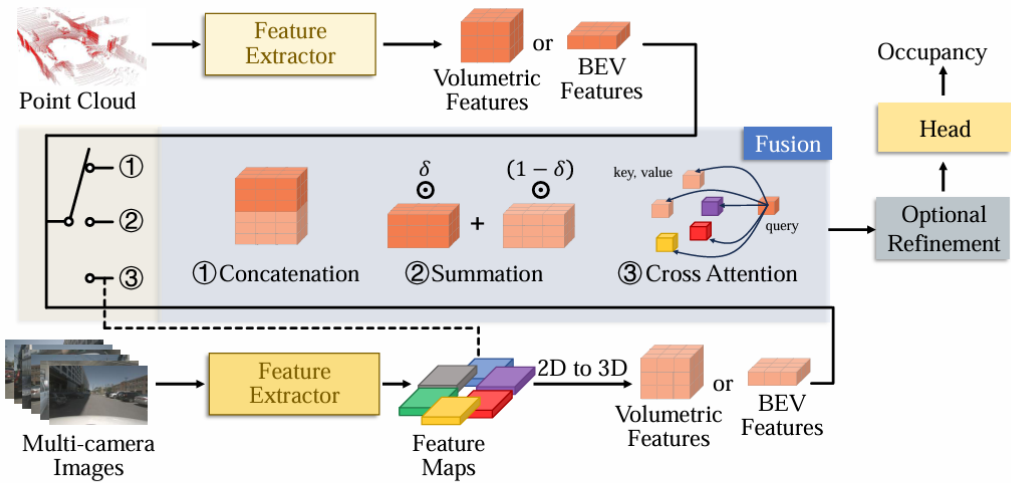
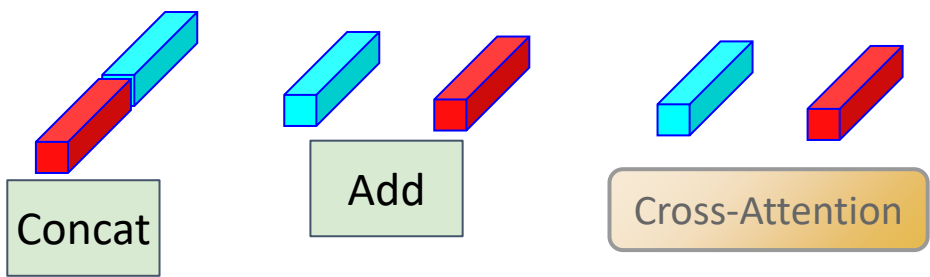


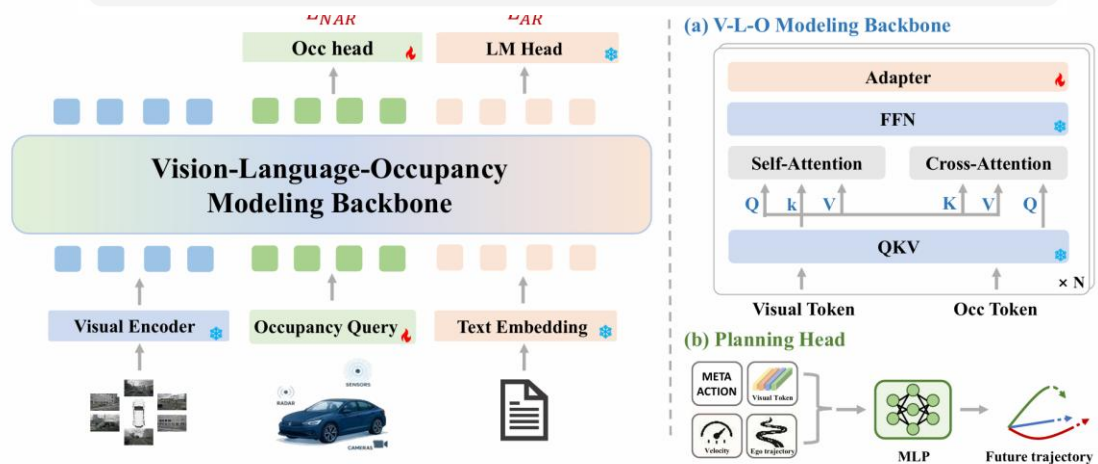
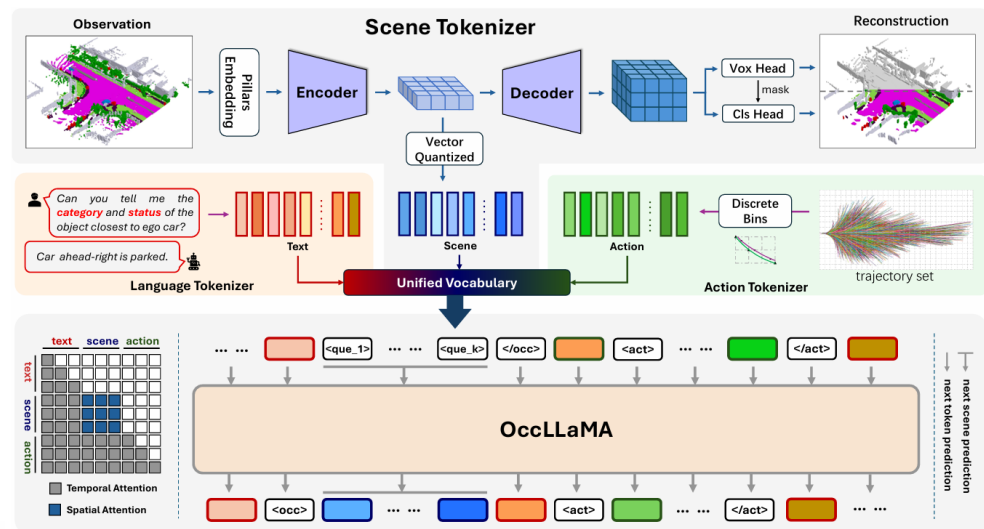
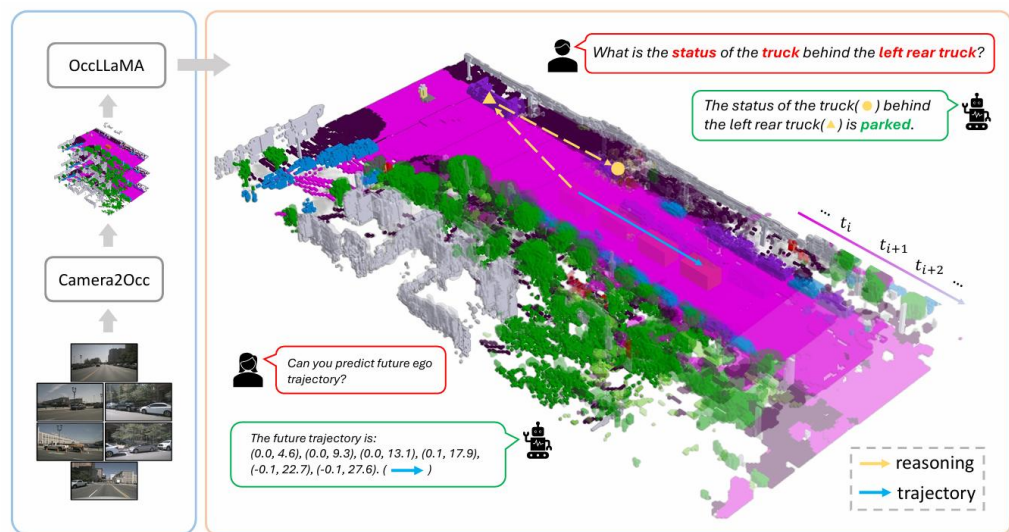
Figure 7: **Architecture for multi-modal occupancy perception:** Fusion of information from point clouds and images [11, 12, 15, 103, 100]. Dashed lines signify additional fusion of perspective-view feature maps [14]. $\odot$ represents element-wise product. δ is a learnable weight.



占据网格感知 (Occupancy)

➤ Occllama/OccVLA: 考考这个框架里用到了哪些占据网格的地方?

➤ 为什么Occ在VLA系统也重要: Occ的表征可以弥补语言模型3D理解能力, 作为严格的安全兜底

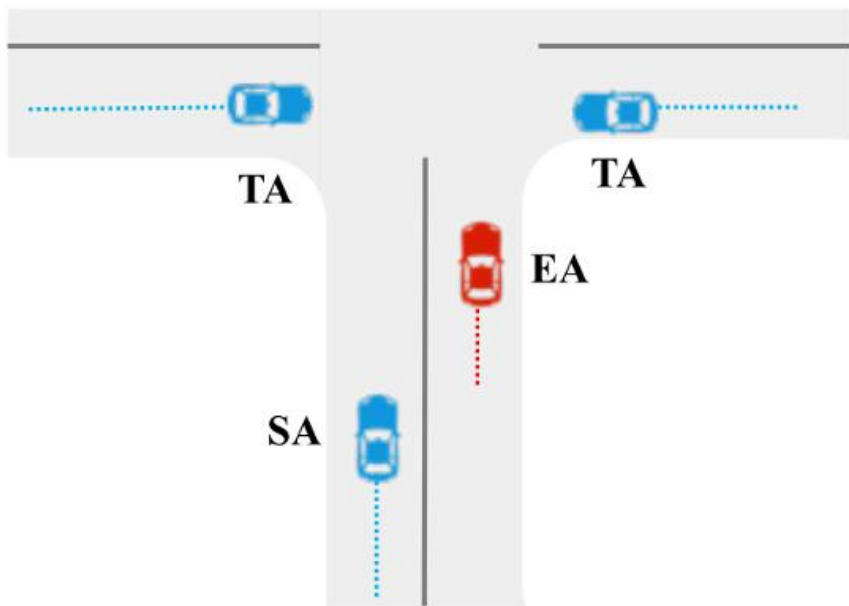


自动驾驶之心



轨迹预测 (Prediction)

➤ 问题定义



目标主体 (TAs)：需预测其未来行为的核心对象

自车主体 (EA)：自动驾驶汽车本身，其行为受周围环境影响

周围主体 (SAs)：可能影响TA行为的其他交通参与者



轨迹预测 (Prediction)

➤ 输入表示

轨迹预测的输入类型包括:

✓ Scenario Representation

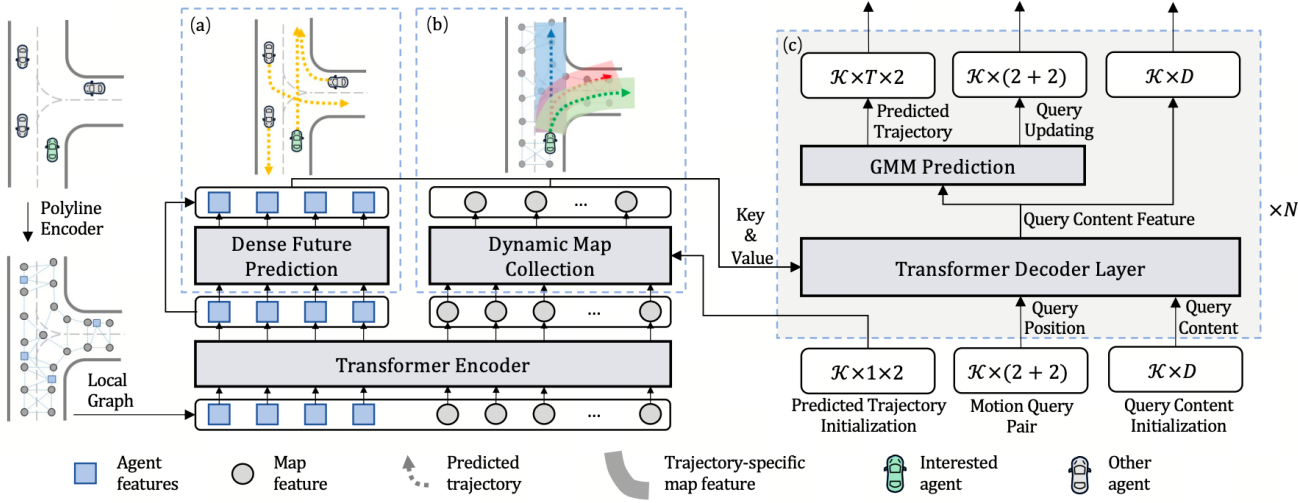
轨迹与高精地图数据: 结构化位置和车道信息

✓ Bird's Eye View (BEV) representation

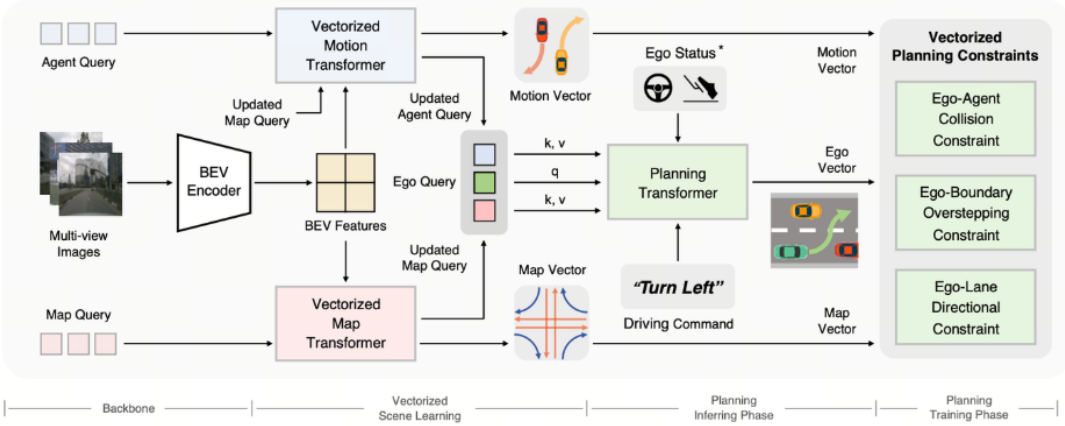
将传感器数据转换为2D网格, 简化空间关系建模

✓ raw perception data

原始感知数据: 激光雷达点云、相机图像、雷达数据等。



MTR (场景表征)



VAD
(BEV表征)



轨迹预测 (Prediction)

➤ 输出表示

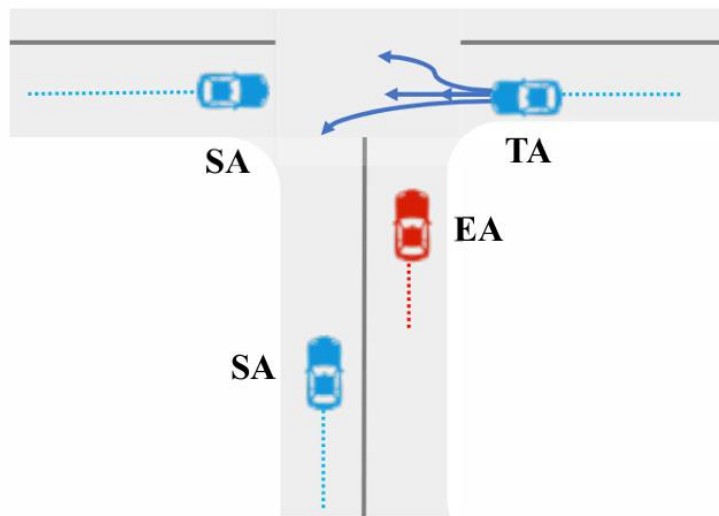
轨迹预测的输出类型包括:

- ✓ Marginal Trajectory Prediction: TA的轨迹不会影响到其他agent, 将每个TA的轨迹分布视作独立的, 如下:

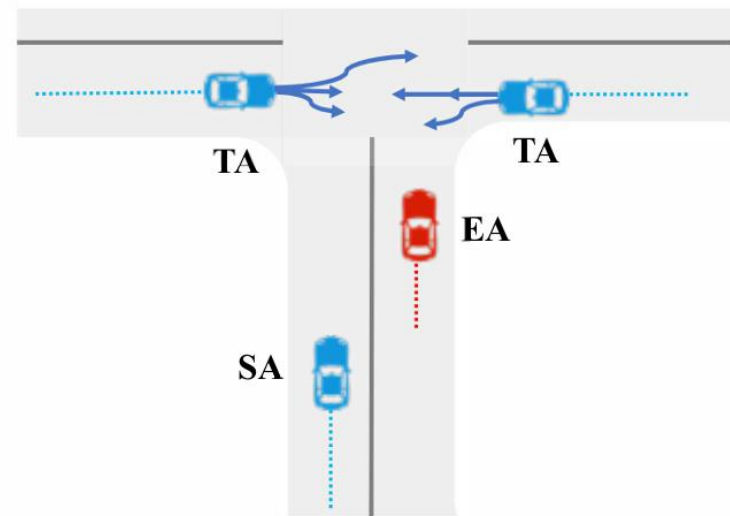
$$Output_Marginal_{T_{pred}} = p(s_1)p(s_2)...p(s_n),$$

- ✓ 联合多智能体预测: 考虑这一场景下所有TA的未来轨迹的交互作用, 将其视为一个整体进行预测, 其目标函数如下:

$$Output_Joint_{T_{pred}} = p(s_1, s_2, ..., s_n),$$



(a) Marginal Prediction



(b) Joint Prediction



轨迹预测 (Prediction)

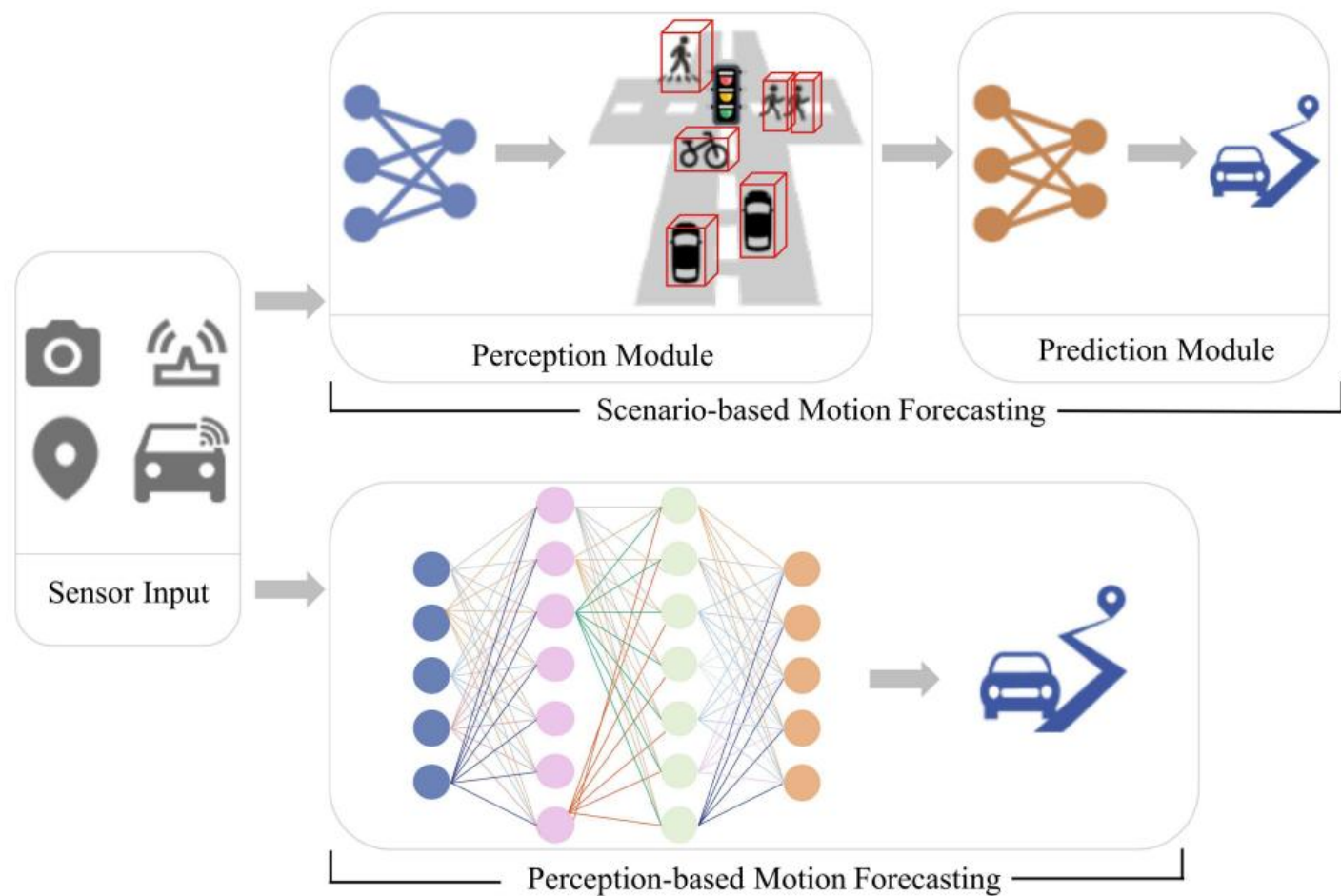
- 准确预测众多交通参与者 (agent) 的未来行为对自动驾驶决策来说十分重要。如何进行轨迹预测？

• 基于场景的预测方法

根据周围agent的历史状态以及环境信息 (如HDmap) 来预测agent的未来状态, 这种方法是一种结构化的方法, 摒弃掉如图像、点云等传感器数据, 更集中于分析交通环境以及agent之间的交互。

• 基于感知的预测方法

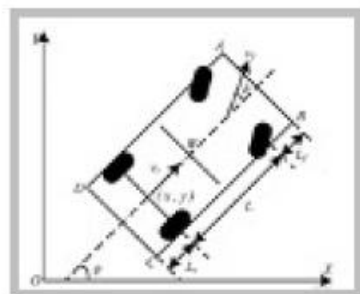
直接来自于传感器的感知数据, 直接预测agent未来的轨迹, 这种方法能更好地利用原始数据, 以理解更复杂的场景。



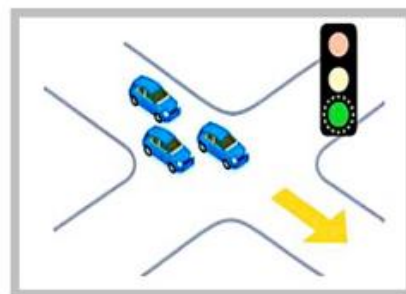
轨迹预测 (Prediction)

- 从发展历史来看，自动驾驶领域的轨迹预测问题经历了三个阶段。即**基于物理动力学模型**，**基于规则模型**，和**基于学习模型**。

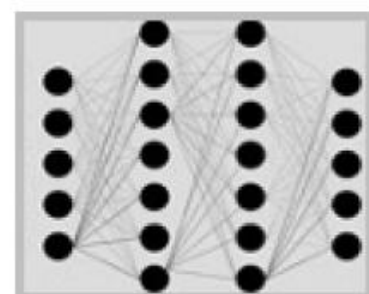
Methods Development



Physics-based



Rule-based



Deep Learning-based

• 基于物理动力学

原理：通过物理规则来预测短期的轨迹，通过车辆位置、加速度，航向角等信息来估计未来的行动。

缺点：但是这种方法忽视了环境因素以及agent之间的交互，无法适应复杂环境。

• 基于规则

原理：基于交通规则以及人类的先验知识，以一种固定的方式预测轨迹。

缺点：这种方法缺乏泛化能力，难以应对复杂场景下的非线性交互。

• 基于学习

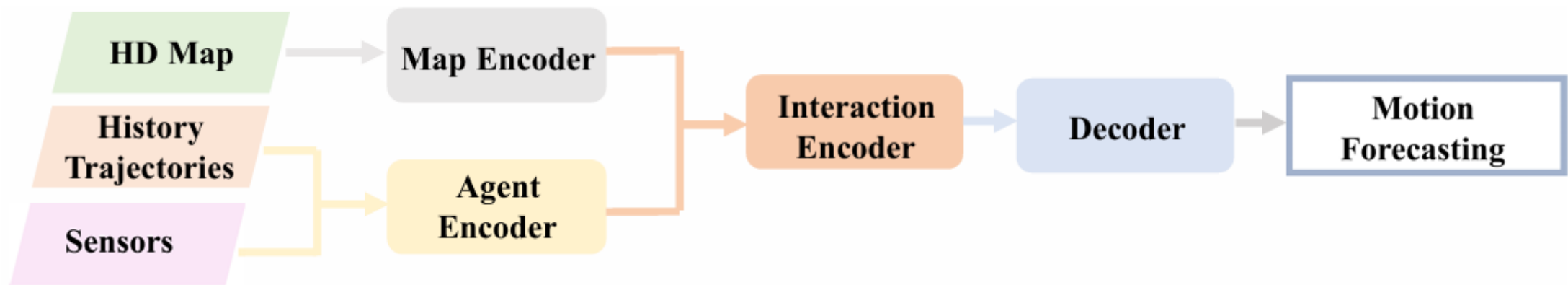
原理：通过RNN graph-based transformer等网络结构来进行轨迹预测，在分析时空数据的复杂性上的表现十分优秀。

缺点：但这类算法需要大量高精度的标注数据



轨迹预测 (Prediction)

➤ 基于监督学习的网络架构分类:



SL模型的架构

Encoder

栅格化→矢量化→attention

Decoder

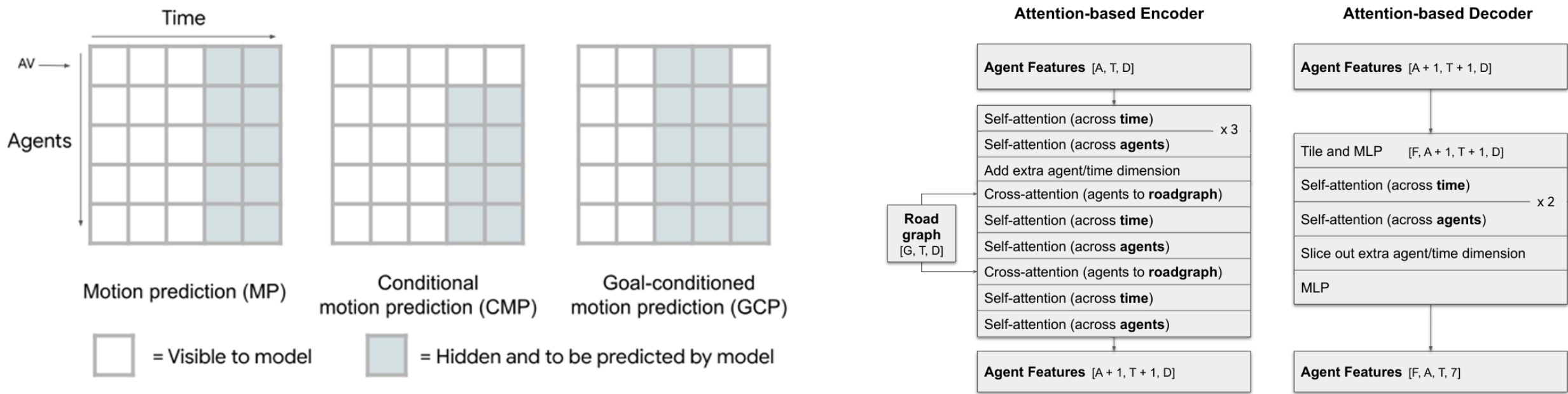
Anchor-Conditioned Decoding
Anchor-Free Decoding



轨迹预测 (Prediction)

➤ 无锚点的解码器设计

无锚点的解码方法不依赖于预定义的锚点，而是直接从解码器输出预测轨迹。这种方法避免了锚点带来的空间先验信息限制，但可能导致模型倾向于学习高频模式，而对低频模式的学习不足，从而在长期预测任务中精度下降。



SceneTransformer

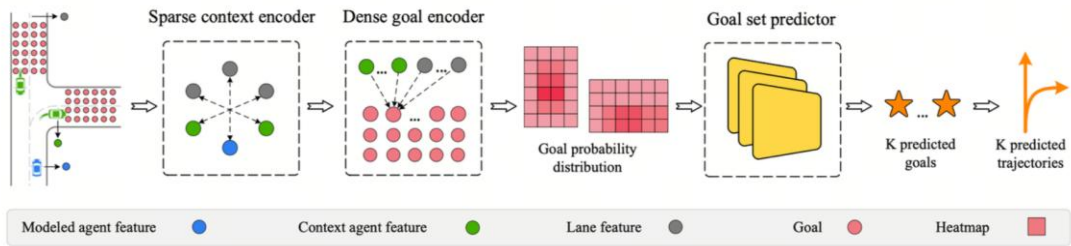


轨迹预测 (Prediction)

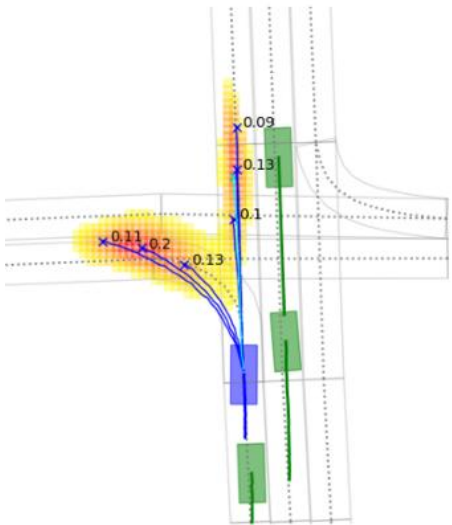
➤ 基于锚点的解码器设计

基于锚点的解码方法通常将数据集中的先验知识作为网络输入的一部分，通过条件概率生成多模态轨迹。这种方法依赖于预定义的锚点 (anchors)，最终输出的轨迹会被限制在这些锚点定义的集合内。其有效性高度依赖于锚点的质量和相关性。基于锚点的解码方法进一步细分为以下几种：

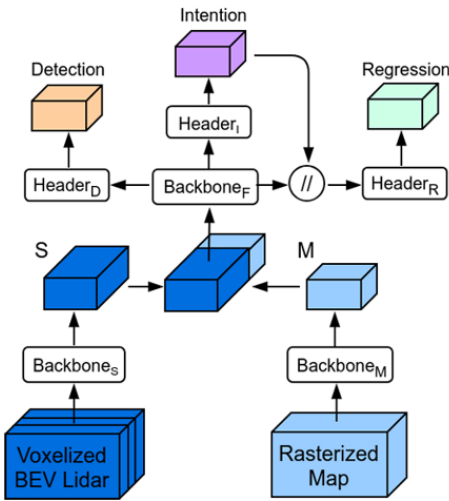
- ✓ Goal-based Decoder (基于目标的解码器) :基于目标的解码器假设轨迹的终点包含了大部分不确定性，因此首先预测交通参与者的终点目标，然后基于该目标生成完整的轨迹。
- ✓ Heatmap-based Decoder (基于热图的解码器) :基于热图的解码器将轨迹预测的输出格式定义为概率热图，通过全卷积模型生成热图来表示交通参与者未来位置的可能性分布。
- ✓ Intention-based Decoder (基于意图的解码器) : 基于意图的解码器通过手动定义交通参与者的意图，并为每种意图学习独立的运动预测器。



DenseTNT



Home



IntentNet



轨迹预测 (Prediction)

➤ 如何设计锚点

大数据聚类+自我迭代

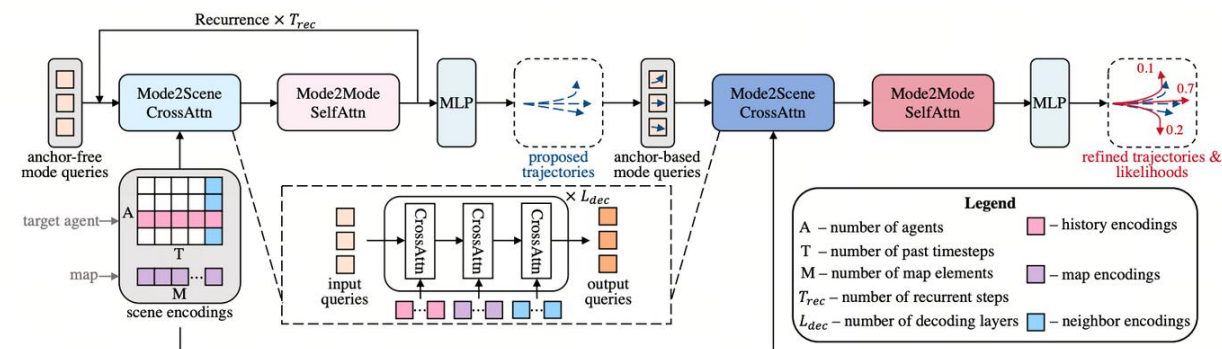
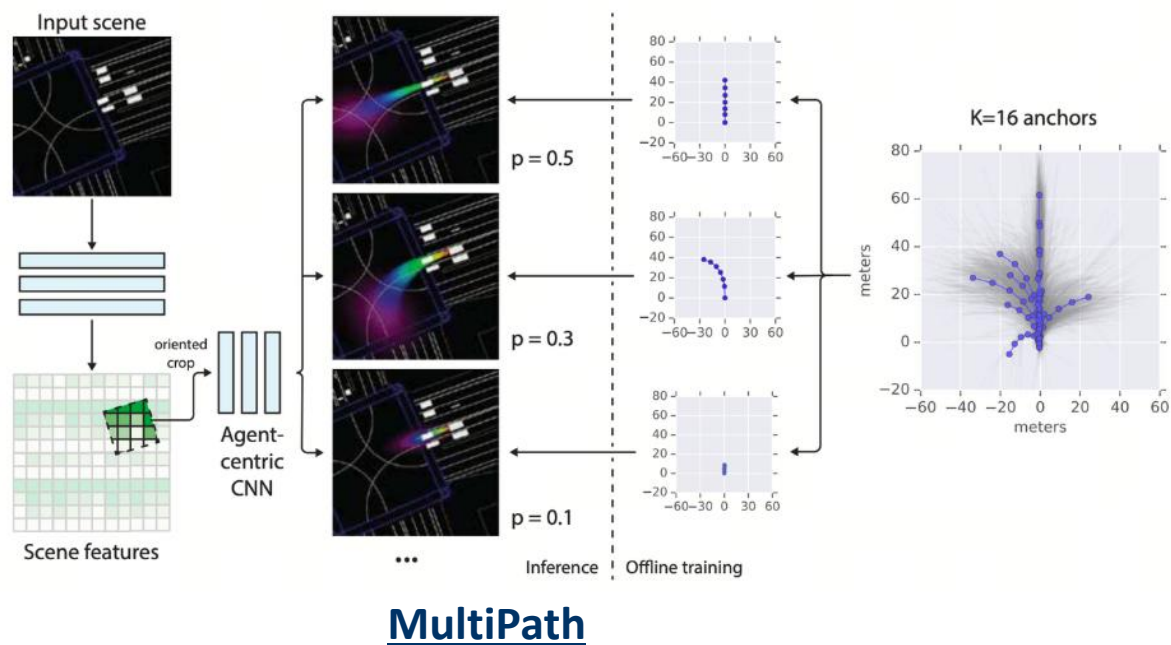
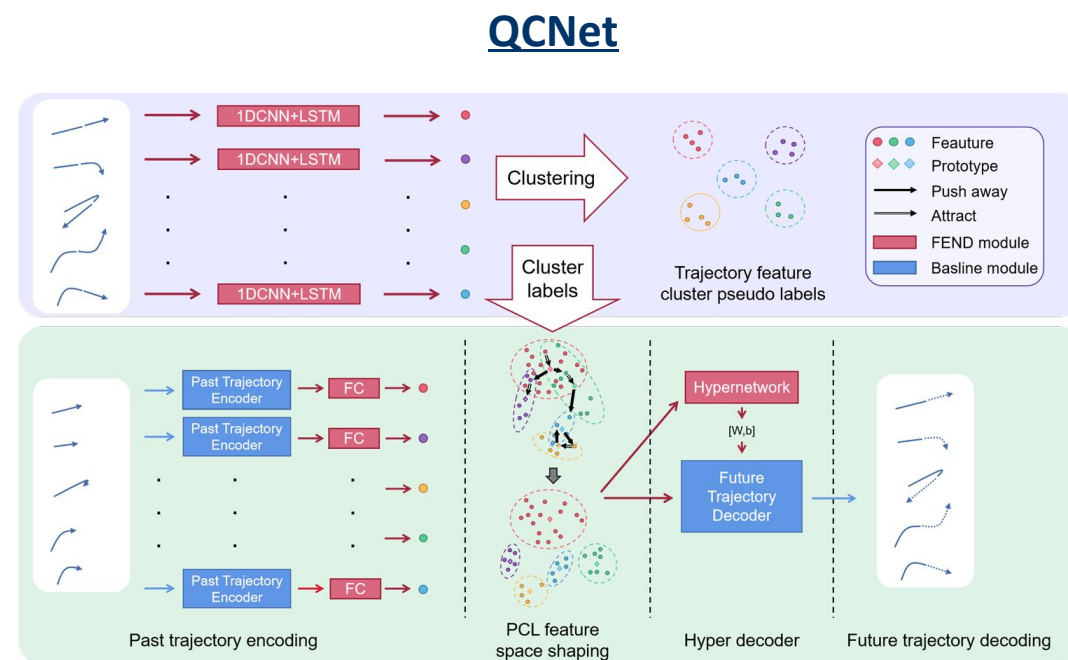


Figure 3. Overview of the **decoding pipeline**. An anchor-free module generates trajectory proposals *recurrently* based on the encoded scene context. These proposals act as the anchors in the refinement module, where an anchor-based decoder refines the anchor trajectories and assigns a probability score for each hypothesis.



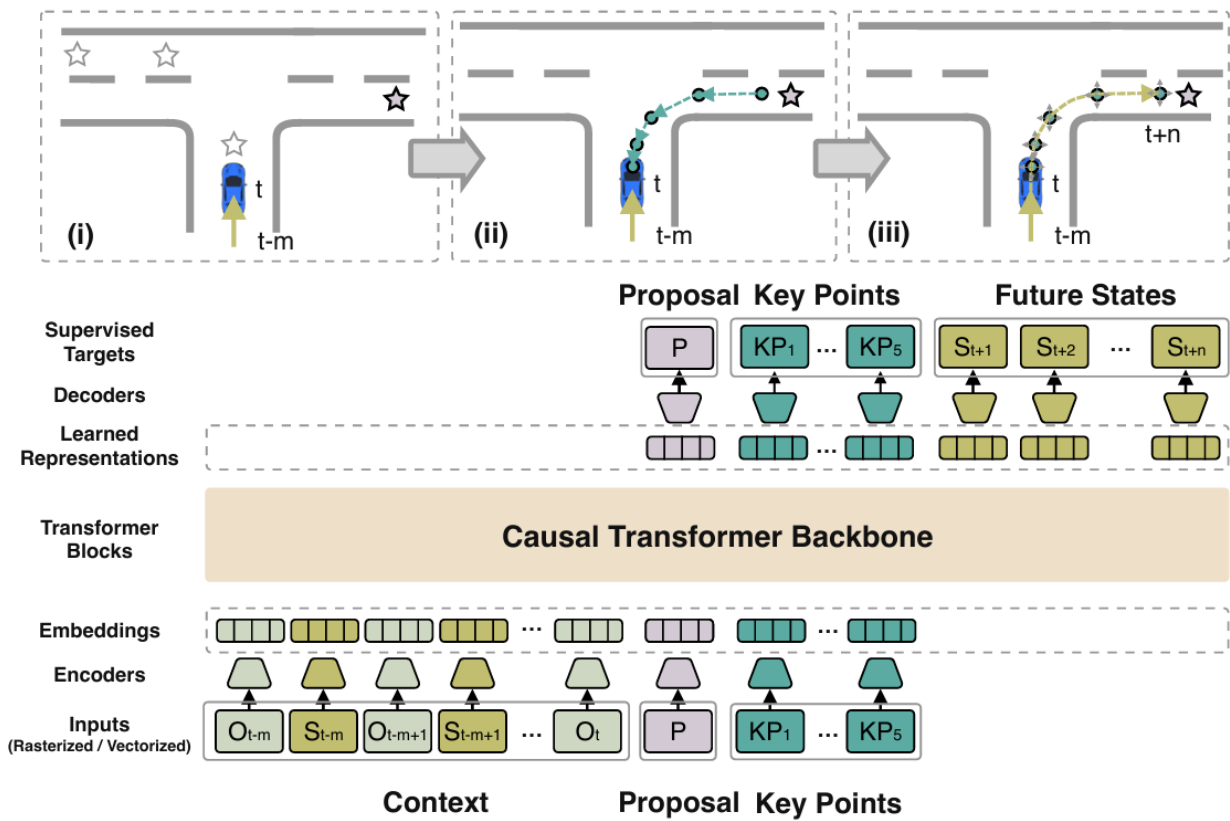
FEND: A Future Enhanced Distribution-Aware Contrastive Learning Framework for Long-tail Trajectory Prediction



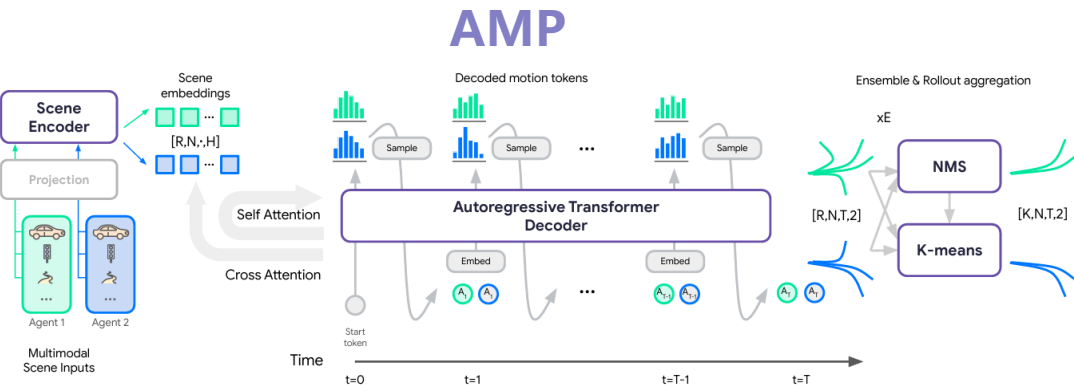
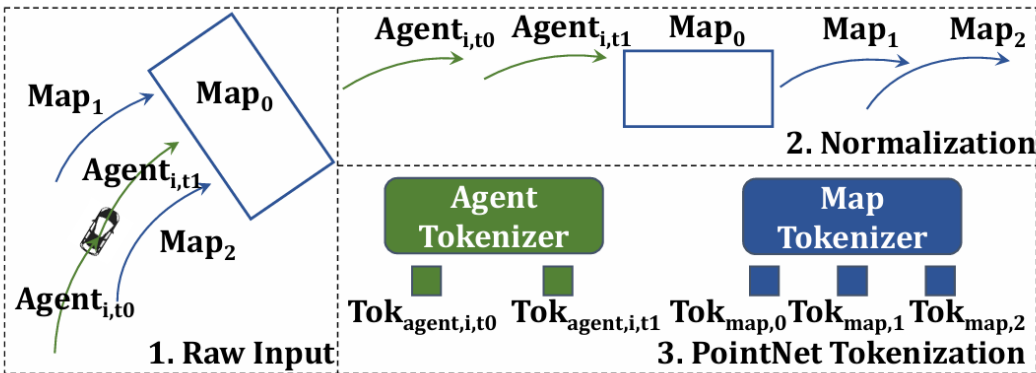
轨迹预测 (Prediction)

➤ 基于语言的轨迹预测建模方法

➤ 为什么一些工作尝试用语言来解预测问题：语言模型强大的逻辑推理能力可以更好的理解复杂交通场景与行为



STR



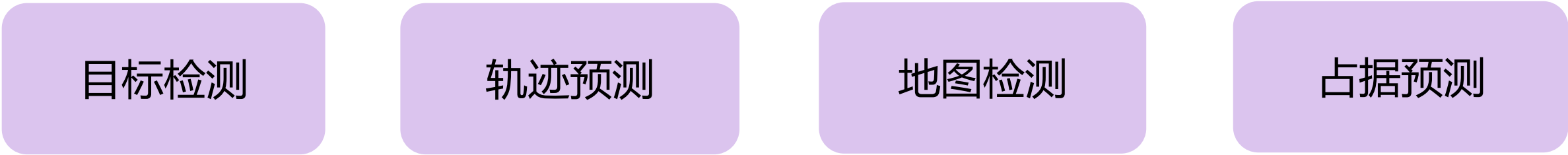
MotionLM



视觉感知相关总结

➤ 自动驾驶环境感知任务和网络设计

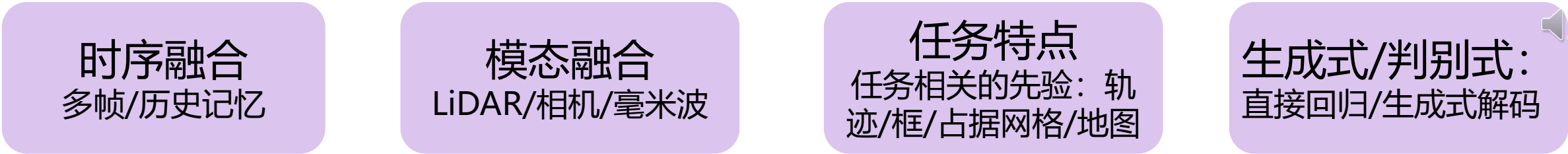
➤ 基本任务



➤ 基本架构

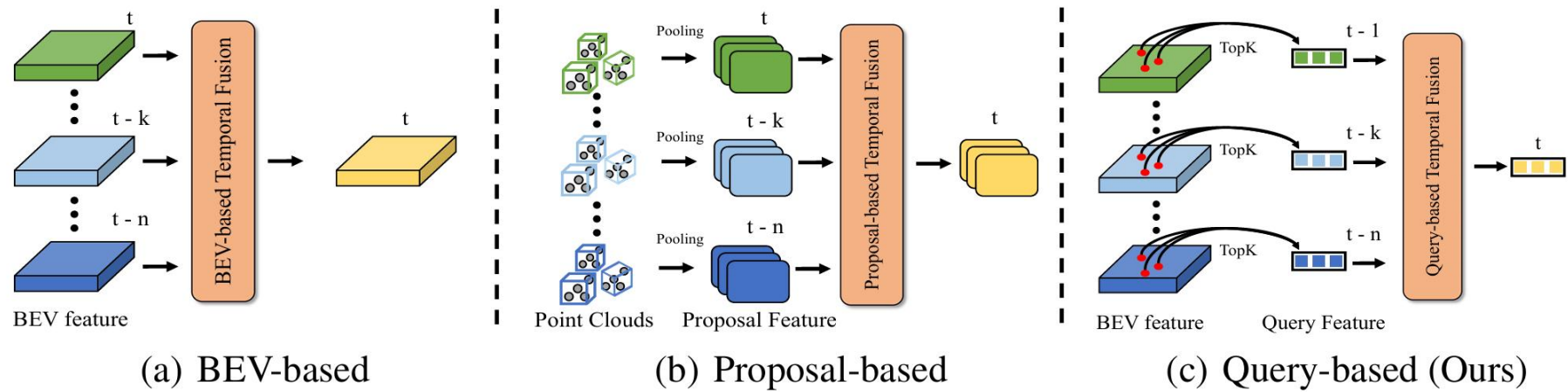


➤ 基本设计（融合+先验注入）



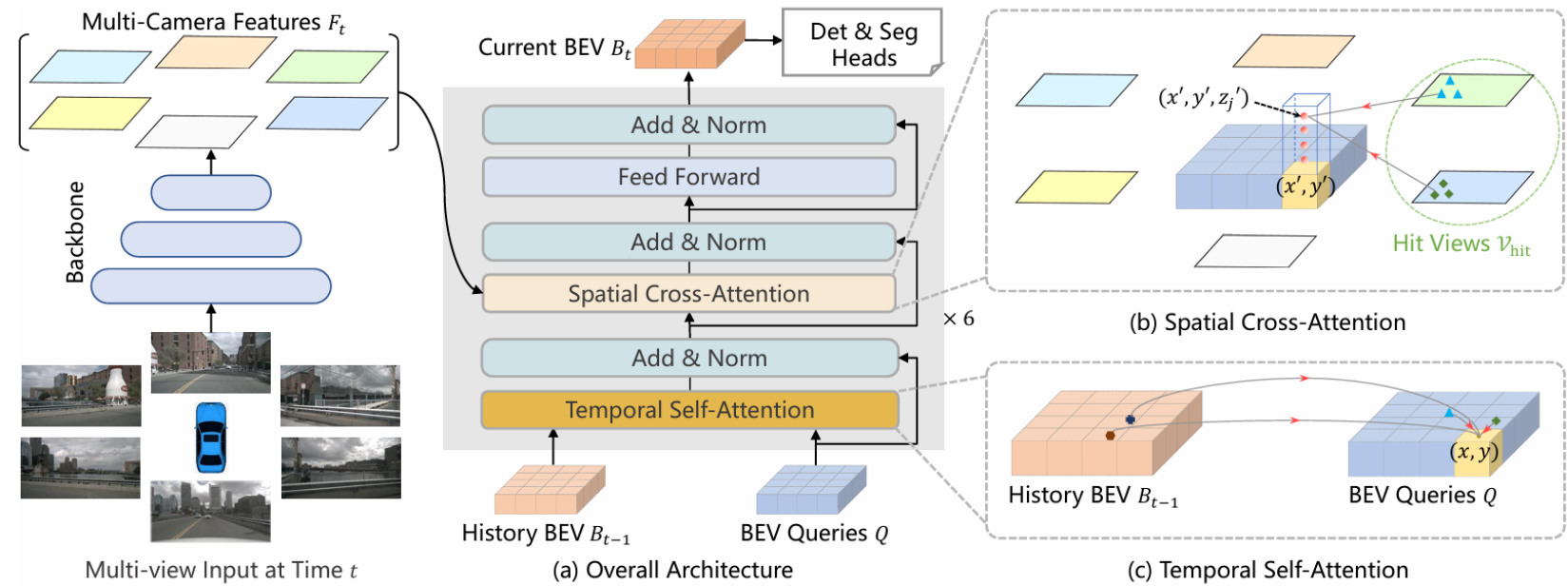
视觉感知相关总结——时序融合

➤ 数据表征



[QTNet: 最新时序融合新方案！点云、图像、多模态检测器全适用（NeurIPS 2023） - 知乎](#)

➤ 融合方式



[fundamentalvision/BEVFormer: \[ECCV 2022\] This is the official implementation of BEVFormer, a camera-only framework for autonomous driving perception, e.g., 3D object detection and semantic map segmentation.](#)



视觉感知相关总结——模态融合

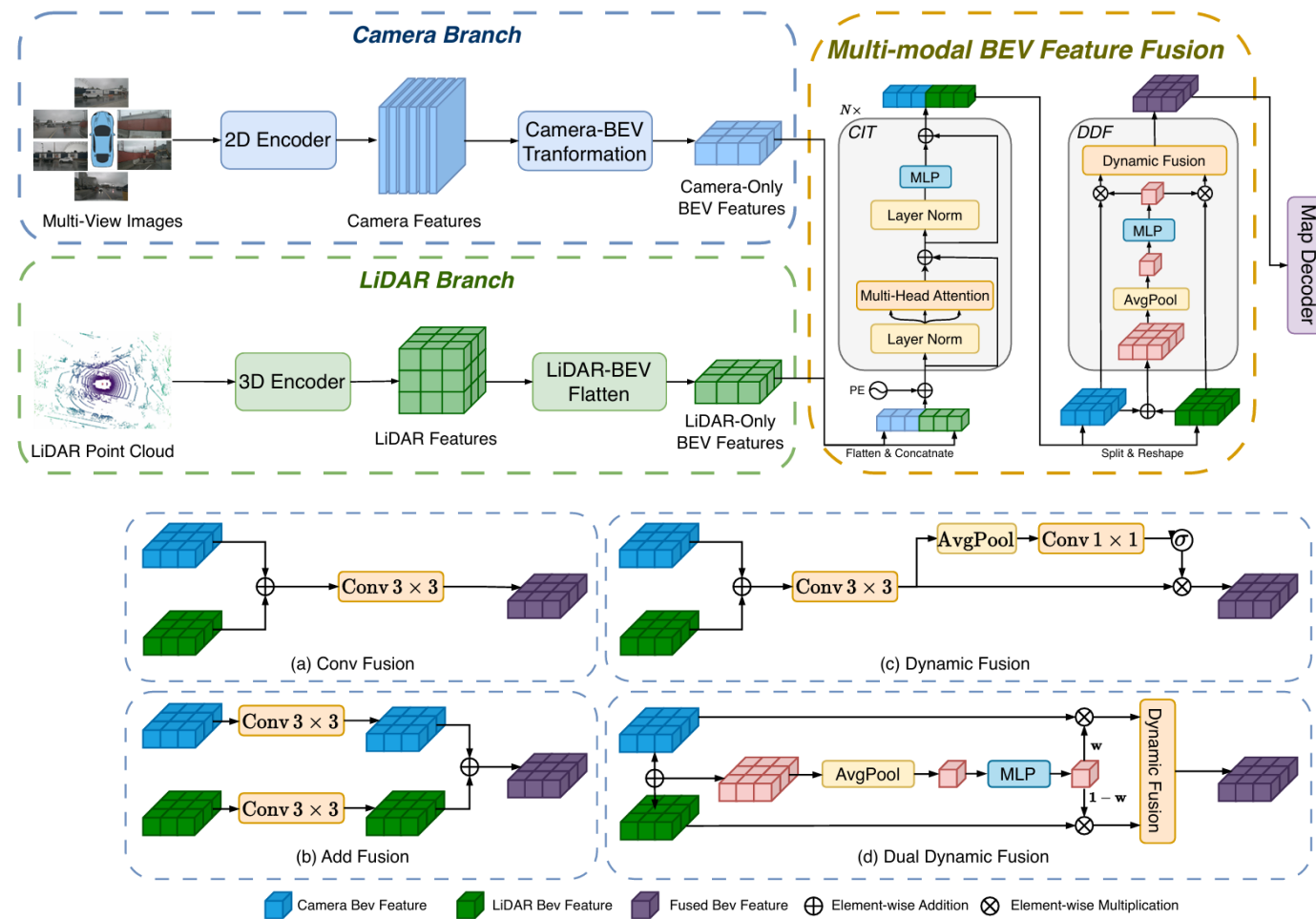
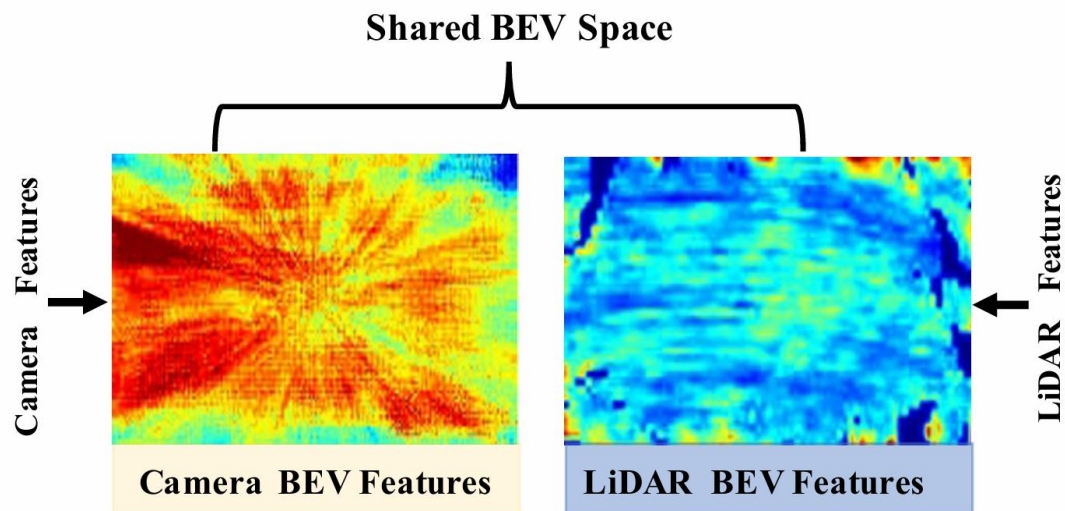
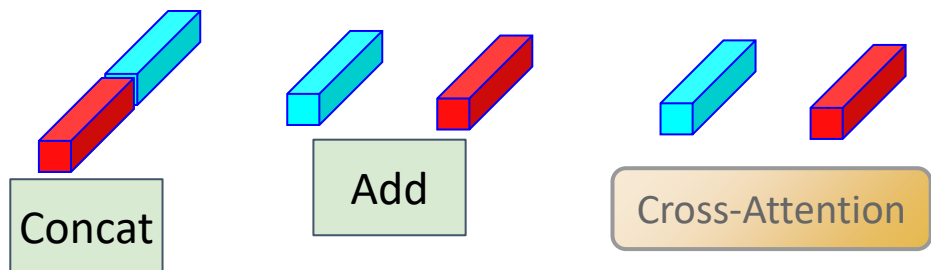
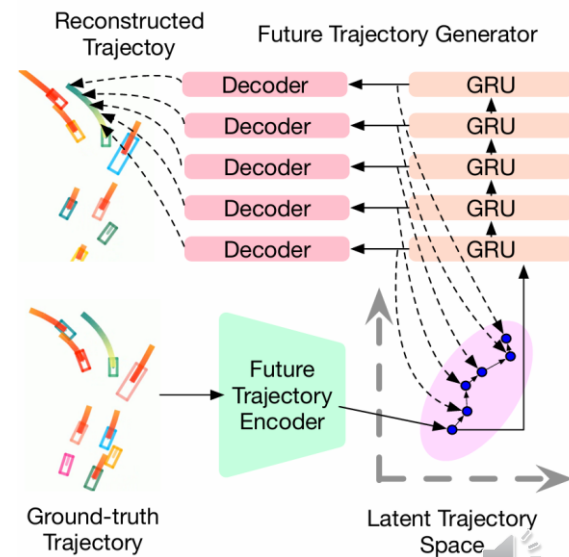
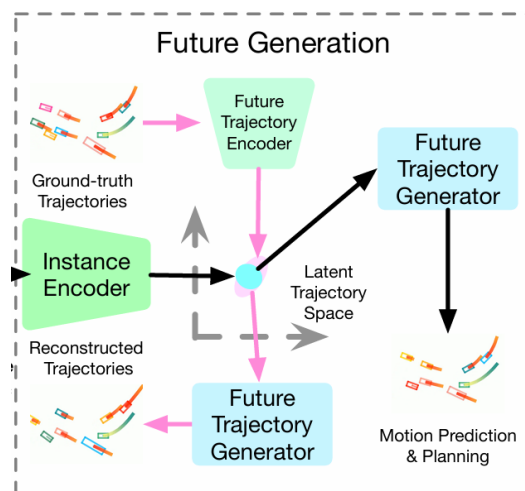
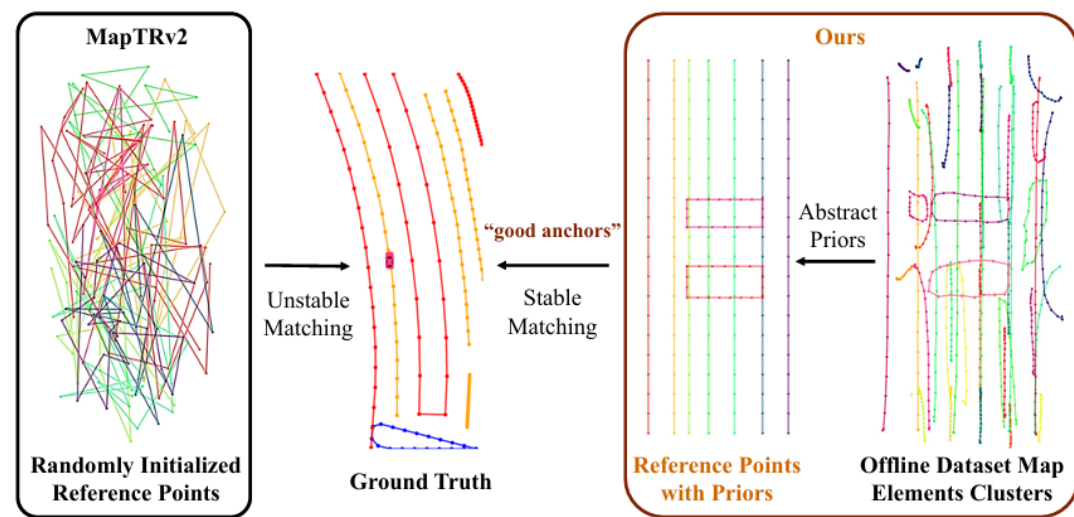
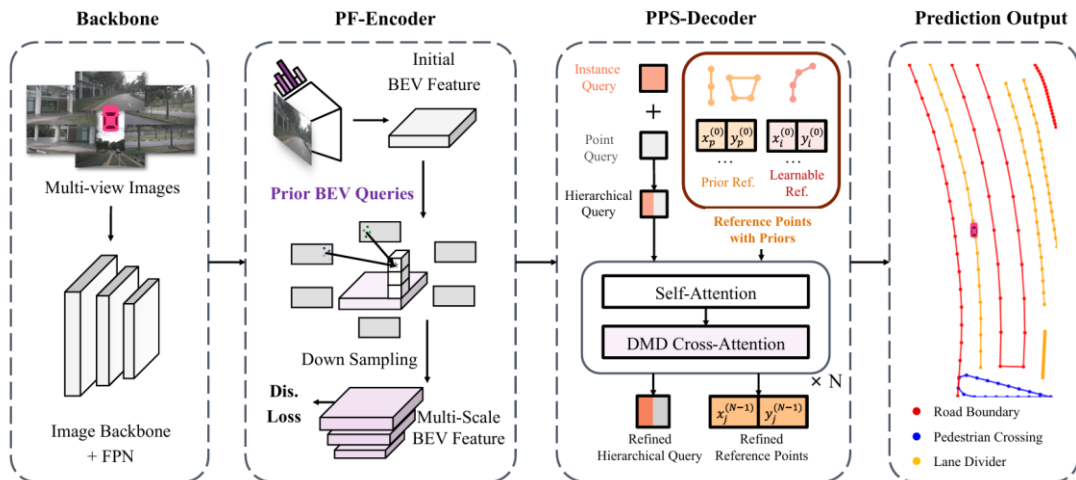
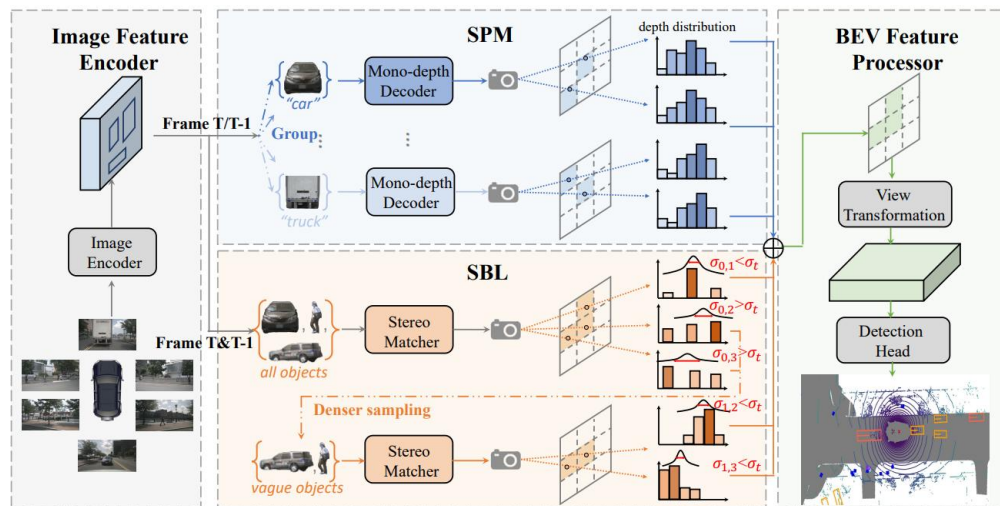


Figure 5: Three existing fusion strategies and our proposed Dual Dynamic Fusion (DDF) strategy.

X-Align采用基于元素相加的方式进行多模态BEV特征的融合过程；BEVFusion采用两种模态加权平均的BEV特征融合方式；HDMaNet采用两种不同模态通道拼接的方式完成BEV特征的融合。



视觉感知相关总结——任务特点



【三维目标检测/自动驾驶】IA-BEV: 基于结构先验和自增强学习的实例感知三维目标检测 (AAAI 2024) 相机3d目标检测-CSDN博客

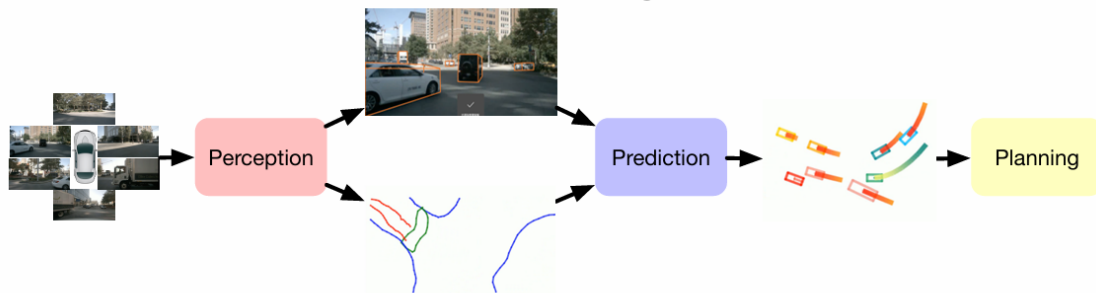
2402.11502: GenAD

[2408.08802] PriorMapNet: Enhancing Online Vectorized Map Construction with Priors

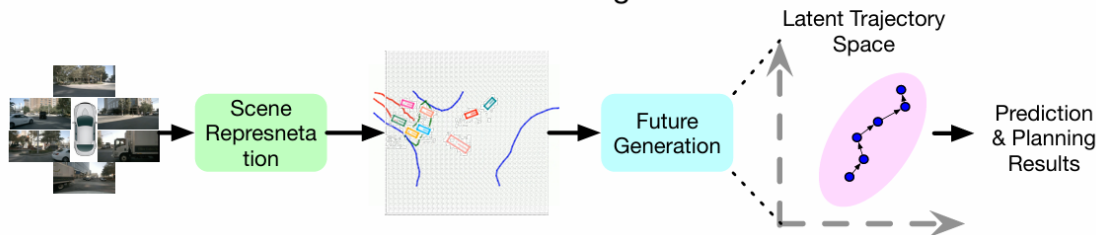


视觉感知相关总结——生成式/判别式

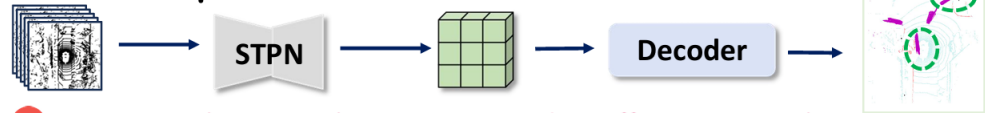
Conventional End-to-End Autonomous Driving



Generative End-to-End Autonomous Driving

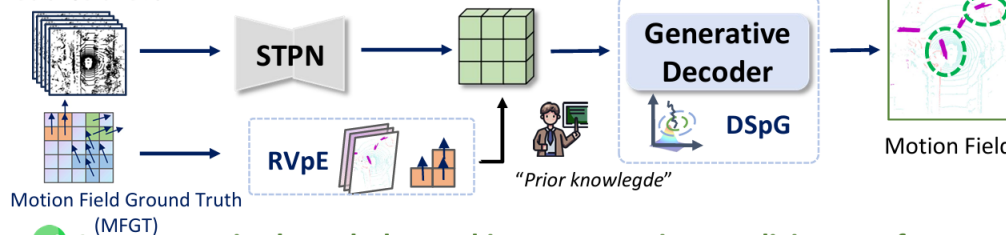


Conventional Pipelines



❌ Poor spatial-temporal consistency and insufficient generalization

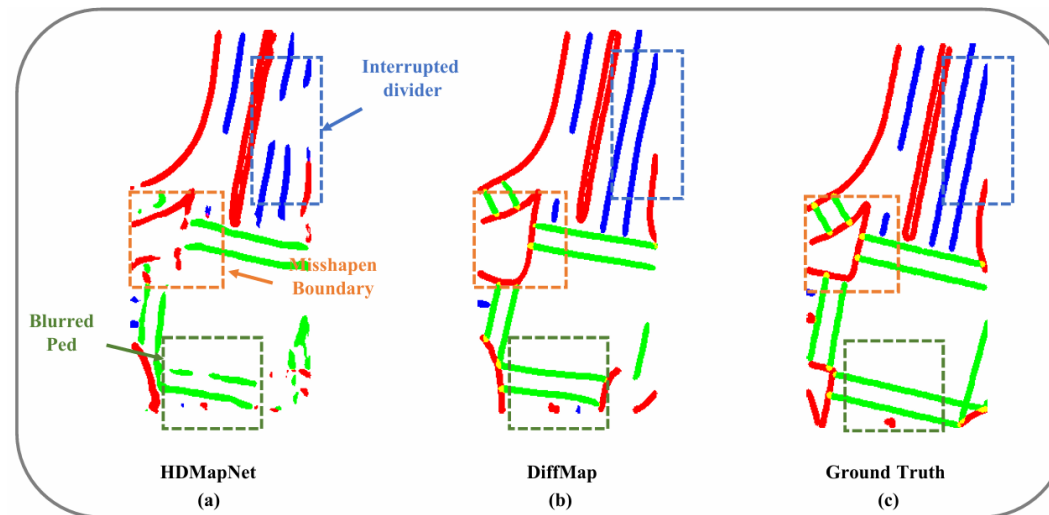
PriorMotion



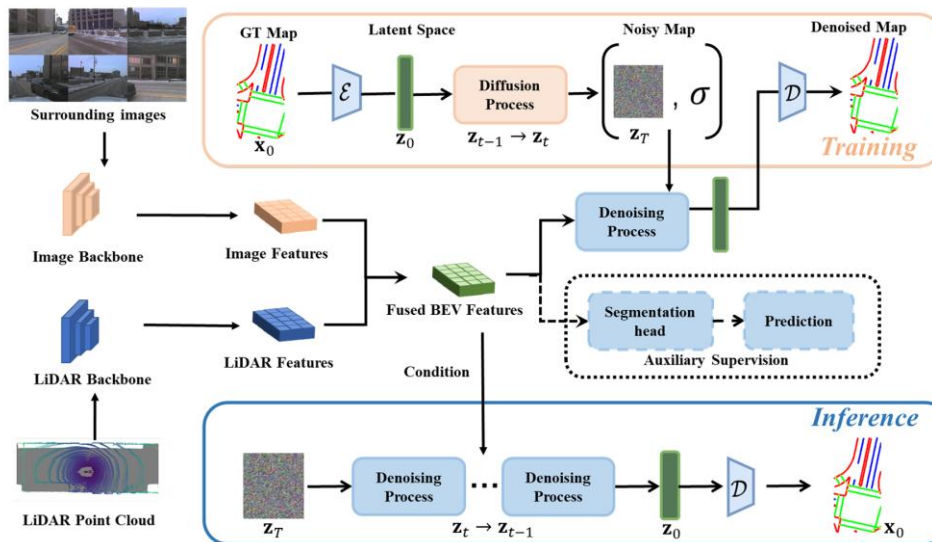
✅ Leverage prior knowledge and improve motion prediction performance!

2402.11502: GenAD

PriorMotion: Generative Class-Agnostic Motion Prediction with Raster-Vector Motion Field Priors



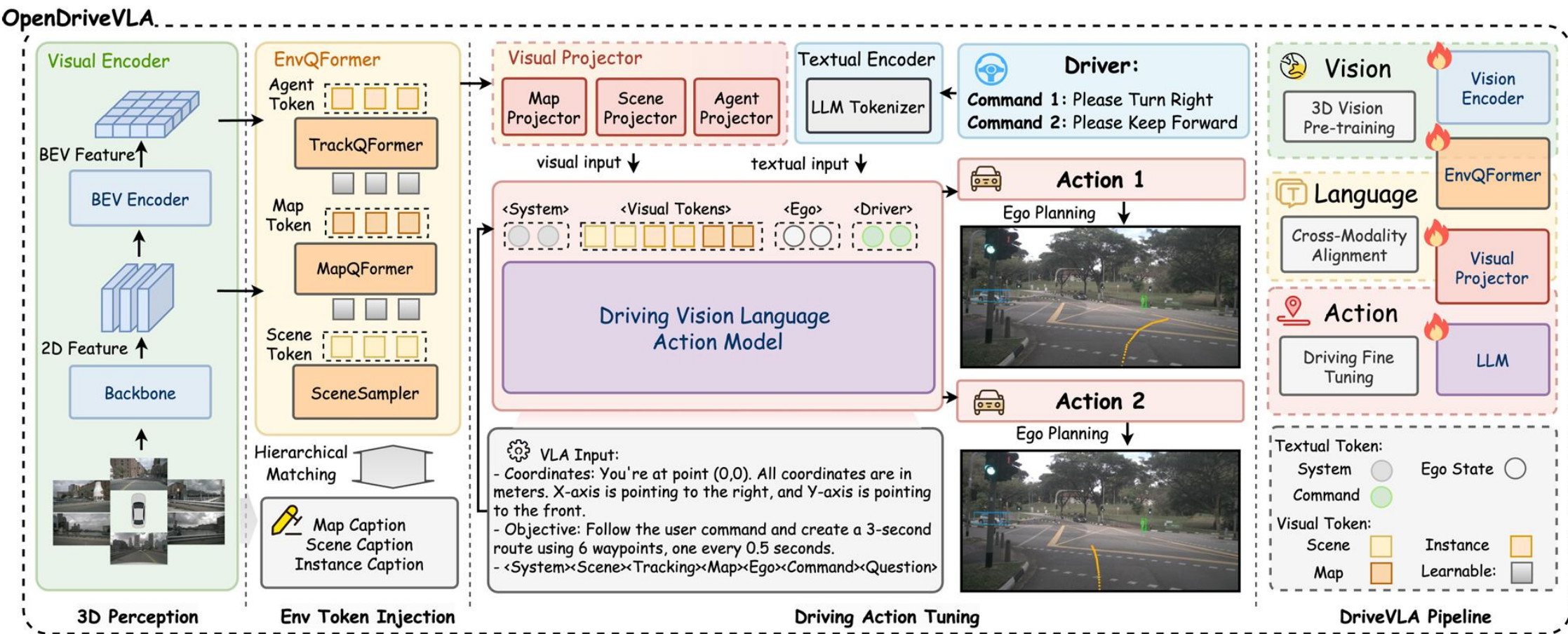
[2405.02008] DiffMap: Enhancing Map Segmentation with Map Prior Using Diffusion Model



视觉感知相关总结

➤ OpenDriveVLA

➤ 这个工作是如何处理视觉感知相关的信息的？





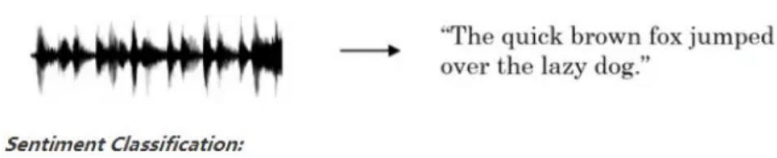
语言相关

序列建模

➤ 什么是序列建模？

一些真实场景中应用到的序列模型示例

□ 语音识别



输入音频剪辑板，生成文本转录

□ 机器翻译



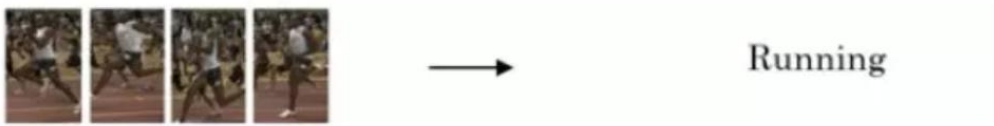
不同语言之间的转换

□ 情感分类



根据文本输入，预测出句子的情感
(积极、消极、愤怒、兴奋等)

□ 视频活动识别



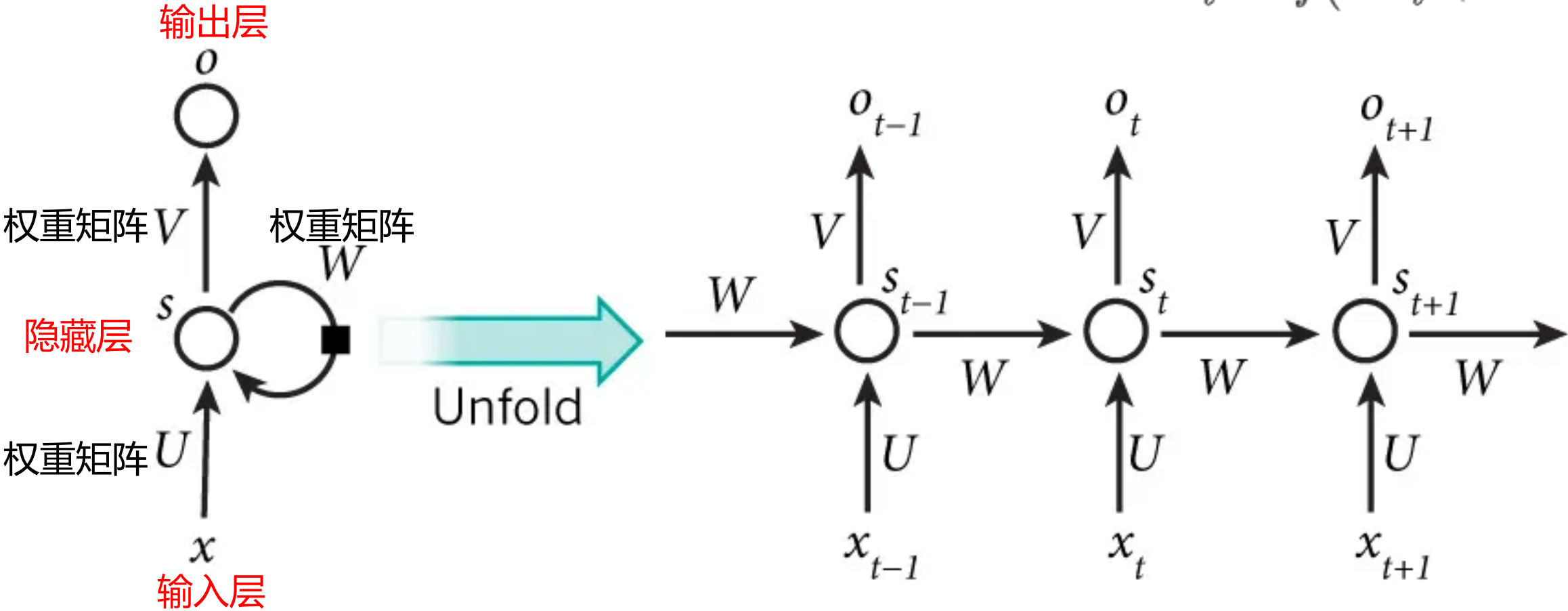
对即将到来的事件（和当前的趋势）
进行预测



序列建模

➤ 如何解决序列建模问题？——循环神经网络（RNN）

$$o_t = g(Vs_t)$$
$$s_t = f(Ux_t + Ws_{t-1})$$

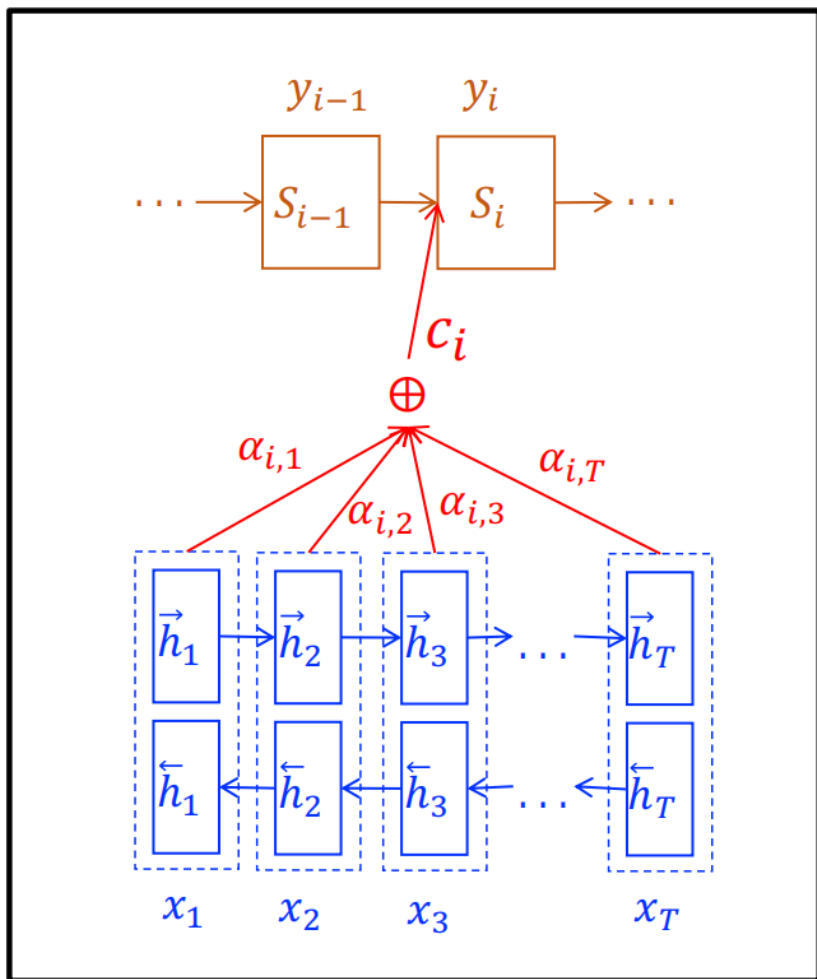


循环神经网络的隐藏层的值s不仅仅取决于当前这次的输入x，还取决于上一次隐藏层的值s



序列建模

➤ 循环神经网络 (RNN) 的问题



□ 模型复杂且浪费 (Wasteful)

- 模型本身复杂
- 有巨大的复杂度、对内存的要求高

□ 难以并行化

- 序列计算 (Sequential computation) 的特性阻碍了并行化。

能否设计一个更简单的模型?

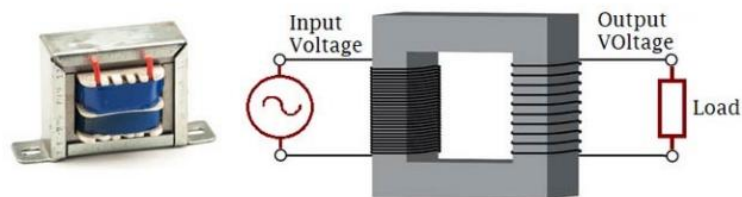


序列建模

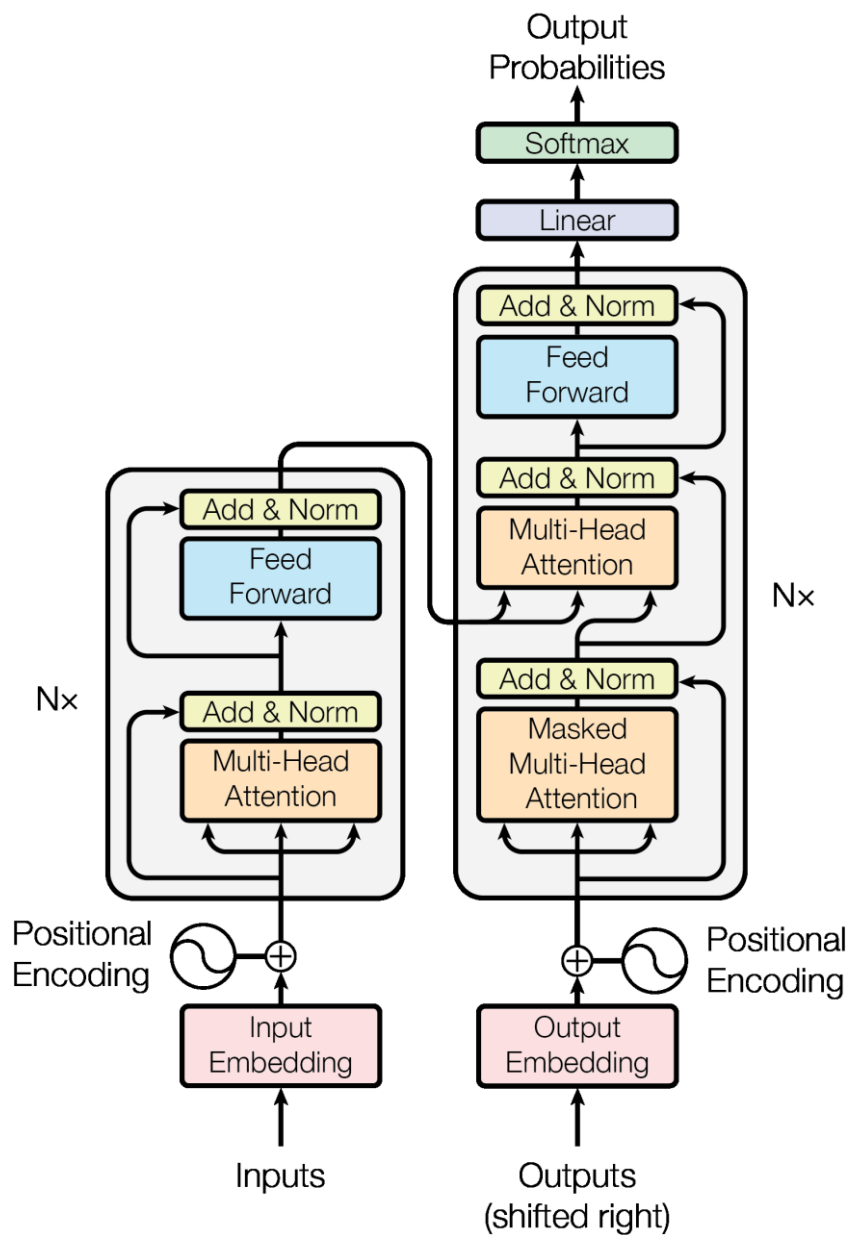
➤ Transformers



变形金刚



Transformer
变压器



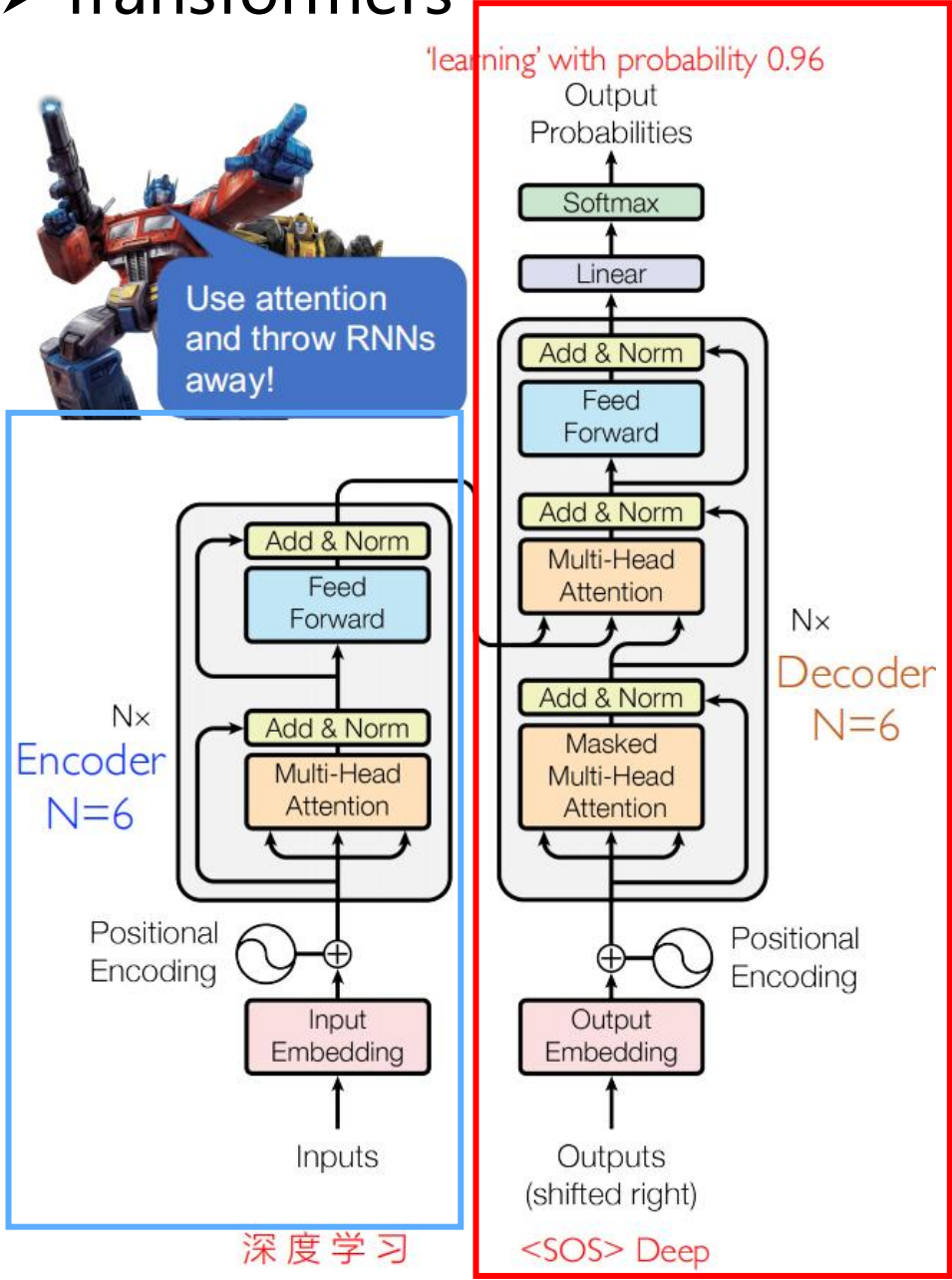
基于注意力机制
的神经网络

A neural network
based on the attention mechanism



序列建模

➤ Transformers



□ 主要组成部分:

- 缩放点积注意力 (Scaled Dot-Product Attention)

$$a(q, k) = q^T k / \sqrt{d_k} \rightarrow \text{Dimension of } q \text{ and } k$$

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- 多头注意力 ((Masked) Multi-Head Attention)
- 逐位置前馈网络 (Position-wise FFN)
- 残差连接 (Residual Connections)
- 层归一化 (Layer Normalization)
- 位置编码 (Positional encoding)



Encoder-decoder 架构

LLM（大型语言模型）的架构命名某种程度上是混乱而反常的。所谓的“decoder-only（仅解码器）”实际上意味着“自回归编码器-解码器”。“encoder only（仅编码器）”实际上包含一个编码器和解码器（非自回归），而所谓的“encoder-decoder（编码器-解码器）”真实含义是“自回归编码器-解码器”

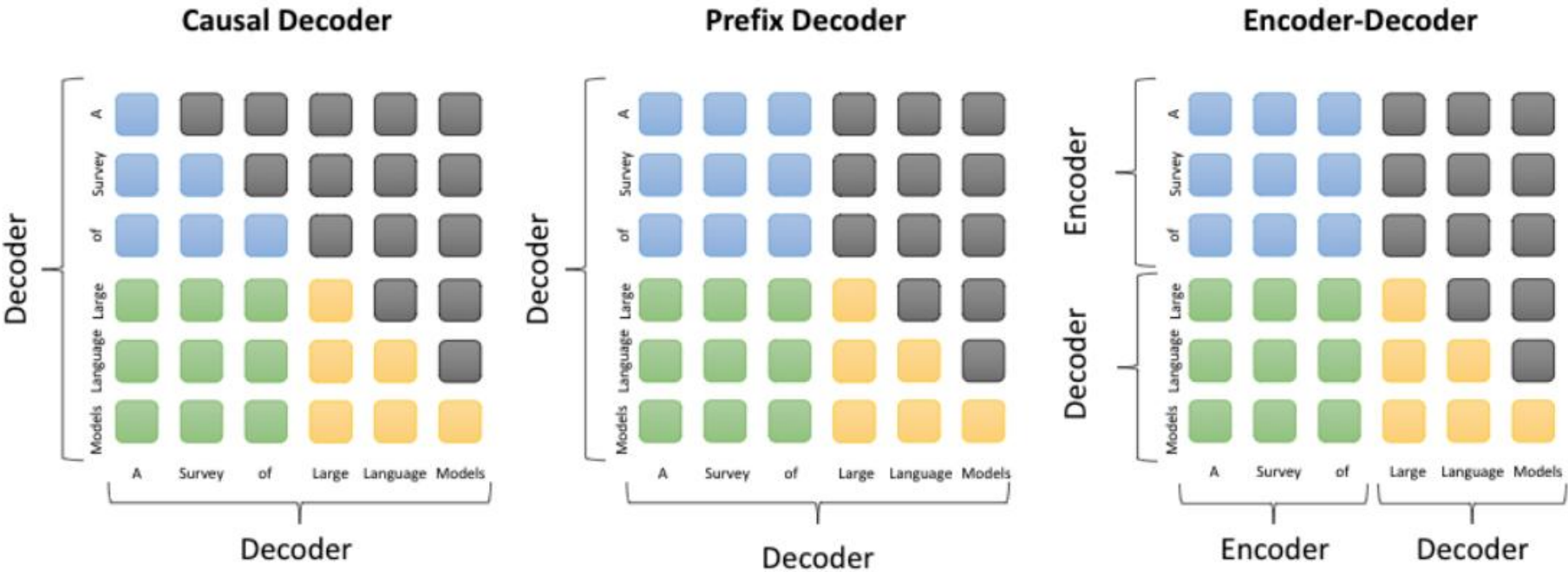
—— Yann Lecun

	解释	处理输入 注意力	处理输出 注意力	是否共享 参数	代表
Decoder-only之Prefix Decoder（前缀解码器）	一种特殊类型的Deco（der，它在生成输出序列时要求输出的前缀与给定的标记相匹配。这种结构通常用于生成特定格式的输出，例如在自然语言生成中，确保生成的句子以特定的词或短语开头。	双向	单向	是	GLM-130B、UniLM
Decoder-only之Causal Decoder（因果解码器）	另一种特殊类型的Decoder，它在生成每个时间步的输出时只依赖于之前的时间步，而不依赖于之后的时间步。这种结构常用于处理时间序列数据，其中每个时间步的预测只能依赖于过去的信息，而不能依赖于未来的信息。	单向	单向	是	GPT、LLaMA
Encoder-Decoder（编码器-解码器）	一个Encoder和一个Decoder组成的模型。这种结构在翻译任务中非常常见。	双向	单向	否	T5
Encoder-only（编码器）	也被称为单向架构，仅包含编码器部分。它主要适用于不需要生成序列的任务，只需要对输入进行编码和处理的单向任务场景，如文本分类、情感分析等。优点是能够更好地理解输入文本的语义和上下文信息，缺点是它无法直接生成文本输出。	双向	双向	是	BERT

差异在于Encoder、Decoder的注意力模式以及如何共享参数



Encoder-decoder 架构



架构	特性
Encoder-Decoder or Causal Decoder Encoder-only	训练：掩码语言模型
	模型类型：判别型
	预训练任务：预测遮蔽词和下一个句子
Prefix Decoder	训练：自回归语言模型
	模型类型：生成型
	预训练任务：预测下一个词

不同语言模型的注意力掩码设计不同，掩码全部取1为双向注意力，掩码下三角矩阵为单向注意力。图中蓝色指处理输入时前缀token彼此的注意力，绿色指输入token和输出token间的注意力，黄色则指的是输出token和掩码token间的注意力

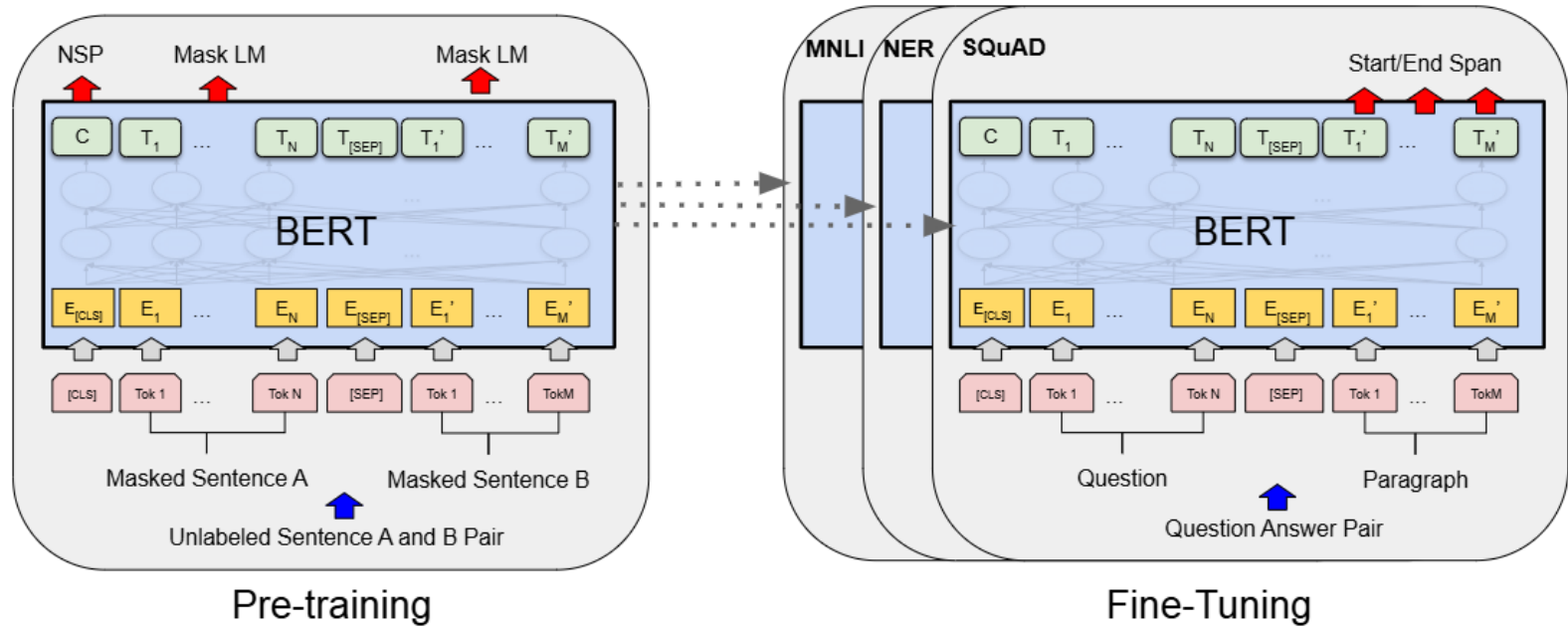


Encoder-only

掩码语言模型 (Masked Language Model)

考虑周围上下文的情况下预测句子中的遮蔽词的训练方法

- ✓ 这类模型在语义理解上展现出强大的能力，在如情感分析和命名实体识别的NLP任务中取得很好的效果。
- × 其完形填空式的训练目标与文本生成任务并不直接相关，且这种架构难以直接应用于从头到尾依次基于上文预测下一个token的标准语言模型目标。



BERT模型

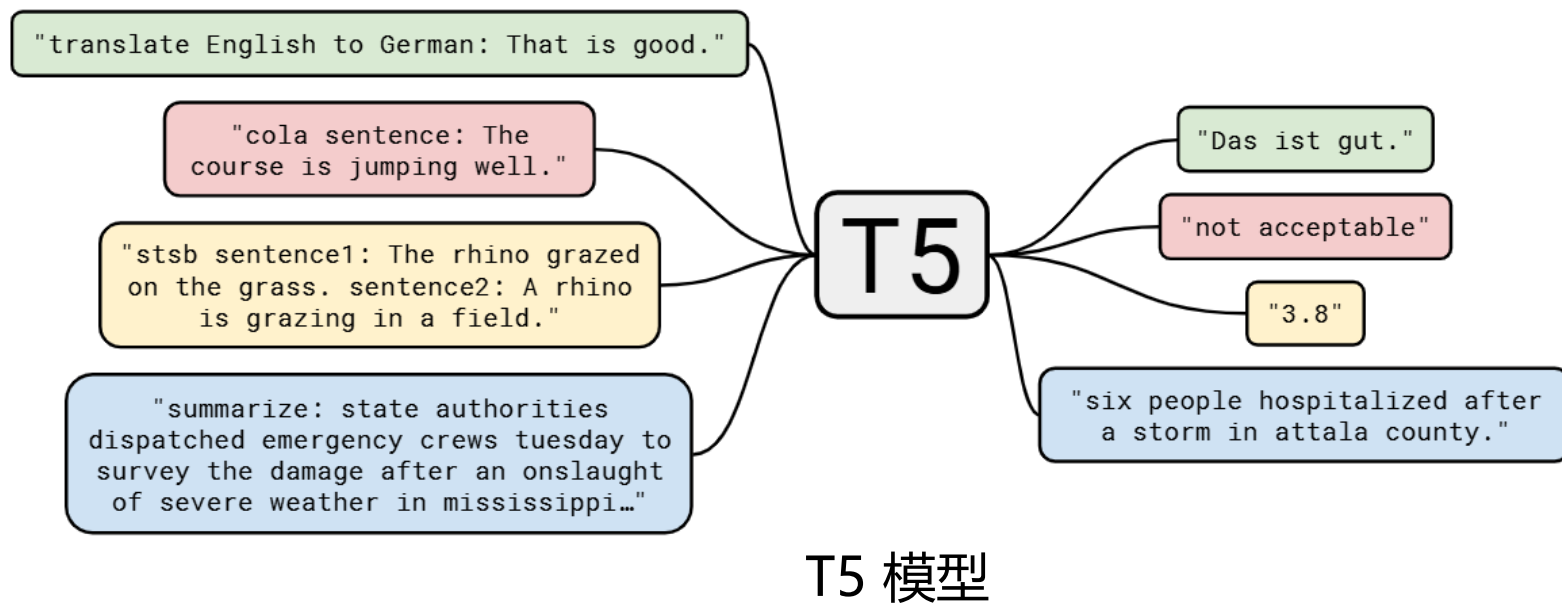
- BERT由Google于2018年10月提出
- 通过Masked LM和Next Sentence Prediction的学习目标，使模型能够更深入地理解单词之间及其使用上下文中的关系



Encoder-Decoder

➤ 在Encoder-only架构的基础上引入了Decoder组件,形成了一个完整的编码-解码系统。

- ✓ 当处理输入和输出的参数不共享时,模型在理解输入和据此生成特定输出方面表现出色。
非常适合于机器翻译、文本摘要等需要精确映射的任务。

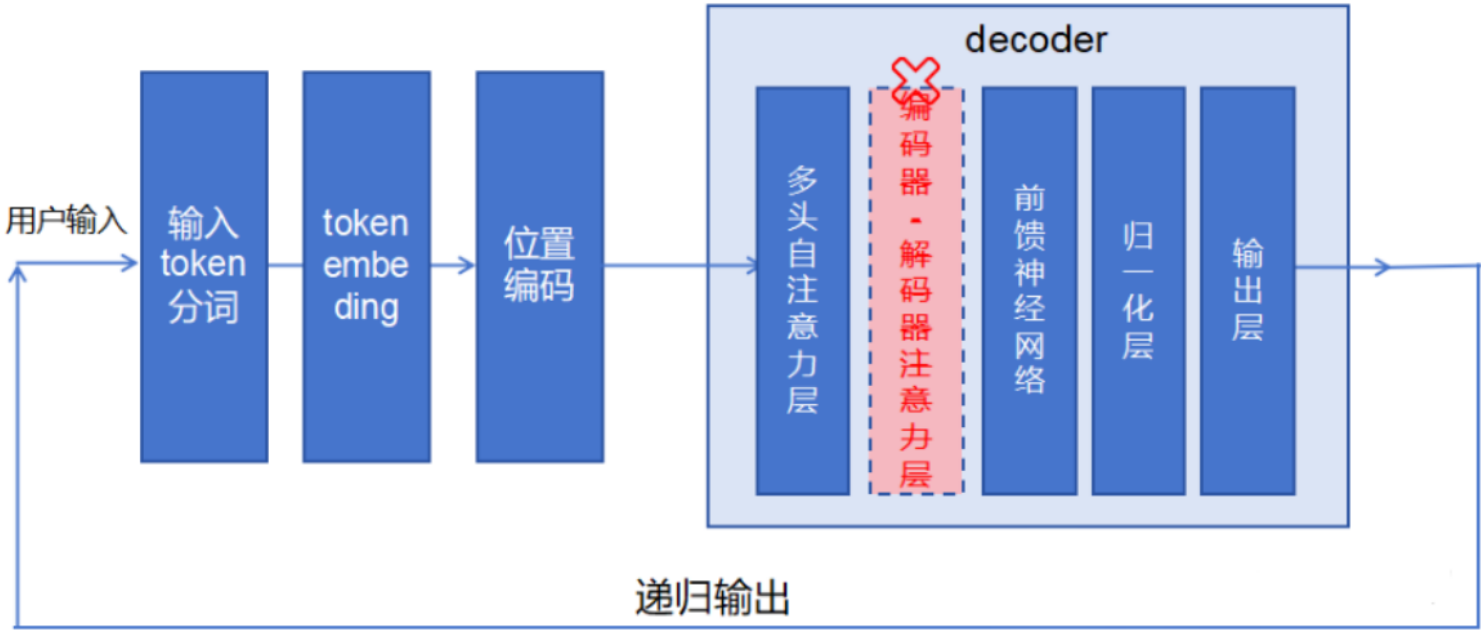


- 将多种NLP任务统一为文本到文本的转换范式
- 通过输入前缀指示不同任务
- 开创了早期的提示(Prompt)技术应用



Decoder-only

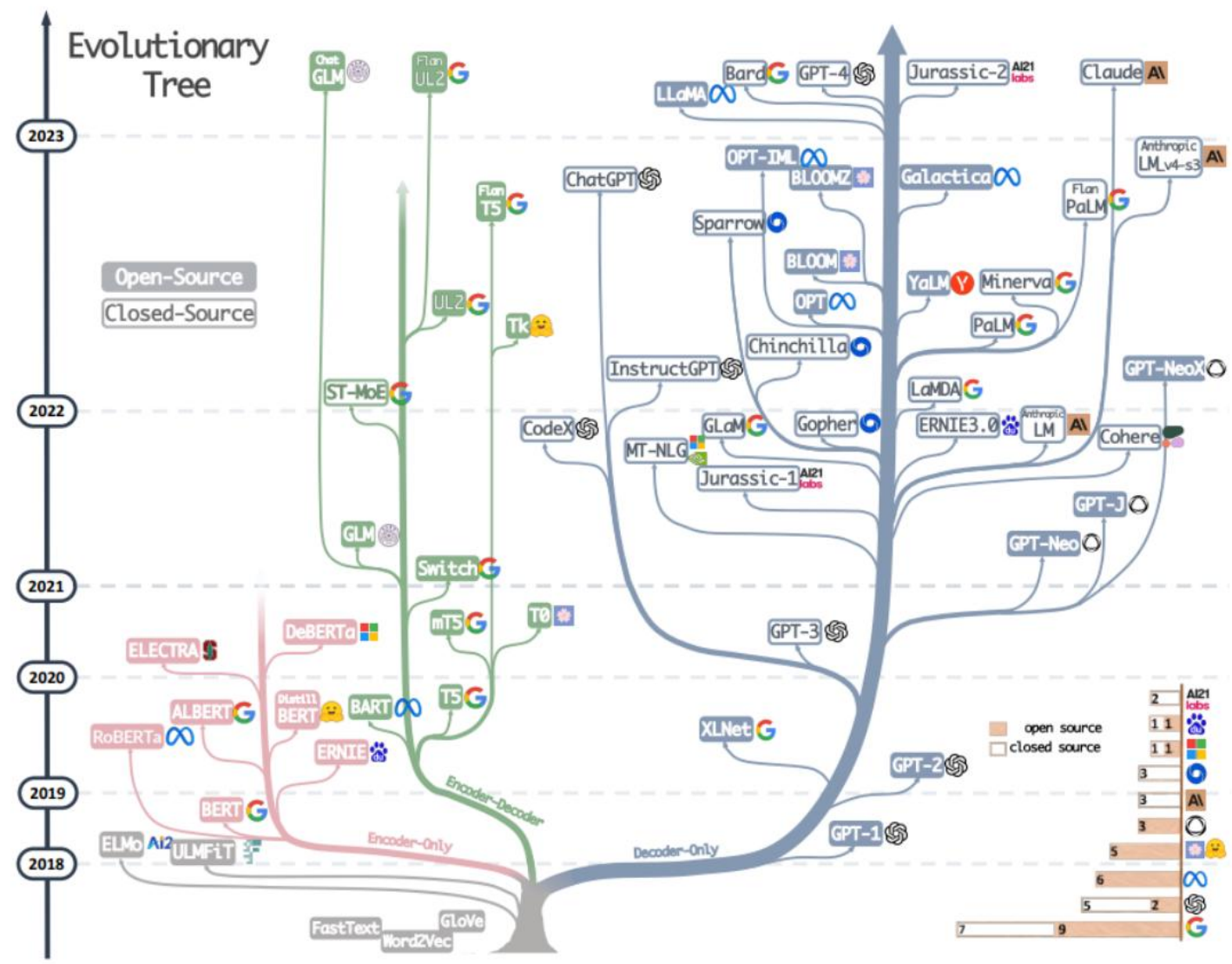
- 在许多开放式生成任务中，输入序列往往较为简单或缺失，此时维持完整编码器处理这类输入可能会显得冗余。在这种场景下，更轻量灵活的Decoder-only架构表现更为出色。
- ❑ Decoder-Only Transformer保留了原始Transformer解码器的核心结构，但对其进行了优化以适应单独使用的场景。



- 过往研究证明decoder-only泛化性能更好
- Decoder only结构比Encoder-Decoder结构更加简单，训练中Decoder only结构只需要一次前向传播，计算效率更高
- Decoder-only架构的Attention矩阵在理论上具有更强的表达能力，建模能力更强。



Encoder-decoder 架构的发展

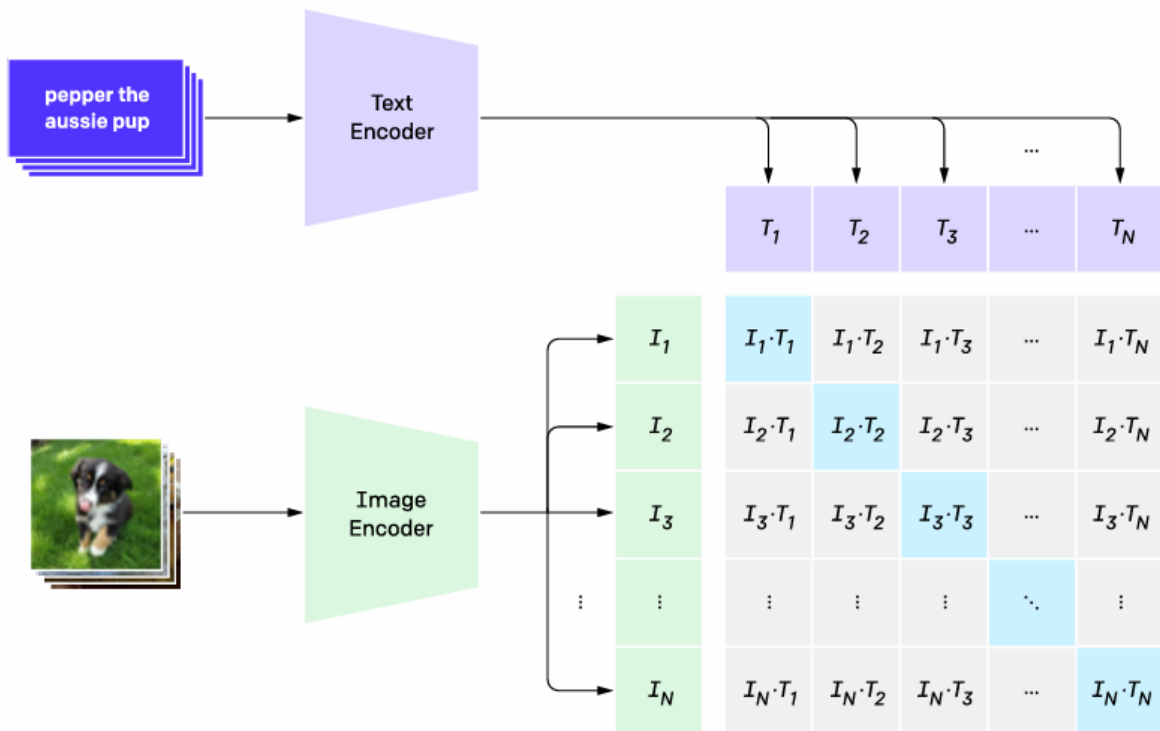


- 在LLM发展得早期阶段，encoder-only和encoder-decoder模型更受欢迎。然而，自2021年起，随着游戏规则改变者GPT-3入局，decoder-only经历了显著的增长并逐渐主导LLMs的发展
- encoder-decoder模型仍具有前景，因为这种类型的架构仍在积极探索中，而且大多数都是开源的。Google对开源该架构做出了重大贡献。然而，decoder-only模型的灵活性和多功能性似乎使得Google坚持这一方向的前景不太乐观。



视觉语言对齐——CLIP的介绍

- V-L模态如何进行对齐和融合，从LLM->VLM?
- 核心问题是如何从语言这个模态构建到视觉的对应关系?



```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t            - learned temperature parameter
```

```
# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]
```

```
# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)
```

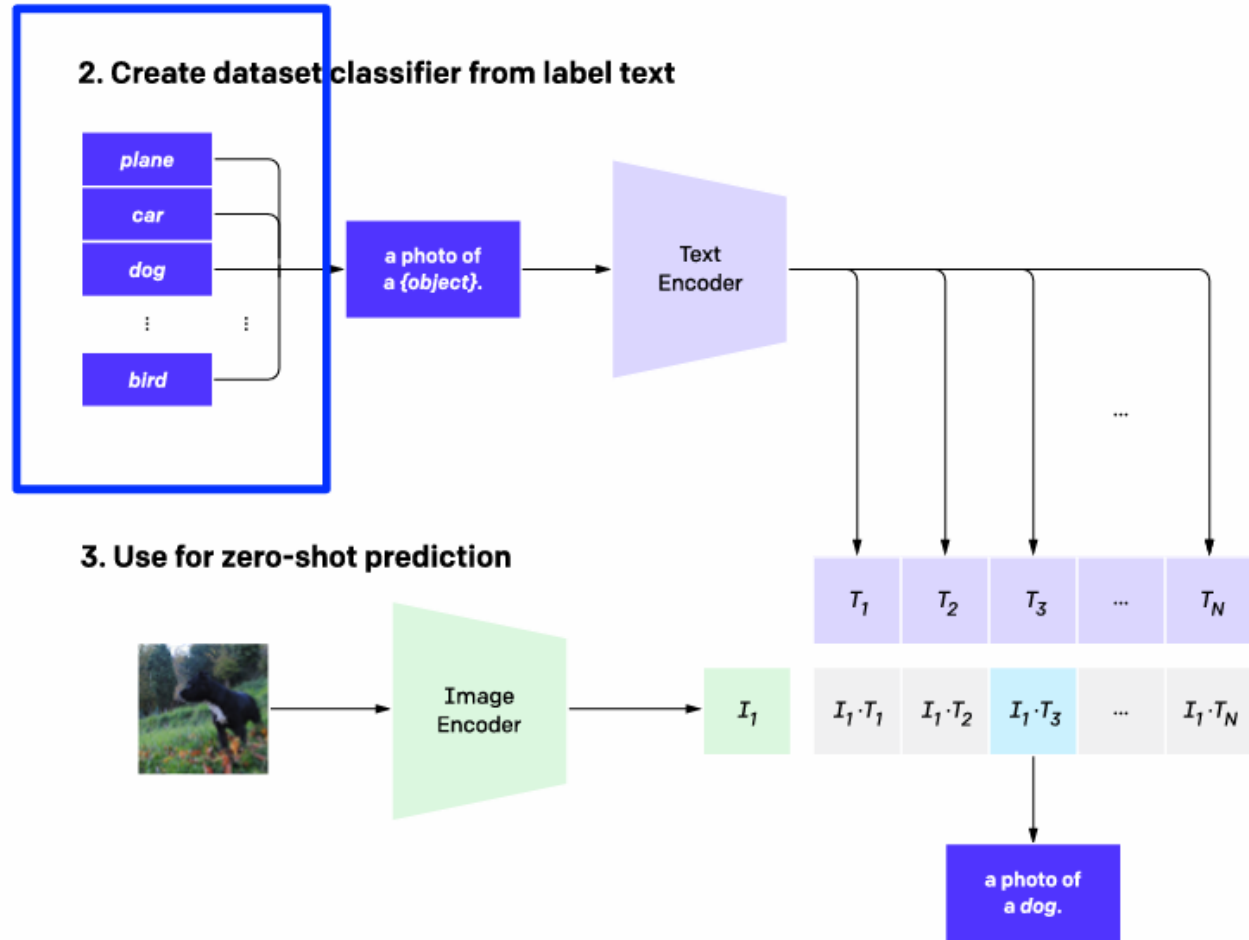
```
# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)
```

```
# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```



视觉语言对齐——CLIP的应用

- 给一个图片和多个词汇的prompt, 模型会选择相似度最高的那个文本标签作为图像的预测结果;
- 这里由大模型的提示工程的使用, 直接给单词不如给一个句子;
- 如何进一步提示零样本分类的性能?
 - 提示集成方法, 相当于一种后处理, 在ImageNet上效果提升达到5%;



多模态基础模型

- 通用形式：当前主流为“图像到文本”的生成式模型。
- 模型架构：通过可训练的连接模块，将视觉特征接入大型语言模型。
- 训练目标：以自回归的方式预测下一个文本标记。
- 未来展望：端到端的图像与文本生成？



Language

Image

Language Model

Connection Module

Vision Encoder

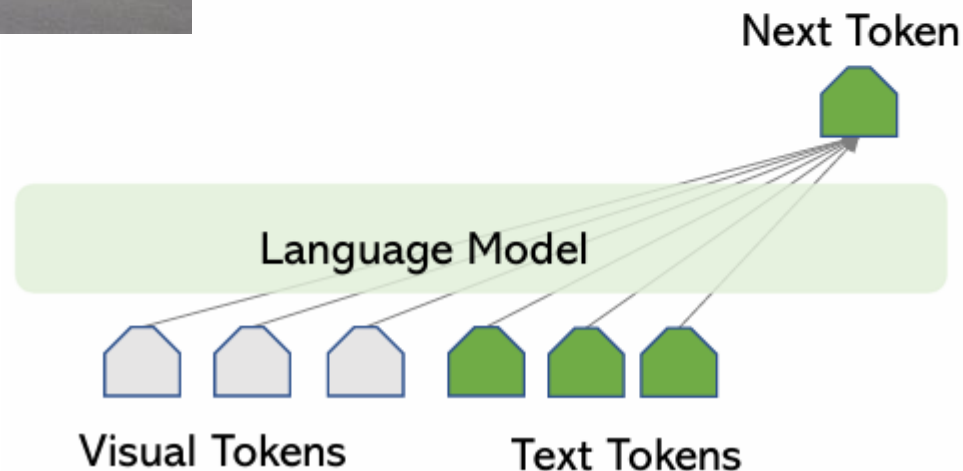
输入：一张图片（例如：这是一个斑马线场景，前方有多辆车）。

视觉编码器：将图片编码为视觉特征。

连接模块：将视觉特征转换为语言模型可以理解的“视觉标记”。

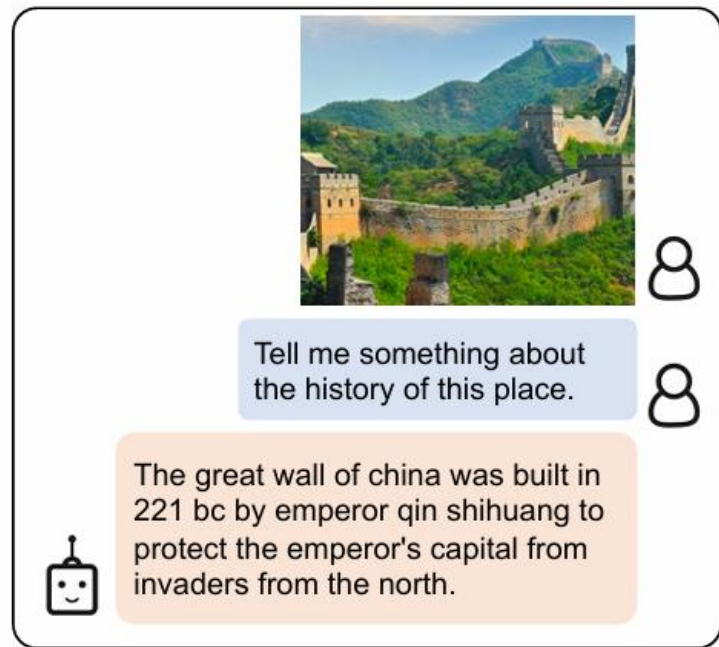
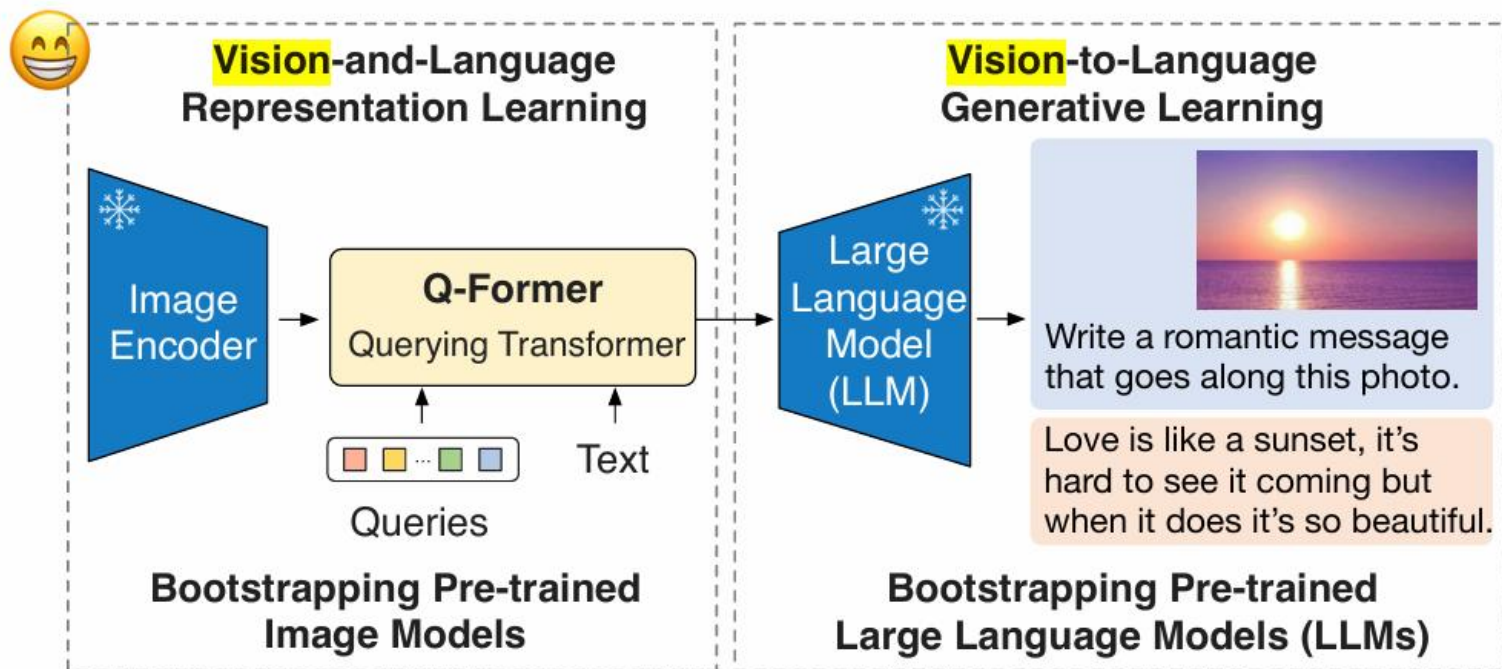
语言模型：接收视觉标记和文本提示，开始自回归地生成文本描述。

训练目标：模型的任务是预测被掩码的文本标记。



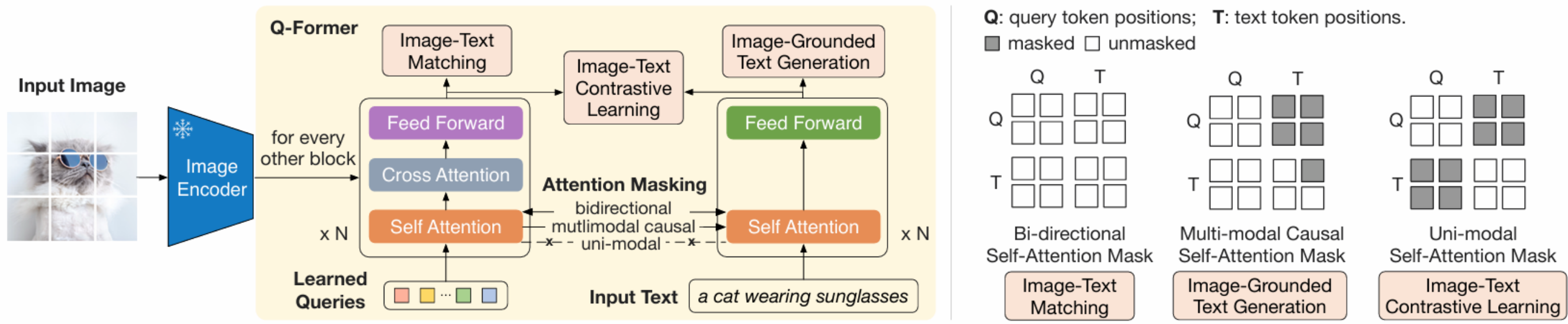
多模态基础模型——BLIP2

- ❑ 视觉编码器: off-the-shelf& frozenViTfromCLIP
- ❑ 连接模块: Lightweight Querying Transformer(Q-Former)
- ❑ 语言模型: off-the-shelf&frozenFLAN-T5orOPT



多模态基础模型——BLIP2

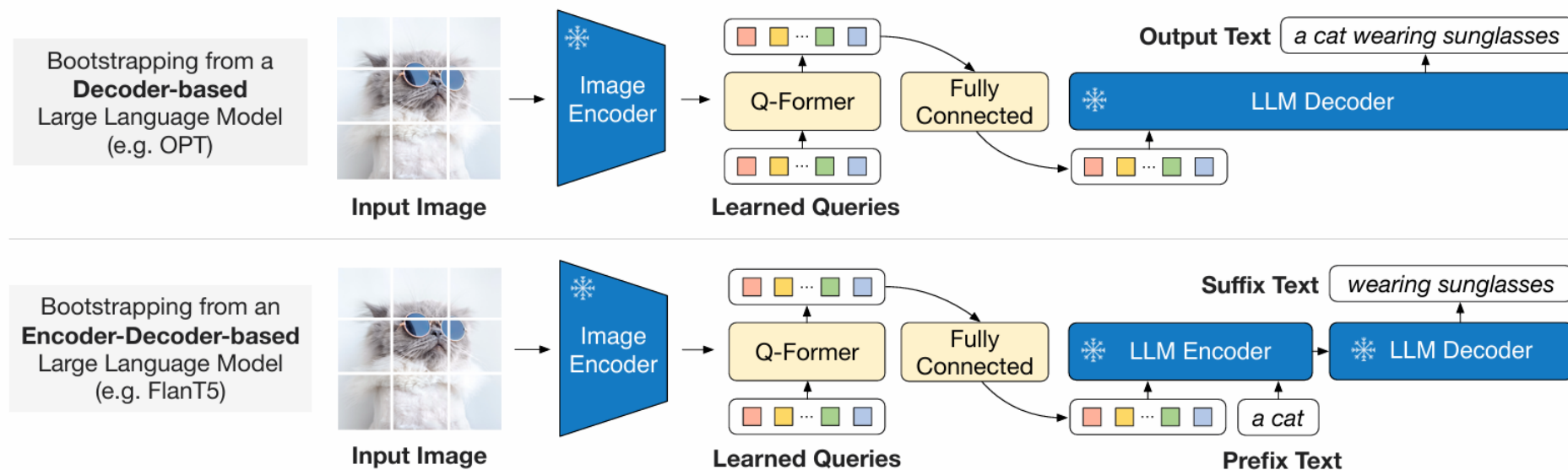
- BLIP2中Q-Former只有188M参数，继承BERT的权重，且图文编码器共享transformer的注意力层
- 三个阶段的训练目标：图文匹配任务，基于图像的文本生成，图像-文本对比学习。



Liet al.BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. ICML 2023.

多模态基础模型——BLIP2

- ❑ 基于冻结大语言模型的视觉到语言生成学习
- ❑ 将Q-Former输出的查询嵌入进行线性投影。
- ❑ 对于基于解码器的大语言模型，使用语言建模损失。
- ❑ 对于基于编码器-解码器的大语言模型，使用前缀语言建模损失。

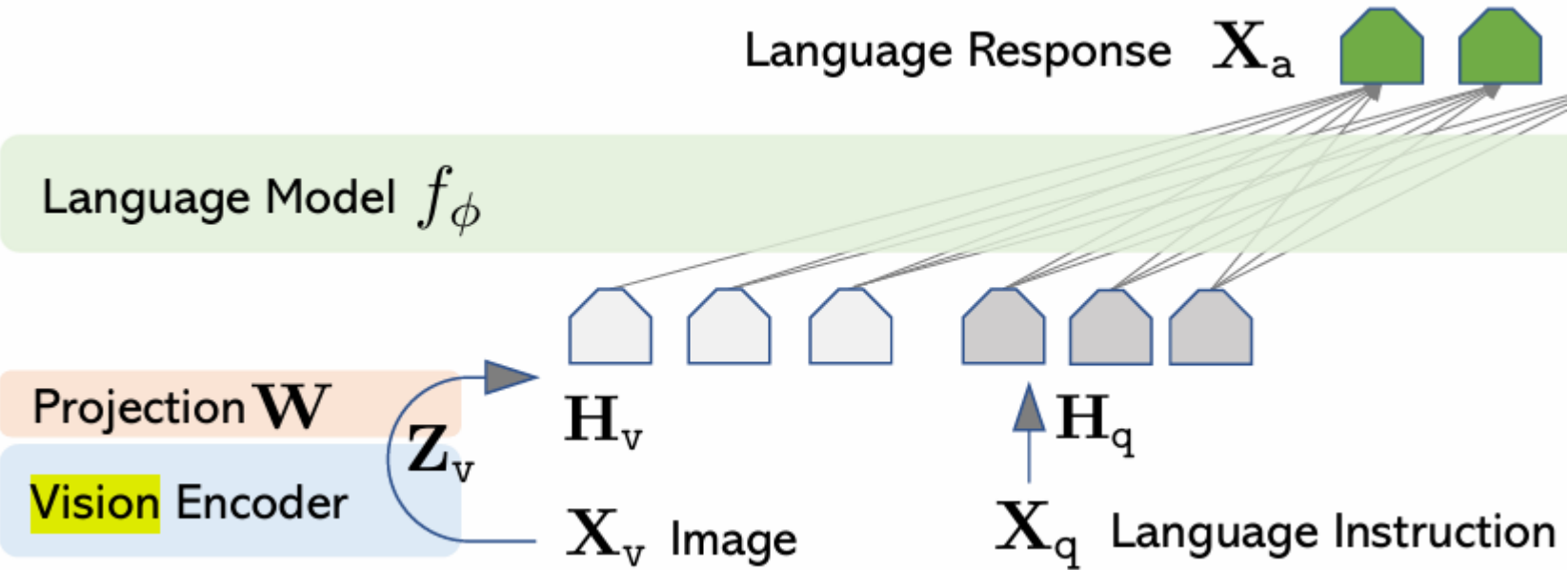


Liet al.BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. ICML 2023.



多模态基础模型——LLaVA

- ❑ 架构：预训练的LLaMA + CLIP-ViT-L + 一个简单的线性层
- ❑ 训练：两阶段指令微调，使用自回归目标
- ❑ 特征对齐预训练：保持视觉编码器和LLM权重冻结，仅更新投影层。
- ❑ 端到端微调：保持视觉编码器权重冻结，更新投影层和LLM。



系统消息 = 一个精通驾驶场景的老司机。

人类：[图像] 请描述这张图片。

助手：图片展示了一个大雾天气下的驾驶场景，前面由一个模糊的车辆剪影。

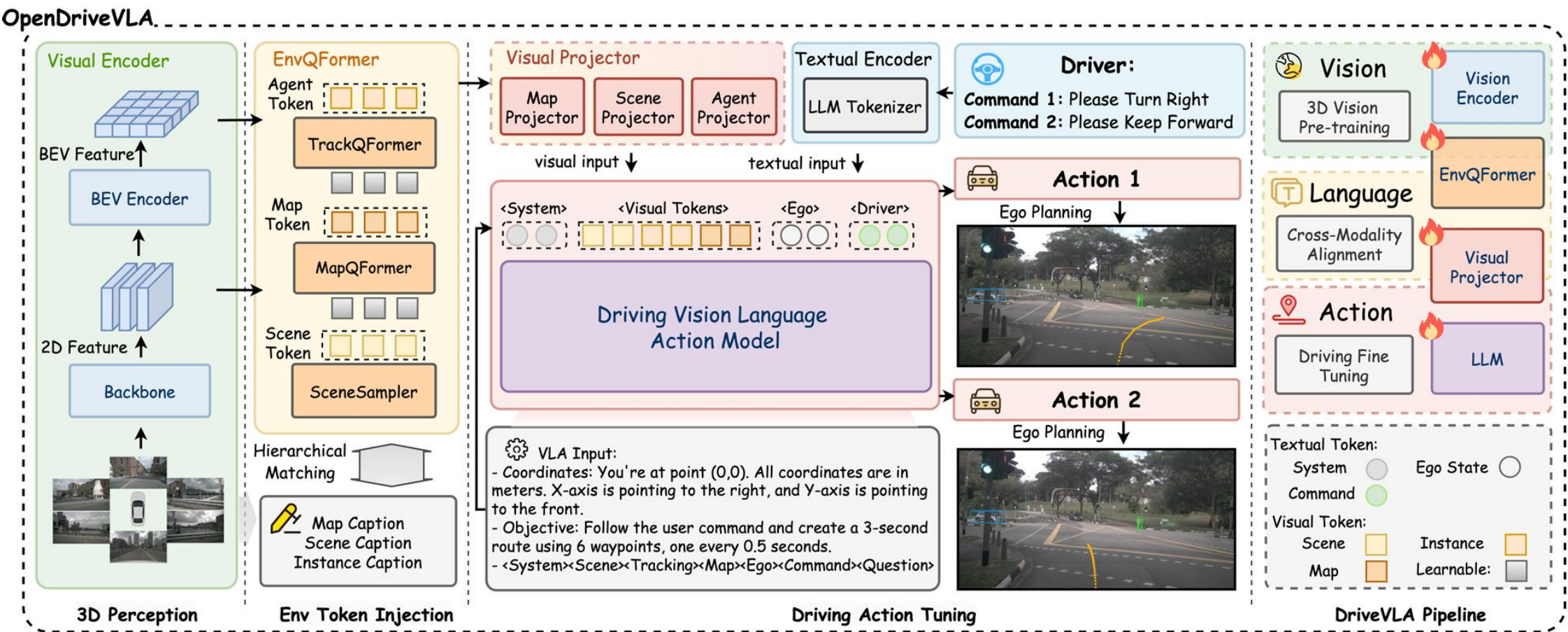
```
X_system-message <STOP> \n
Human : X_instruct^1 <STOP> \n Assistant: X_a^1 <STOP> \n
Human : X_instruct^2 <STOP> \n Assistant: X_a^2 <STOP> \n ...
```



语言相关总结

➤ OpenDriveVLA

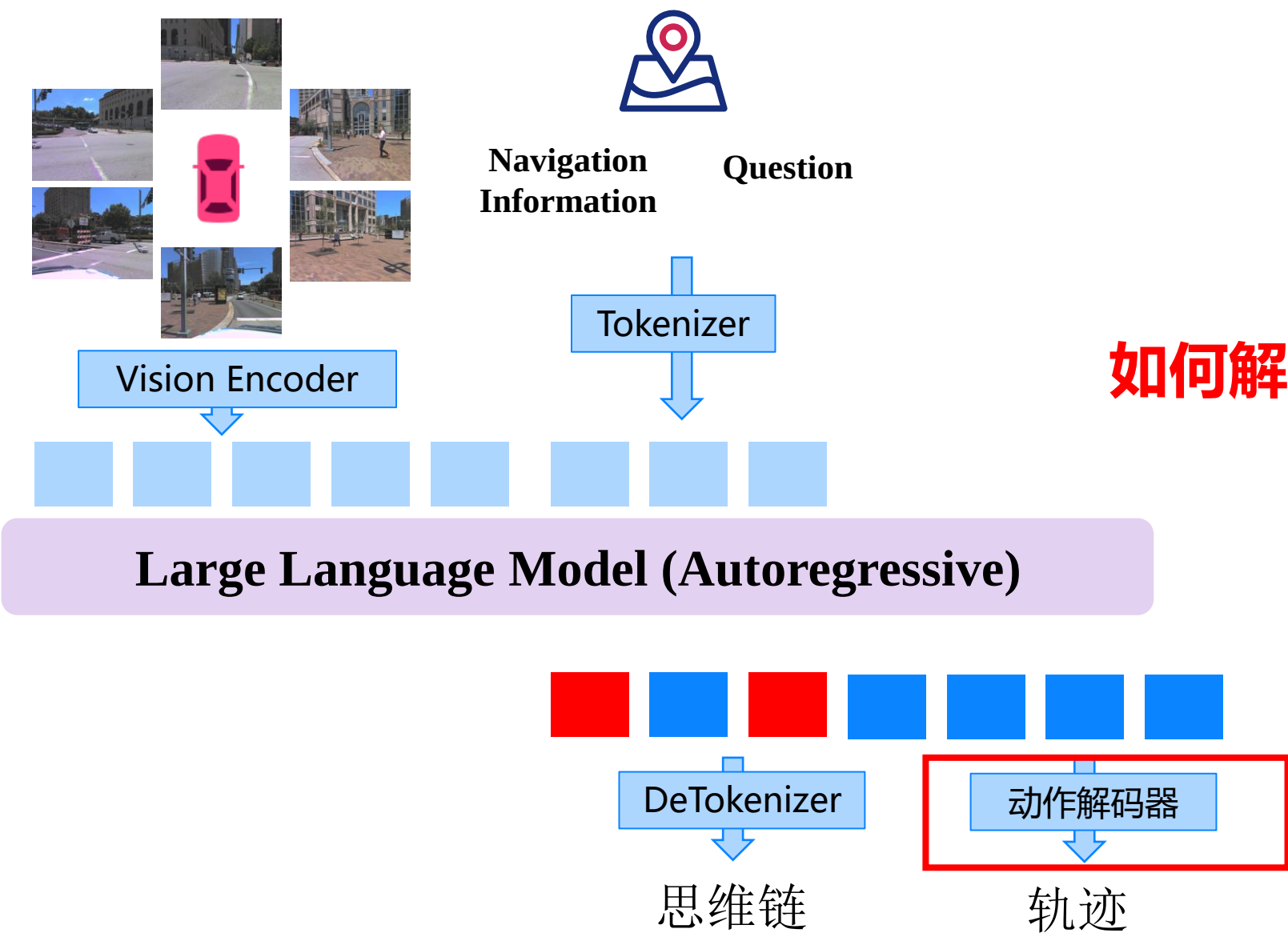
➤ 这个工作是如何处理语言相关以及视觉-语言对齐的信息的?



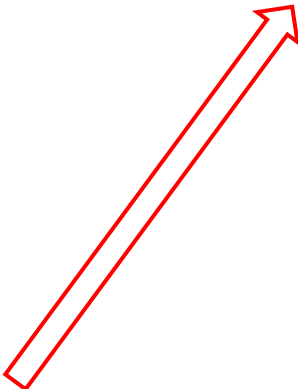


Action相关基础

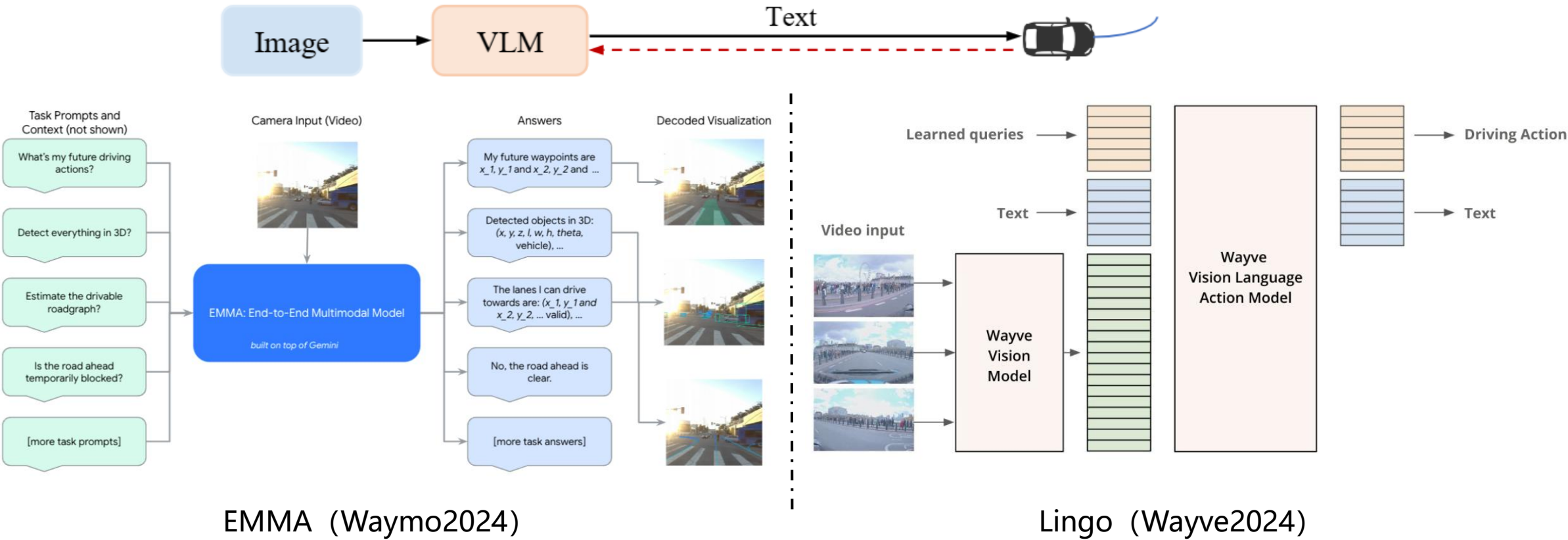
VLA整体框架



如何解码得到更精准数值预测?



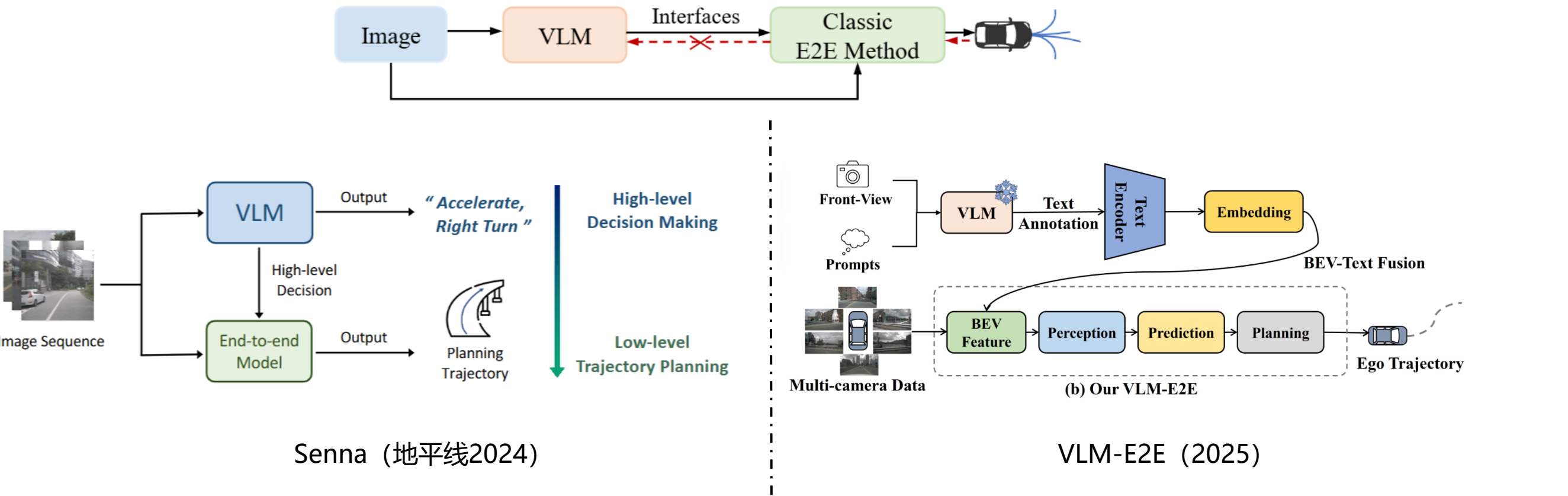
直接使用视觉语言模型（VLM） 输出基于文本的规划结果



➤ **数值预测缺陷：** LLMs的输出空间为离散语言符号，难以直接生成精确数值预测，可能导致性能不足或安全隐患。



利用 VLM 输出元动作辅助经典端到端系统

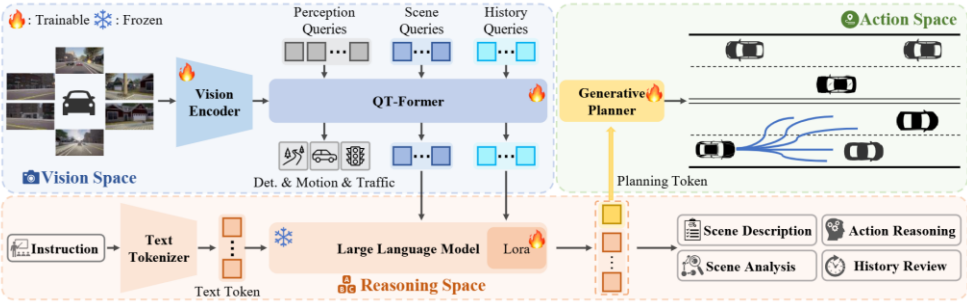
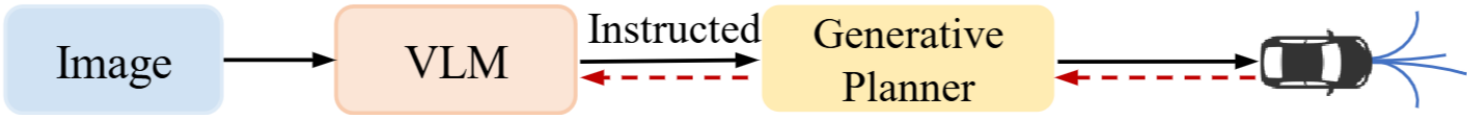


➤ **VLM作用有限：** 将推理空间与动作空间解耦，阻碍了轨迹优化和 VLM 推理过程之间的协同优化。

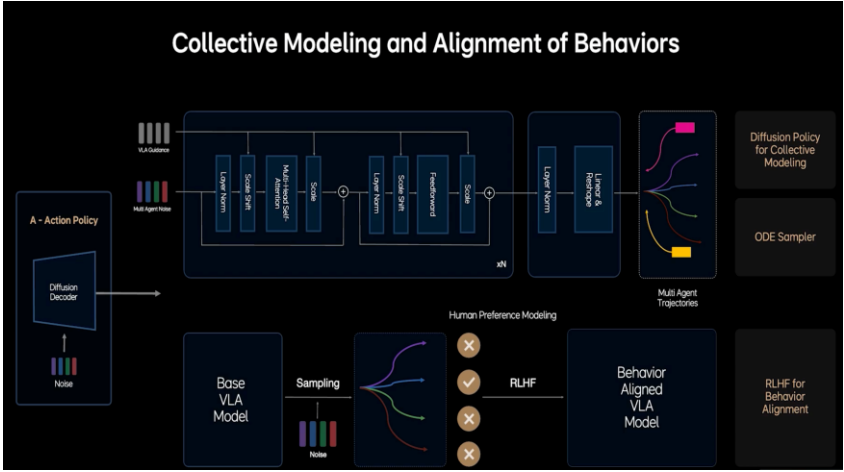


轨迹解码器——生成模型

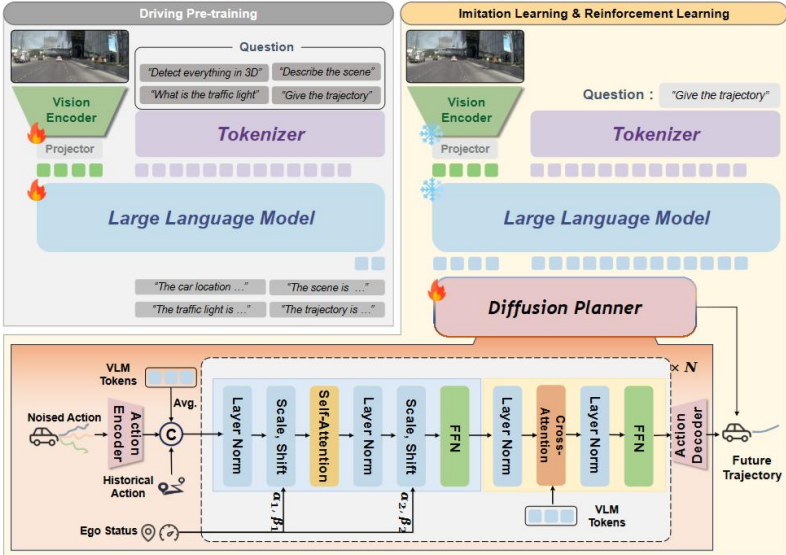
利用生成式规划器对齐动作空间



ORION (小米2025)



MindVLA (理想2025)

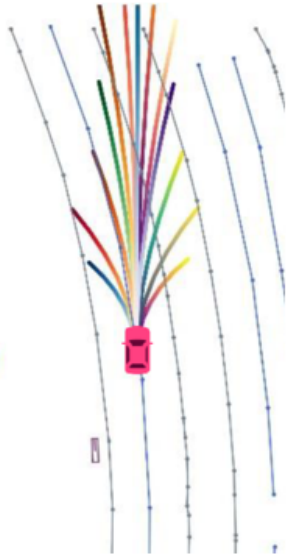


RecogDrive (小米2025)

➤ 利用生成式规划器解决LLM数值预测缺陷

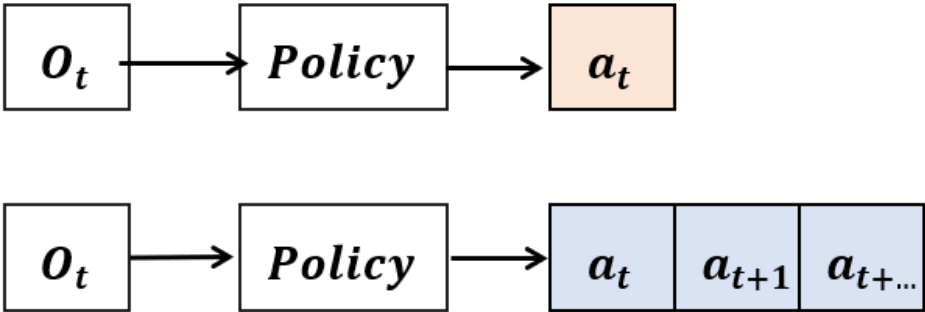


动作侧目前需要解决的问题



多模态分布

 模式坍缩



时序相关性

 动作不一致性



(c) Two-modal, time $T + 3$



(d) Single-modal, time $T + 3$

符合驾驶规则或
环境约束

 过拟合





大模型技术与智驾 VLA碰撞的火花

提示词工程Prompt Engineering

- 提示词工程是通过设计特定的输入提示来引导模型生成期望输出的方法。
- 打个比方：提示词工程就像我们课题组老师提出一个任务，这里规定任务的形式和产出物，我们学生根据这样的“任务要求”去得到老师期望的任务目标。
- 怎么样做好提示词工程？

一个提示词包含**清晰的指示**、**相关的上下文**、有助于理解的**例子**、明确的**输入**以及期望的**输出格式**描述。



➤ 你是一个**雅思考官**，请根据**雅思官方的写作评价标准**，来**评阅我的大作文**，比如**这样的写作可以被评价为7+**的分数，我将发送给您**雅思作文考题**以及**我的写作用本**，请给出我的写作的**各项评分**，要求有**单独维度**和**总评的分数**，并且给出一些**修改建议**。



提示词工程Prompt Engineering

- 自动驾驶领域如何做好提示词工程？
- 考考你，这是GPT-Driver/DriveMLM的提示词设计

一个提示词包含**清晰的指示**、**相关的上下文**、**有助于理解的例子**、**明确的输入**以及**期望的输出格式描述**。

System Message: You are a driving assistant to drive the car. You need to follow the navigation command and traffic rules. The traffic rule is ... Path decision definitions: 'FOLLOW' means ..., Speed decision definitions: 'KEEP' means ..., Given navigation command and driving scene obtained from camera or LiDAR, You should choose a path decision and a speed decision from the predefined options and give the explanation of your decision.

Q1(Human): Caption instruction (e.g. Describe the current driving environment.)
A1(DriveMLM): Caption response (e.g. It is currently daytime. A red car is driving away in front of ego.)
Q2(Human): Navigation instruction (e.g. The navigation command is turned right. Please choose a path decision and a speed decision.)
A2(DriveMLM): Speed and path decision (e.g. **RIGHT_CHANGE, KEEP**)
Q3(Human): Explanation instruction (e.g. Please explain why to choose these decisions.)
A3(DriveMLM): Explanation response (e.g. Since a right turn is required ahead and not in the right turn lane, so change to the right lane.)
Q4(Human): Instruction (e.g. I'm in a hurry. Can you overtake the front car?)
A4(DriveMLM): Speed and path decision (e.g. **LEFT_CHANGE, ACCELERATE**)
Q5(Human): Explanation instruction (e.g. Please explain why to choose these decisions.)
A5(DriveMLM): Explanation response (e.g. Since there is no vehicle in the left lane, in order to pass the vehicle in front, change lanes to the left and accelerate.)



Human High-level Instruction: ...If special vehicles like police or ambulance are behind, yield and allow them to pass first...
Assistant: **RIGHT_CHANGE, KEEP**
Human Instruction: Explain why you make this decision.
Assistant: I'll make a right lane change, because there is an ambulance behind.



Human High-level Instruction: ...Traffic light indications: a) Red: vehicles may proceed; b) Yellow: ...
Assistant: **FOLLOW_LANE, ACCELERATE**
Human Instruction: Explain why you make this decision.
Assistant: I'll accelerate since the traffic signal is red.

Autonomous Driving Planner

Role: You are the brain of an autonomous vehicle. Plan a safe 3-second driving trajectory. Avoid collisions with other objects.

Context

- Coordinates: X-axis is perpendicular, and Y-axis is parallel to the direction you're facing. You're at point (0,0).
- Objective: Create a 3-second route using 6 waypoints, one every 0.5 seconds.

Inputs

1. Perception & Prediction: Info about surrounding objects and their predicted movements.
2. Ego-States: Your current state including velocity, heading angular velocity, can bus data, heading speed, and steering signal.
3. Historical Trajectory: Your past 2-second route, given by 4 waypoints.
4. Mission Goal: High-level goal for the next 3 seconds.

Task

- Thought Process: Note down critical objects and potential effects from your perceptions and predictions.
- Action Plan: Detail your meta-actions based on your analysis.
- Trajectory Planning: Develop a safe and feasible 3-second route using 6 new waypoints.

Output

- Thoughts:
 - Notable Objects
 - Potential Effects
- Meta Action
- Trajectory (MOST IMPORTANT):
 - [(x1,y1), (x2,y2), ... , (x6,y6)]

Perception and Prediction:

- **animal** at **(-1.93,7.00)**, moving to **(-2.31,10.89)**.
- **car** at **(-8.67,0.12)**, moving to **(-8.50,-0.08)**.
- **adult** at **(-1.21,6.78)**, moving to **(-1.29,10.48)**.

Ego-States:

- Velocity (vx,vy): **(0.00,1.46)**
- Heading Angular Velocity (v_yaw): **(-0.00)**
- Acceleration (ax,ay): **(0.01,-0.15)**

Historical Trajectory (last 2 seconds): [(-0.00,-6.74), (-0.03,-4.73), (-0.03,-3.07), (-0.02,-1.46)]

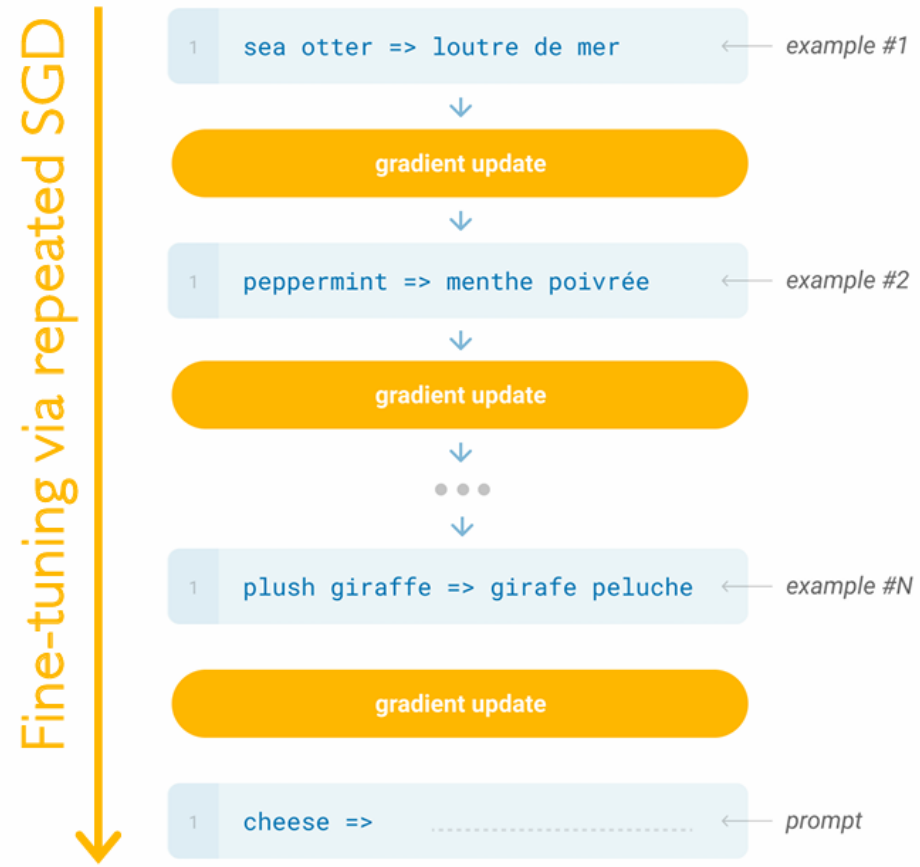
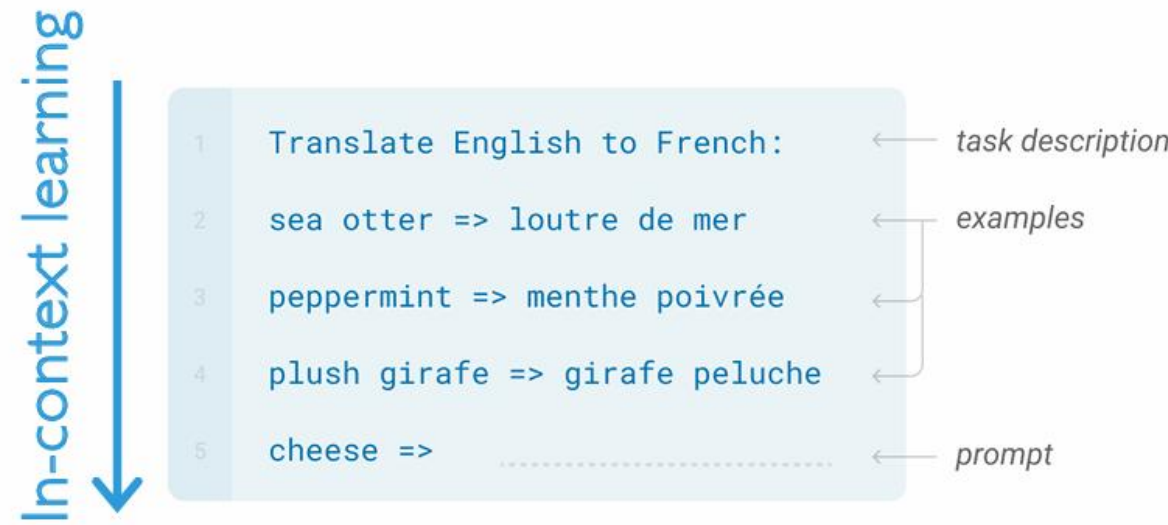
Mission Goal: **RIGHT**



上下文工程In-Context Engineering

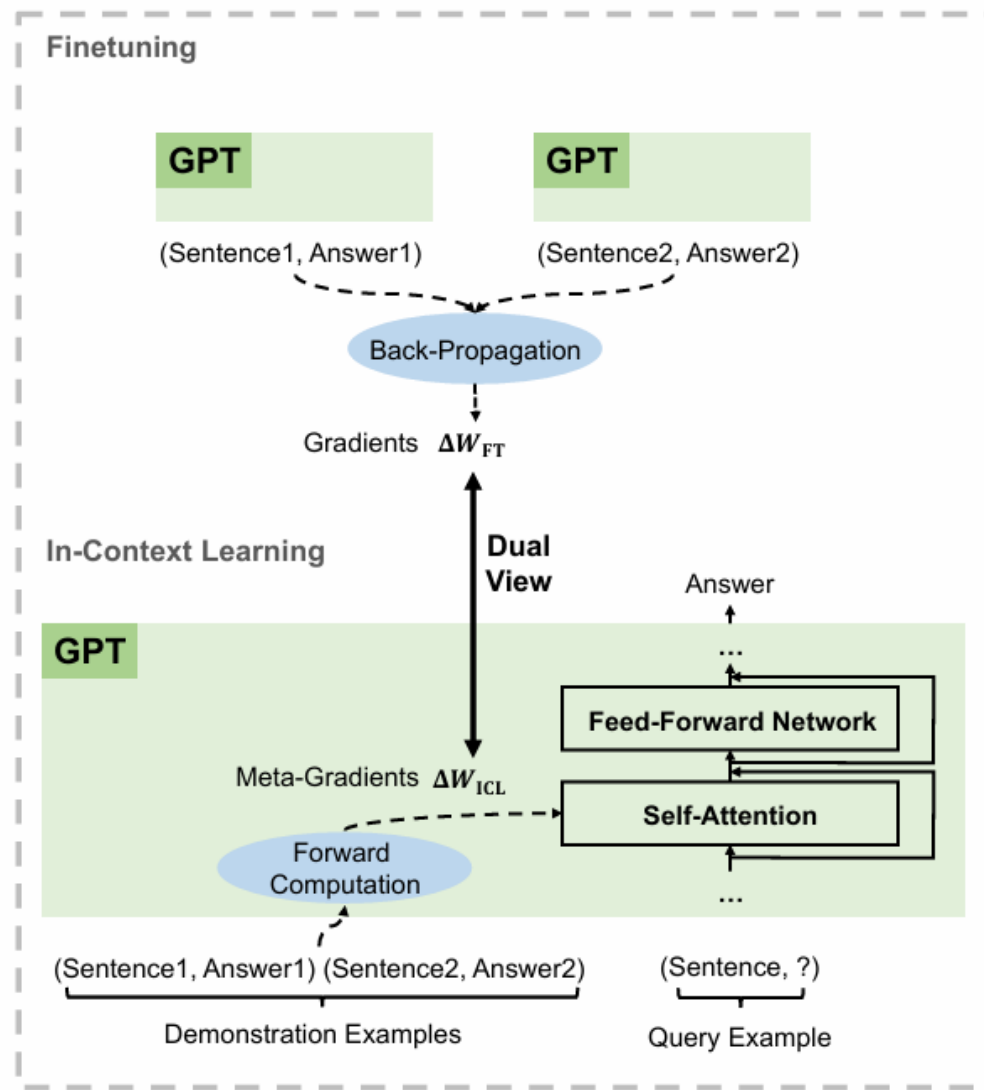
- [任务描述 + 示例1($f(x_1,y_1)$) + ... + 示例k($f(x_k,y_k)$) + 新输入($f(x_{k+1})$)] → [答案($\hat{y}_{k+1}$)];
- 示例的选择、排列顺序和格式都会极大地影响模型的表现。提供高质量、有代表性的示例是用好ICL的关键技巧。

ICL vs. 微调 (Fine-tuning) : 不需要任何参数更新。模型利用它在预训练阶段学到的丰富知识, 直接根据你提供的几个示例进行推理。它快速、灵活, 几乎是“零成本”地适应新任务。



上下文工程In-Context Engineering

- 为什么大模型能进行ICL?
- 任务识别 (Task Recognition) : 模型在预训练时“阅读”了海量的互联网文本, 这些文本中隐含了无数种任务和模式。ICL中的示例的作用, 是帮助模型激活并识别出它记忆中的某种特定任务 (例如“翻译”)。一旦模型通过示例推断出“用户现在想让我做翻译”, 它就会调用与之相关的知识来回答问题。
- 隐式微调 (Implicit Fine-tuning) : 这是更前沿的一种理论解释。研究者 (如PPT中引用的Dai et al.) 认为, Transformer模型 (LLM的基石) 的注意力机制 (Attention Mechanism) 在ICL中扮演了关键角色。



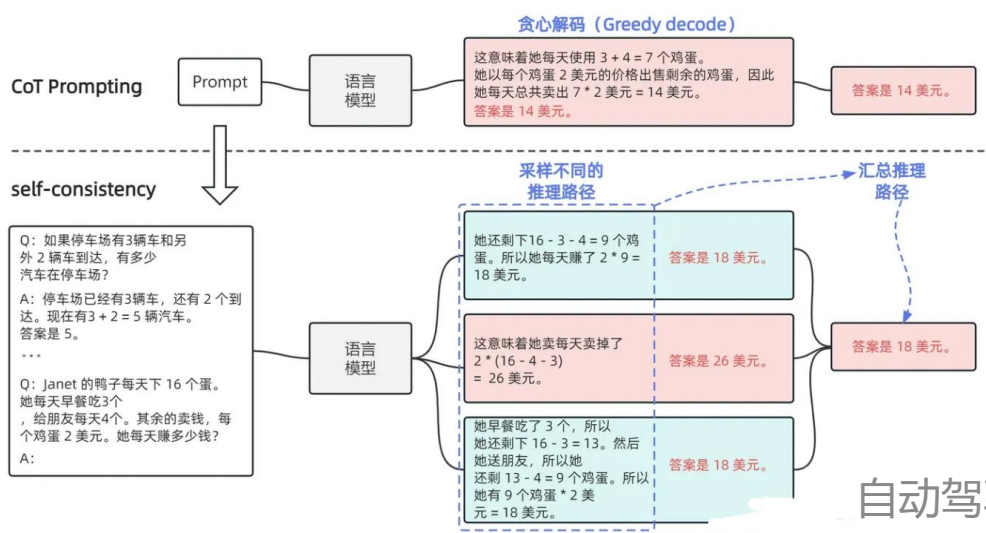
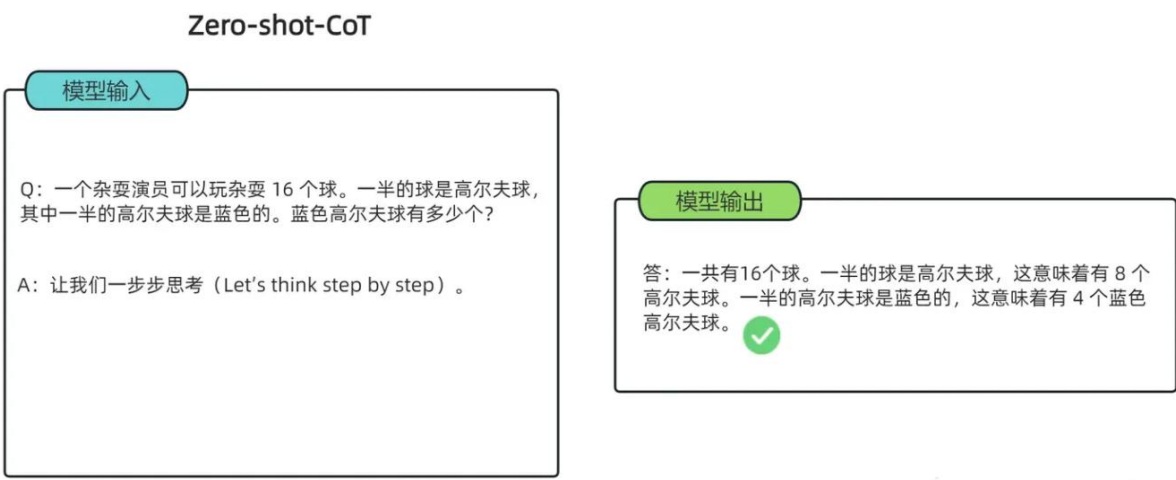
Xie et al. An Explanation of In-context Learning as Implicit Bayesian Inference. ICLR 2022.

Dai et al. Why Can GPT Learn In-Context? Language Models Secretly Perform Gradient Descent as Meta-Optimizers. 2022



思维链技术Chain-of-Thought

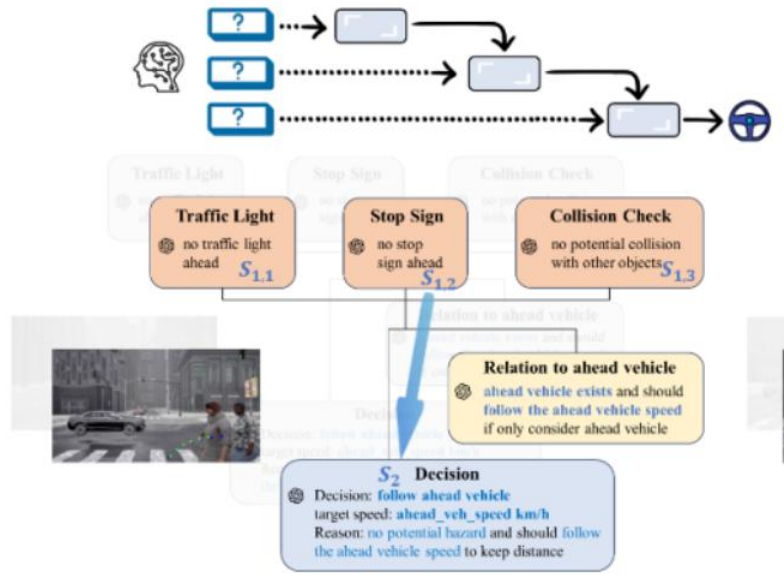
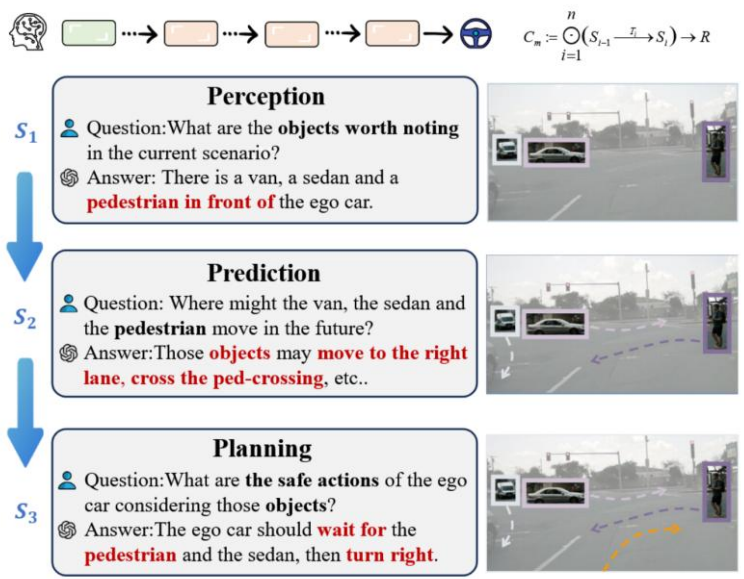
- 思维链技术，就是把一个多步骤推理问题，分解成很多个中间步骤，分配给更多的计算量，生成更多的 token，再把这些答案拼接在一起进行求解。
- 怎么实现CoT技术？ CoT技术的实现可以是一种特殊的提示词工程。
 - ❑ Zero-shot prompting engineering: Let 's think step by step!
 - ❑ 自洽性
 - ❑ LtM （Least to Most prompting）：问题分解为子问题，然后逐个解决。与前面的思维链不同的是，先前子问题的解决方案被输入到提示中，增强上下文，以尝试解决下一个问题。



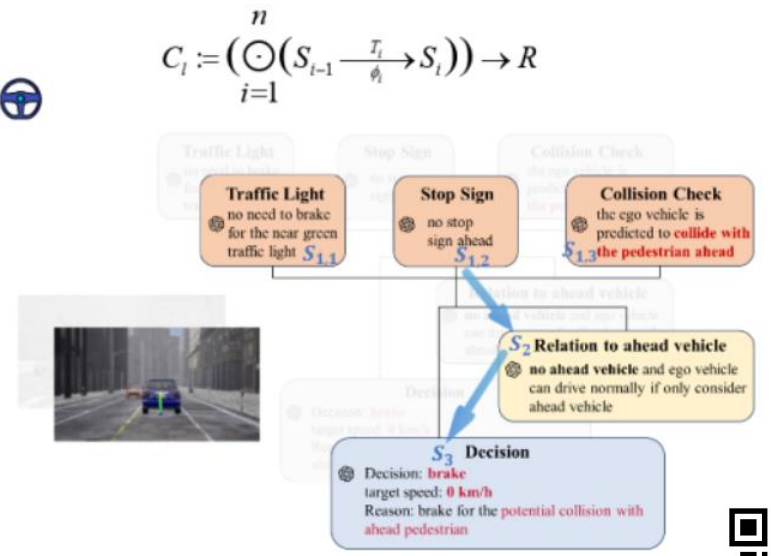
自动驾驶中的CoT技术

➤ CoT在自动驾驶的几大流派

- 模块化CoT: 遵循传统自动驾驶系统的层次结构，将整个驾驶任务拆解为感知、决策、规划、控制等独立子模块，各模块作为子推理单元独立执行并串行传递结果。
- 逻辑推理CoT: 逻辑推理范式在模块化结构基础上引入更强的逻辑一致性约束。系统不仅执行顺序思维步骤，同时也嵌入逻辑判断函数。确保推理路径在符合逻辑约束的同时完成状态转移。
- 反思性CoT



(a) Brake for potential collision with pedestrian



(b) Follow ahead vehicle

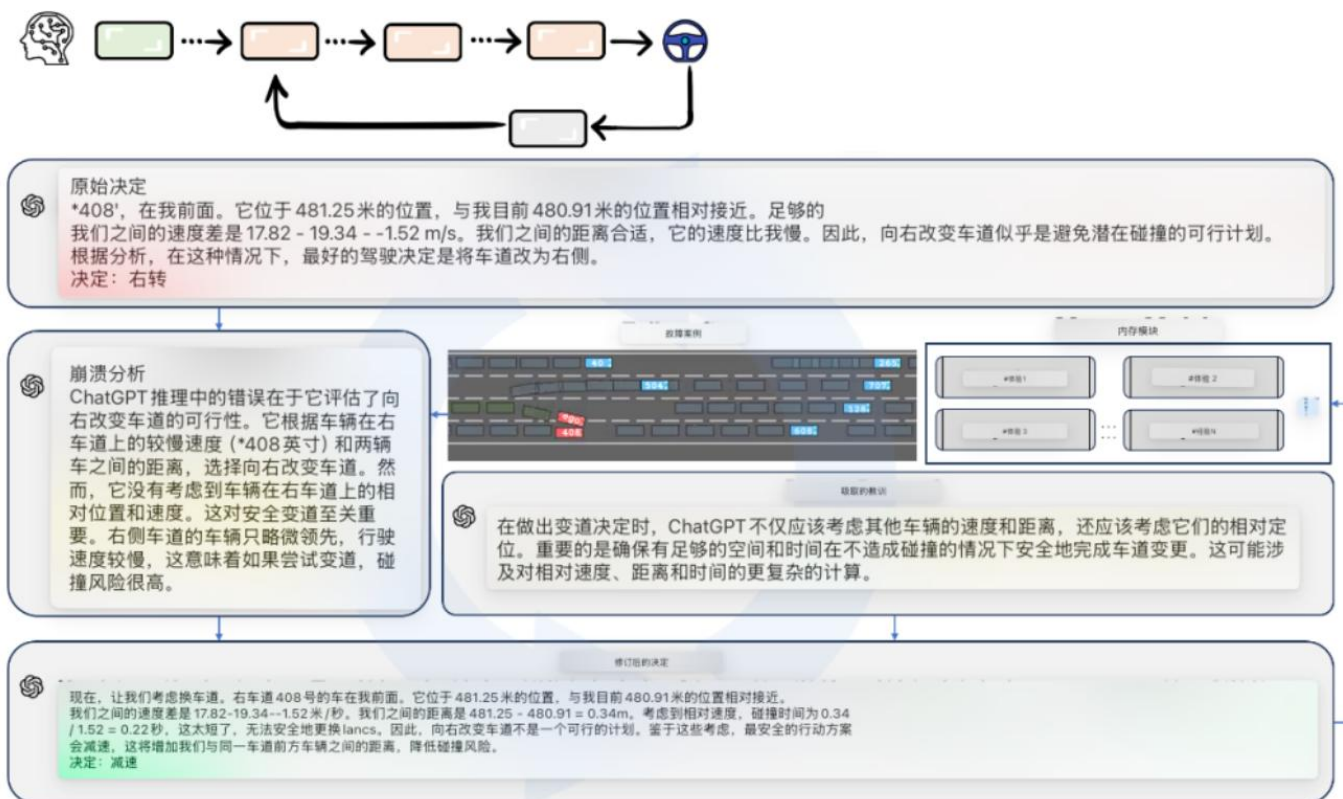


自动驾驶中的CoT技术

➤ CoT在自动驾驶的几大流派

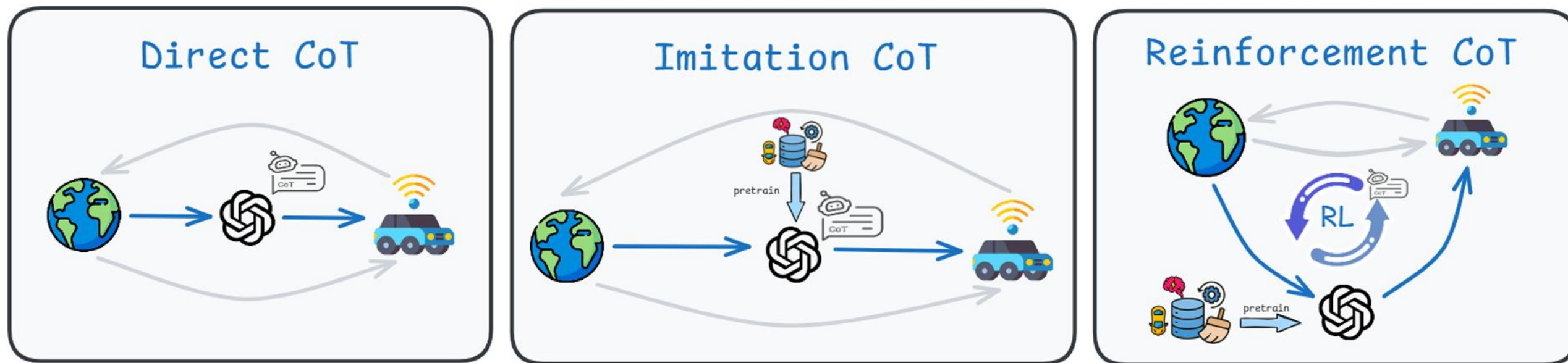
- 反思性CoT：扩展 CoT 推理链至双向闭环结构，融合实时反馈机制，实现系统自我修正与长期记忆回调。此结构体现出类人反思能力，系统在执行过程中可感知当前行为与期望输出之间的偏差，进而在记忆模块中存储决策链条、进而修正逻辑。

反思型思维链首先基于当前状态信息进行初步判断，生成原始决策，随后因推理不完善导致碰撞风险。在识别失败后，系统自动触发“反思机制”，调用记忆模块中的历史经验进行误判溯源与修正。



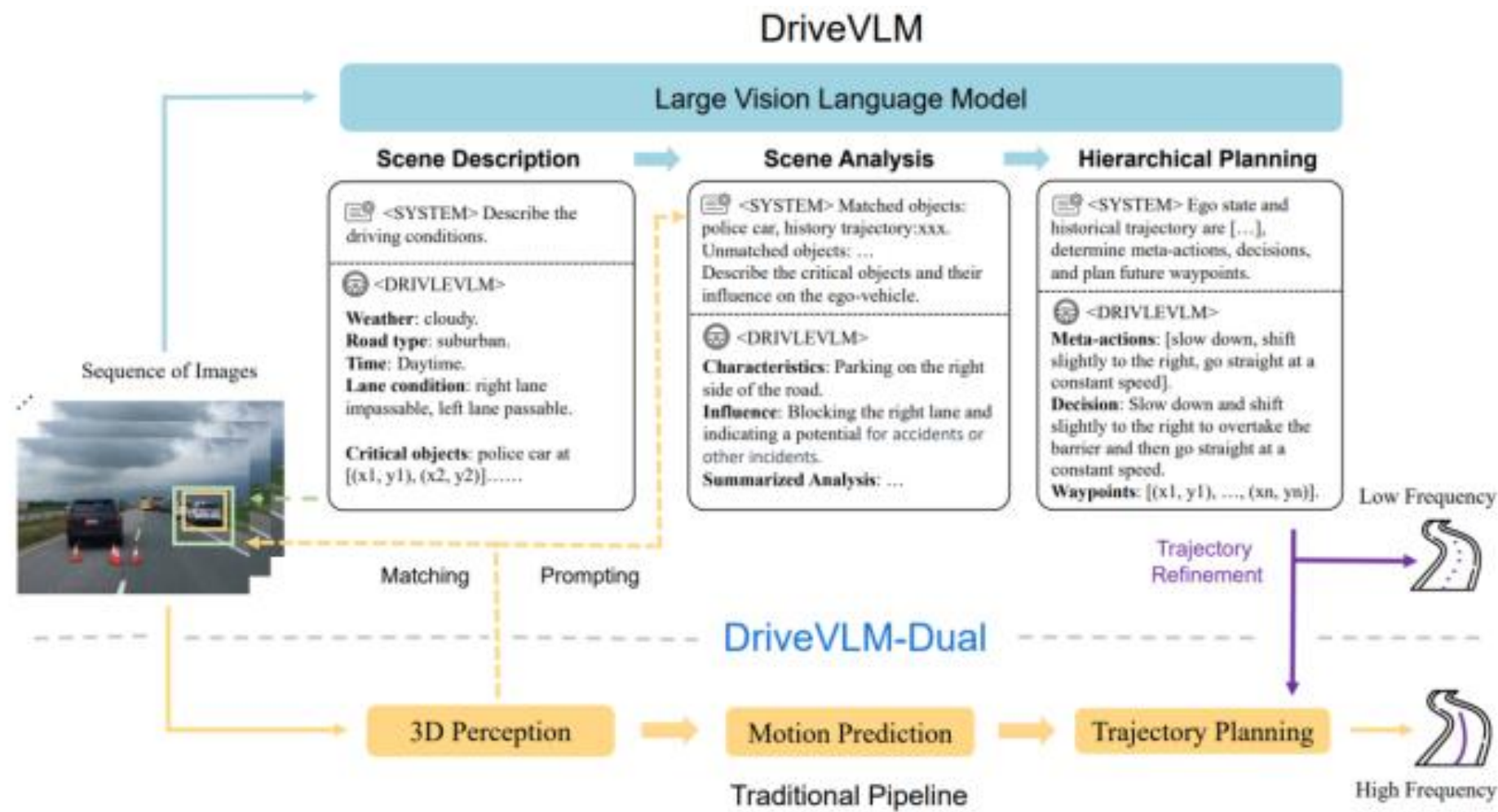
自动驾驶中的CoT技术

- 如何让自动驾驶系统具备CoT的能力？回顾一下大模型中CoT技术。
- 三种方法：直接CoT的提示词，通过模仿学习微调大模型学会推理，通过强化学习让模型学会推理（比如GRPO在DeepSeek的成功实践）



自动驾驶中的CoT技术

➤ 考考你，下面论文中CoT的技术在哪个部分？如何理解？

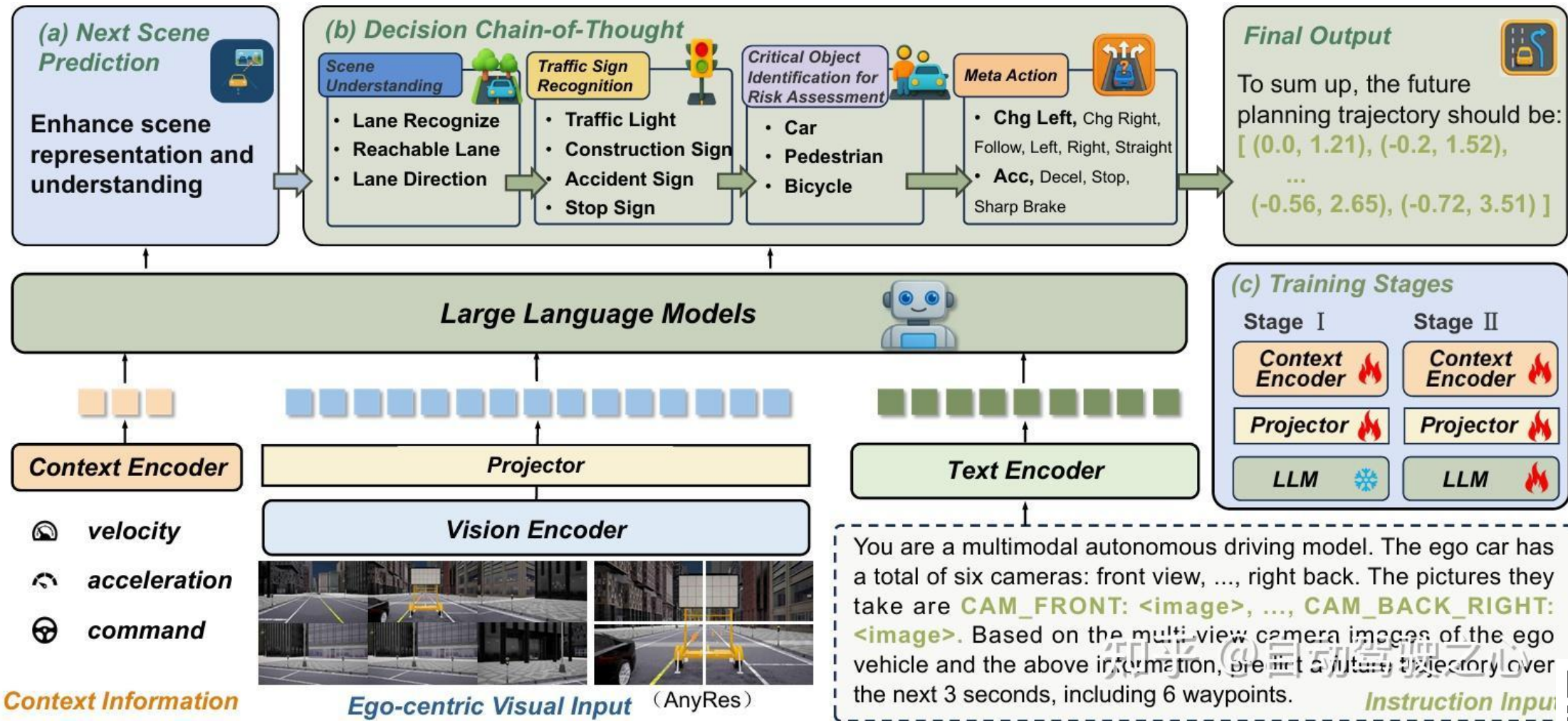


自动驾驶之心



自动驾驶中的CoT技术

➤ 考考你，下面论文中CoT的技术在哪个部分？如何理解？

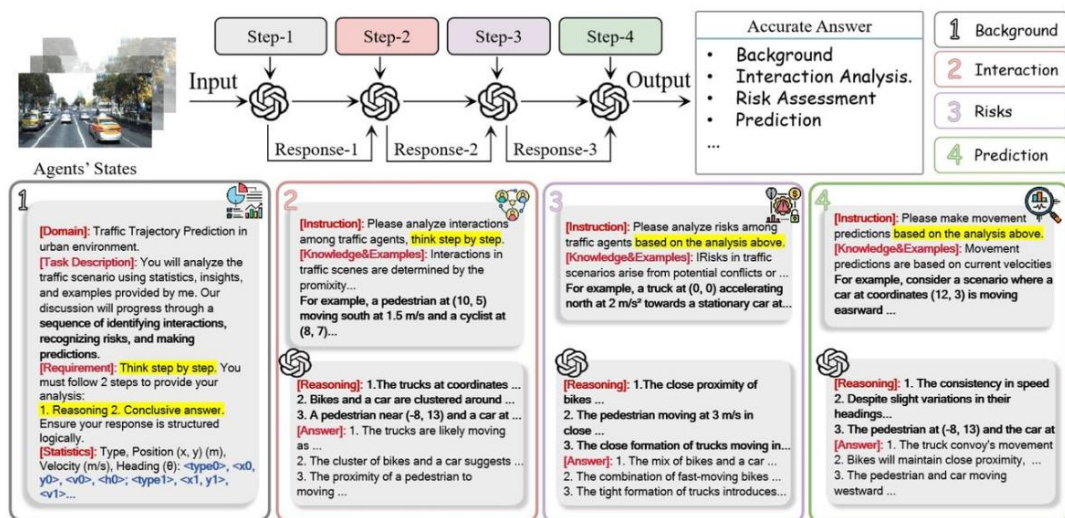
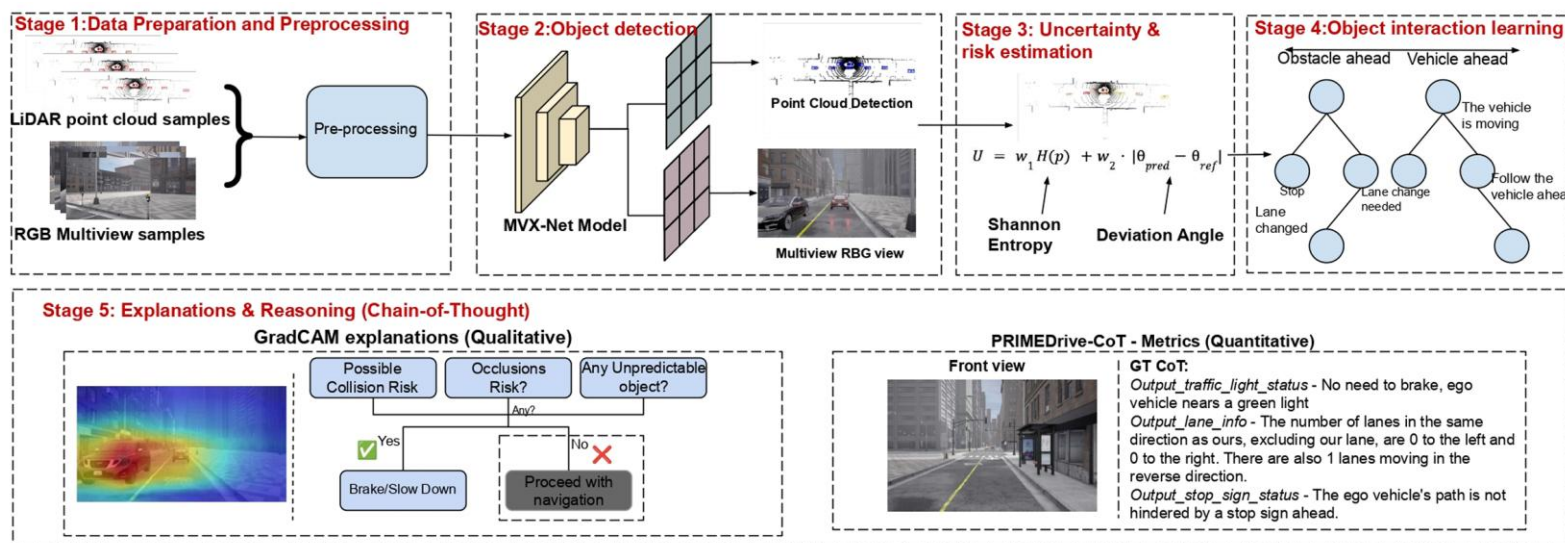


自动驾驶之心



自动驾驶中的CoT技术

➤ 考考你，下面论文中CoT的技术如何理解？



Question:

What actions could the ego vehicle take based on object at back right camera? Why?

Step 1

Need to find what is the exact nature and position of object mentioned around the ego vehicle.



[Get_Surrounding_Object_Detections]

continue reasoning

Step 2

I found obj "o1" (Truck). The predicted future trajectory of o1 relative to ego vehicle is needed.



[Get_Future_Trajectories_for_Objects]

continue reasoning

Step 3

What is the lane positioning of ego vehicle that affects possible maneuvers regarding object o1?



[Get_Current_Lane_Divider]

continue reasoning

Step 4

I can then decide what action ego vehicle should take based on the position and action of object o1.

conclude

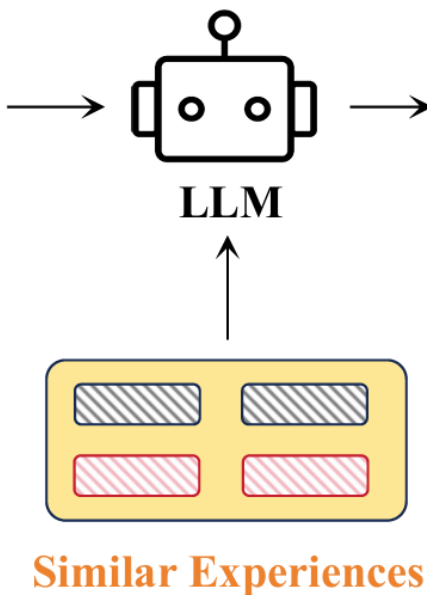
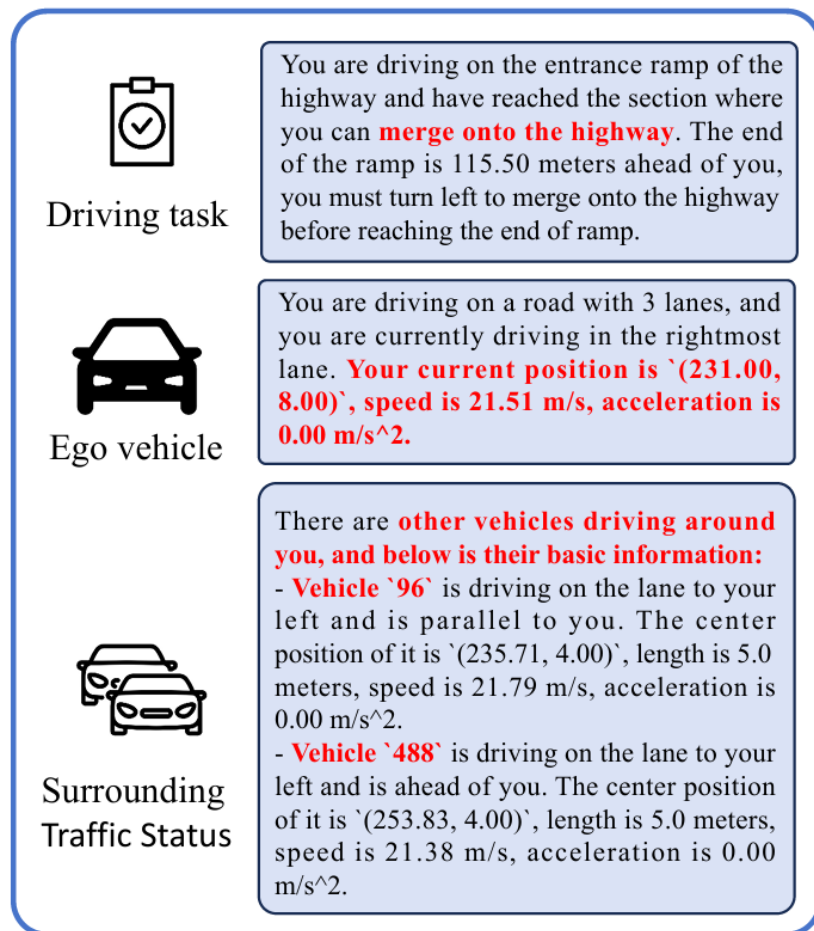
Final answer: The ego vehicle should maintain its speed or slightly increase it to ensure a safe distance from the object detected behind in the back right view (likely a truck) to avoid potential collisions.



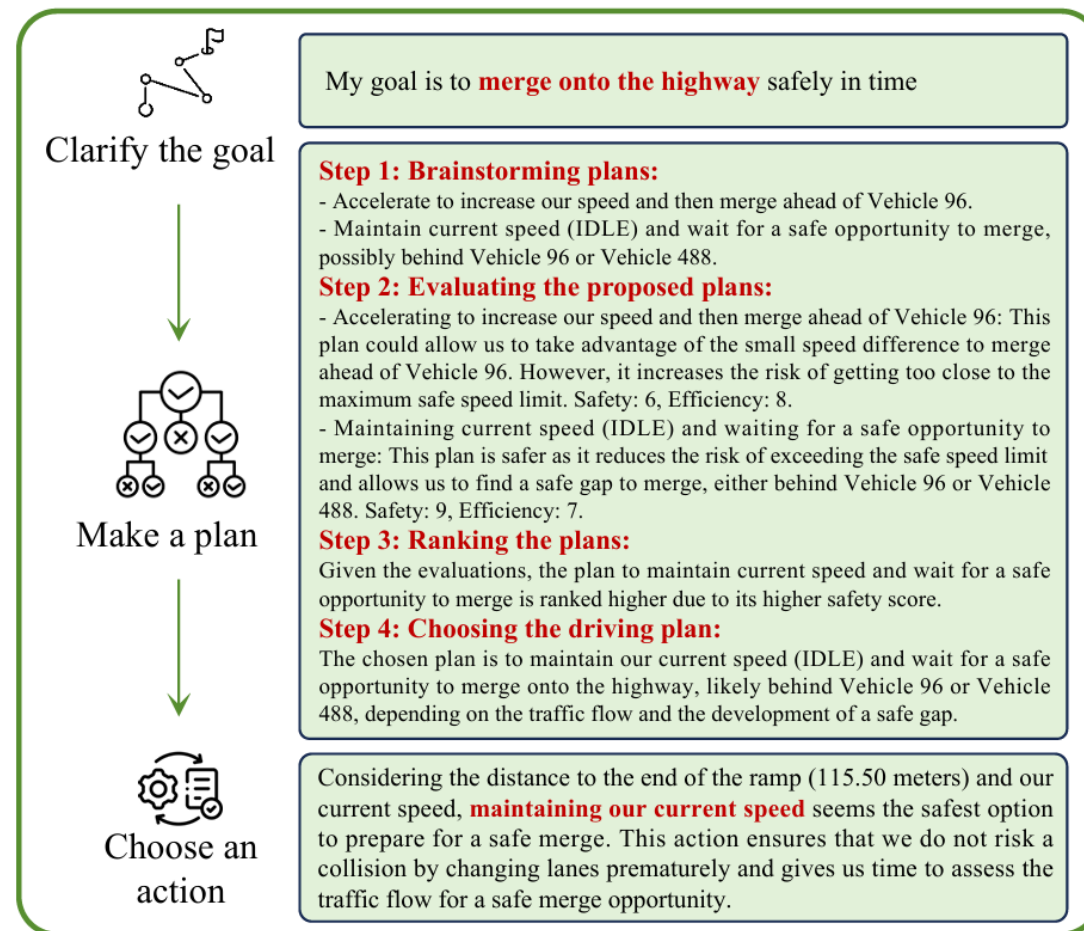
自动驾驶中的CoT技术

➤ 考考你，下面论文中CoT的技术在哪个部分？如何理解？

Scenario Description

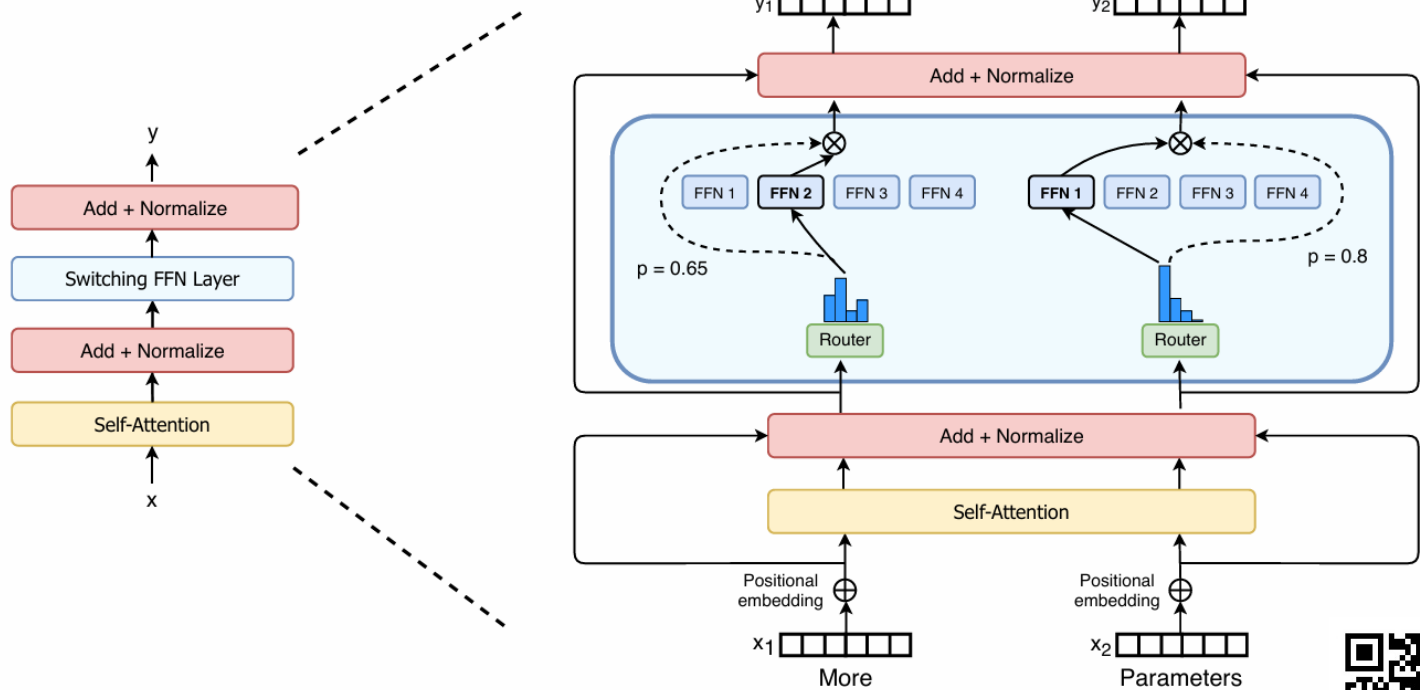


Chained Decision Making



MoE技术

- ❑ MoE，全称为Mixed Expert Models，翻译过来就是混合专家模型。
- ❑ MoE的最大优势就是与Dense模型相比，在相同计算资源下，训练速度更快，而且可以训练更大的模型。前向计算过程中，由于只激活了部分专家，实际参与计算的参数量被称为激活参数量；
- ❑ MoE具体如何实现？MoE 层由两个核心部分组成：一个门控网络和若干数量的专家。
- ❑ 一个典型的门控函数通常是一个带有 softmax 函数的简单的网络。

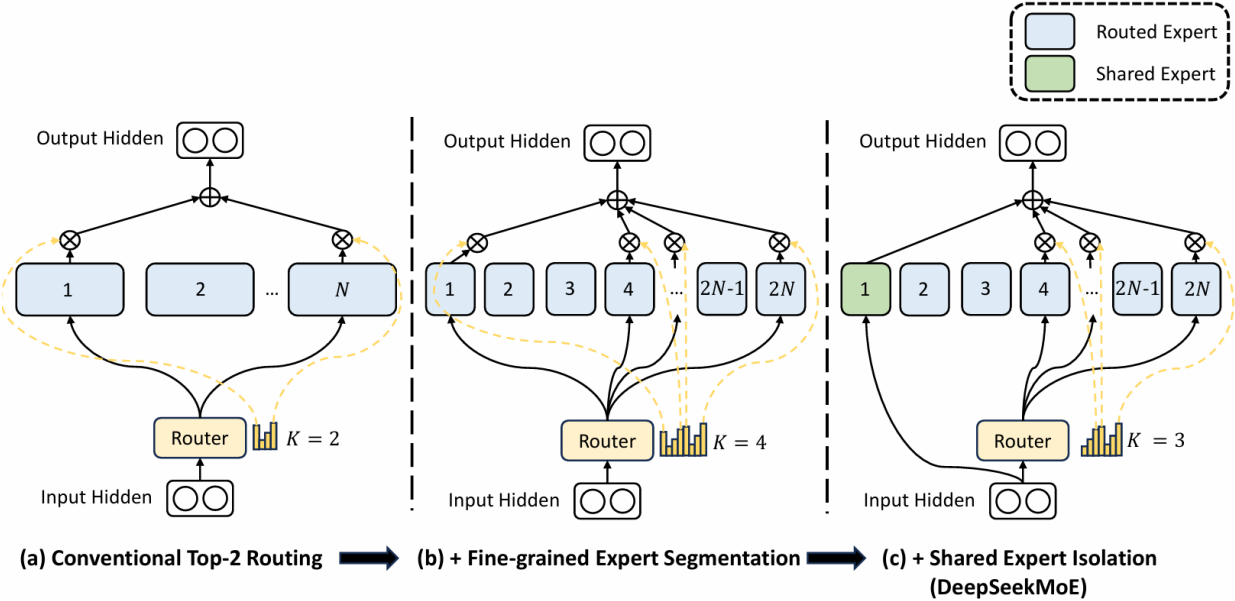


DeepSeekMoE技术

- ❑ 知识混杂：现有的MoE实践通常使用有限数量的专家(例如，8或16)，因此分配给特定专家的Token将可能覆盖不同的知识。因此，被指定的专家将打算在其参数中汇集不同类型的知识，这些知识很难同时使用。
- ❑ 知识冗余：分配给不同专家的Token可能需要共同的知识。因此，多个专家在获取他们各自参数的共享知识时可能会收敛，从而导致专家参数的冗余。

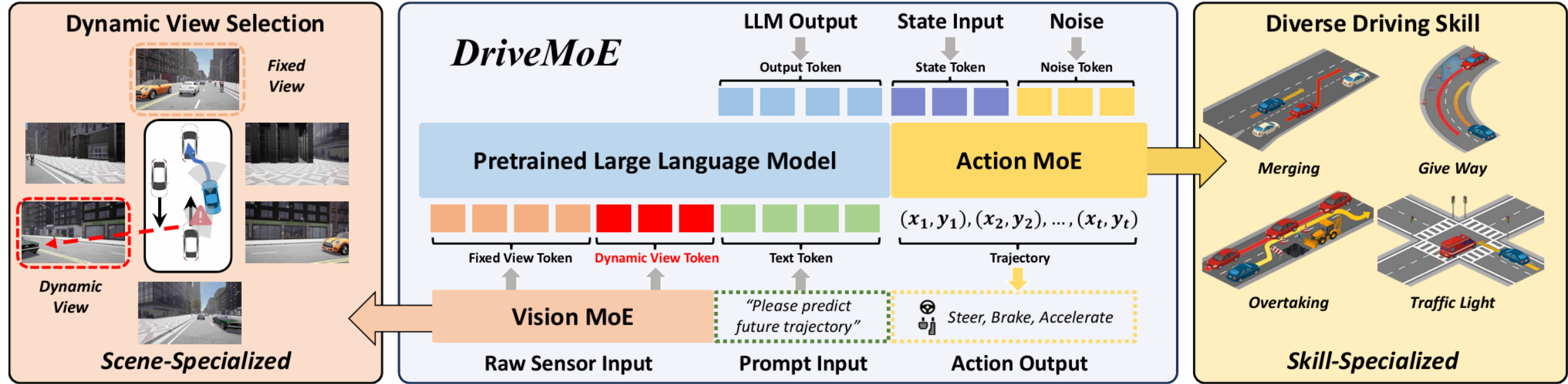
❑ DeepSeekMoE 架构：细粒度专家分割（Fine-Grained Expert Segmentation）和共享专家隔离（Shared Expert Isolation）

- 在保持专家总参数数量不变的情况下，通过将N个FFN中间隐藏维度缩小为原来的m分之1，即每个专家FFN分割成m个更小的专家，即总共得到mN个细粒度专家；
- 传统路由策略下，不同专家可能会学习相同的通用知识，造成参数冗余。而共享专家专门用于捕捉和整合不同上下文的通用知识，这样能使路由专家能更专注于独特知识的学习。



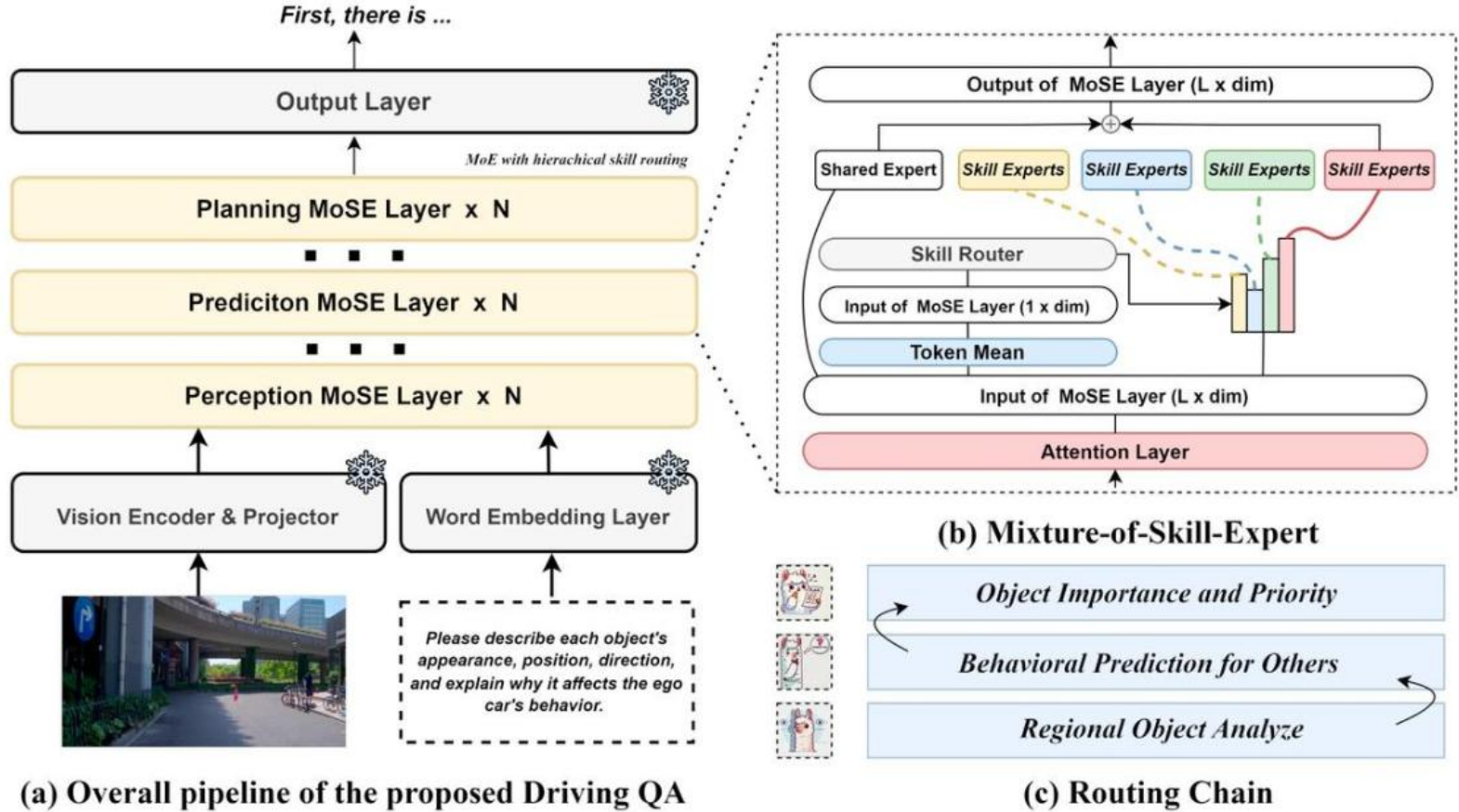
MoE技术在自动驾驶中的应用

- 考考你？下面是DriveMoE的框架图，具体哪些部分用到了MoE技术？
- MoE的思想应用于感知和决策层面：环境信息的提取，在不同的场景下，需要聚合的知识不同；在不同的驾驶行为下，比如超车、并道、紧急刹车下，所需的不同的驾驶技能也是不同的；



MoE技术在自动驾驶中的应用

- 考考你？下面是MoSE的框架图，具体哪些部分用到了MoE技术？
- MoE的思想应用于感知，预测和决策层面；



-
- The diagram illustrates the framework of the proposed Adaptive Driving Knowledge (ADK) module. It shows the flow of information from perception to action adaptation. Perception inputs (Original scene input, Ego States, Lidar Points, Navi Info) are processed into Observation (O_t). This observation is used in a Cross Attention mechanism with Action Emb (A_t^{K-1}) to produce Action Emb (A_t^K). The Action Emb is then used in a Mixture of Experts (MoE) structure, where a Top-K Router selects K experts (E_1 to E_n) based on a Gaussian distribution. These experts are used to adapt the driving knowledge (Driving Knowledge Corpus) for Interaction, Maneuvering, and Adaptation. The final output is the adapted driving knowledge, which is used to generate the next Action Emb (A_t^K) for the next iteration.

常用开源大模型的部署和使用

- ❑ 使用 Ollama 部署：交互式GUI
- ❑ 使用vllm部署：利用Openai的调用接口格式
- ❑ 使用 LM Studio 部署
- ❑ 使用源码部署（最推荐！）主要有Transformers和一些二次开发的框架：比如ms-swift和 llama-factory

 Models GitHub Discord Docs Cloud

[Sign in](#) [Download](#)



Cloud models are now available in Ollama

Chat & build with open models

[Download](#)

Available for macOS, Windows, and Linux

 LM Studio Models Docs Work Use

[Download](#) [Login](#)

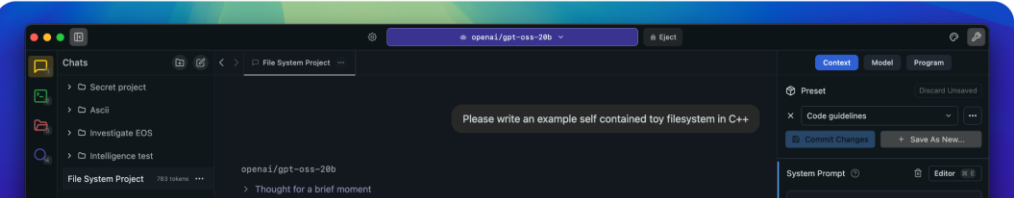
 LM Studio

Local AI, on your computer.

Run local AI models like `gpt-oss`, `Qwen`, `Gemma`, `DeepSeek` and many more on your computer, privately and for free.

 [Download for Windows](#) 0.3.26

LM Studio is free for home and work use • [terms](#)



大模型本地部署保姆级教程！三种方法教会你如何部署~本地部署大模型-CSDN博客

自动驾驶之心



常用开源大模型的部署和使用

使用源码部署（最推荐！）

❑ Qwen官方仓库使用Transformers进行部署

❑ 二次开发的框架：ms-swift有独立的推理API

```
from transformers import Qwen2_5_VLForConditionalGeneration, AutoProcessor
from qwen_vl_utils import process_vision_info
```

default: Load the model on the available device(s)

```
model = Qwen2_5_VLForConditionalGeneration.from_pretrained(
    "Qwen/Qwen2.5-VL-7B-Instruct", torch_dtype="auto", device_map="auto"
)
```

default processor

```
processor = AutoProcessor.from_pretrained("Qwen/Qwen2.5-VL-7B-Instruct")
```

```
messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "image",
                "image": "https://qianwen-res.oss-cn-beijing.aliyuncs.com/Qwen-
VL/assets/demo.jpeg",
            },
            {"type": "text", "text": "Describe this image."},
        ],
    }
]
```

Preparation for inference

```
text = processor.apply_chat_template(
    messages, tokenize=False, add_generation_prompt=True
)
image_inputs, video_inputs = process_vision_info(messages)
inputs = processor(
    text=[text],
    images=image_inputs,
    videos=video_inputs,
    padding=True,
    return_tensors="pt",
)
inputs = inputs.to(model.device)
```

Inference: Generation of the output

```
generated_ids = model.generate(**inputs, max_new_tokens=128)
generated_ids_trimmed = [
    out_ids[len(in_ids) :] for in_ids, out_ids in zip(inputs.input_ids,
generated_ids)
]
output_text = processor.batch_decode(
    generated_ids_trimmed, skip_special_tokens=True,
    clean_up_tokenization_spaces=False
)
print(output_text)
```



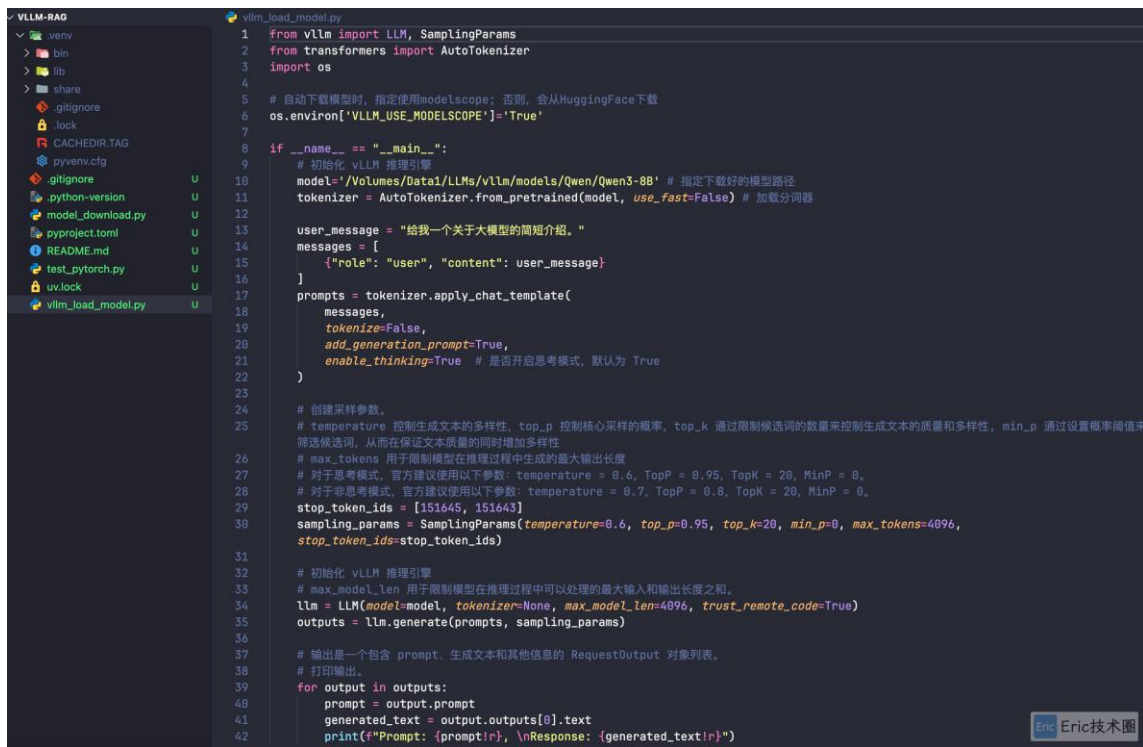
常用开源大模型的部署和使用

使用源码部署（最推荐！）

❑ vllm

❑ 二次开发的框架：ms-swift有独立的推理API

找一找，导入模型，导入数据和模型对输入的数据如何进行推理？



```
1 from vllm import LLM, SamplingParams
2 from transformers import AutoTokenizer
3 import os
4
5 # 自动下载模型时，指定使用modelscope: 否则，会从HuggingFace下载
6 os.environ['VLLM_USE_MODELSCOPE']='True'
7
8 if __name__ == "__main__":
9     # 初始化 vLLM 推理引擎
10    model = "/Volumes/Data1/LLMs/vllm/models/Qwen/Qwen3-8B" # 指定下载好的模型路径
11    tokenizer = AutoTokenizer.from_pretrained(model, use_fast=False) # 加载分词器
12
13    user_message = "给我一个关于大模型的简短介绍。"
14    messages = [
15        {"role": "user", "content": user_message}
16    ]
17    prompts = tokenizer.apply_chat_template(
18        messages,
19        tokenize=False,
20        add_generation_prompt=True,
21        enable_thinking=True # 是否开启思考模式，默认为 True
22    )
23
24    # 创建采样参数。
25    # temperature 控制生成文本的多样性，top_p 控制核心采样的概率，top_k 通过限制候选词的数量来控制生成文本的质量和多样性，min_p 通过设置概率阈值来
26    # 筛选候选词，从而在保证文本质量的同时增加多样性
27    # max_tokens 用于限制模型在推理过程中生成的最大输出长度
28    # 对于思考模式，官方建议使用以下参数：temperature = 0.6, TopP = 0.95, TopK = 20, MinP = 0.
29    # 对于非思考模式，官方建议使用以下参数：temperature = 0.7, TopP = 0.8, TopK = 20, MinP = 0.
30    stop_token_ids = [151645, 151643]
31    sampling_params = SamplingParams(temperature=0.6, top_p=0.95, top_k=20, min_p=0, max_tokens=4096,
32    stop_token_ids=stop_token_ids)
33
34    # 初始化 vLLM 推理引擎
35    # max_model_len 用于限制模型在推理过程中可以处理的最大输入和输出长度之和。
36    llm = LLM(model=model, tokenizer=None, max_model_len=4096, trust_remote_code=True)
37    outputs = llm.generate(prompts, sampling_params)
38
39    # 输出是一个包含 prompt、生成文本和其他信息的 RequestOutput 对象列表。
40    # 打印输出。
41    for output in outputs:
42        prompt = output.prompt
43        generated_text = output.outputs[0].text
44        print(f"Prompt: {prompt}\nResponse: {generated_text}")
```

qwen-vl-chat

```
import os
os.environ['CUDA_VISIBLE_DEVICES'] = '0'

from swift.llm import (
    get_model_tokenizer, get_template, inference, ModelType, get_default_template_type,
)
from swift.utils import seed_everything

model_type = ModelType.qwen_vl_chat
template_type = get_default_template_type(model_type)
print(f'template_type: {template_type}') # template_type: qwen

model, tokenizer = get_model_tokenizer(model_type, model_kwargs={'device_map': 'auto'})

template = get_template(template_type, tokenizer)
seed_everything(42)
query = '<image>What is this'
images = ['https://qianwen-res.oss-cn-beijing.aliyuncs.com/Qwen-VL/assets/demo.jpeg']
response, history = inference(model, template, query, images=images)
print(f'query: {query}')
print(f'response: {response}')
query = 'Output the bounding box for the high-five'
response, history = inference(model, template, query, history, images=images)
print(f'query: {query}')
print(f'response: {response}')
print(f'history: {history}')

def _fetch_latest_picture(*args, **kwargs):
    return images[0]

tokenizer._fetch_latest_picture = _fetch_latest_picture
image = tokenizer.draw_bbox_on_latest_picture(response, history)
image.save('output_chat.jpg')
```

[LLM Inference Documentation — swift 2.6.1 documentation](#)

[vLLM 本地部署大模型快速入门 - Eric技术圈](#)

自动驾驶之心



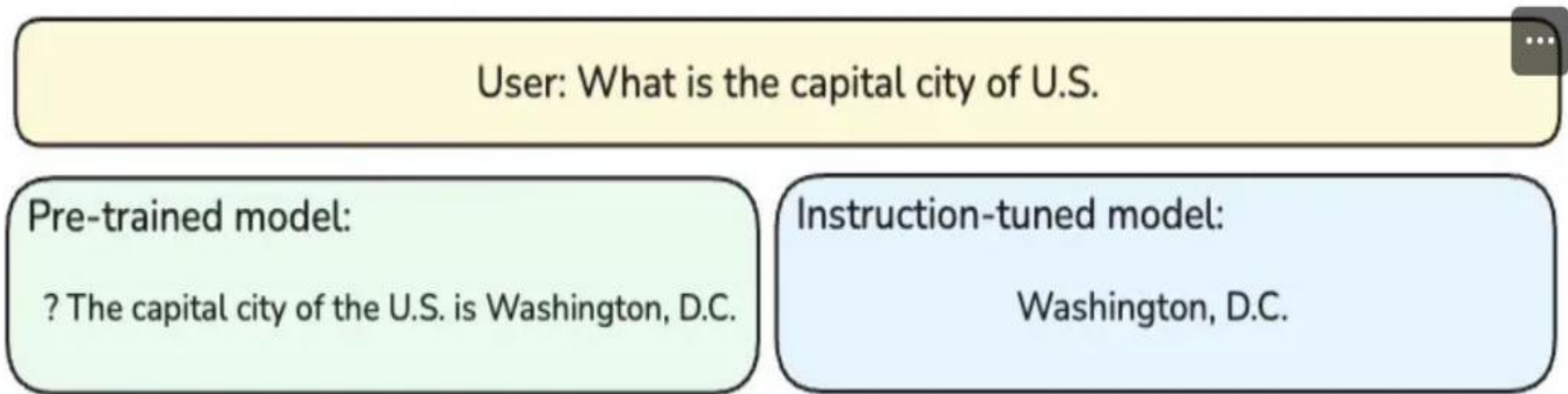


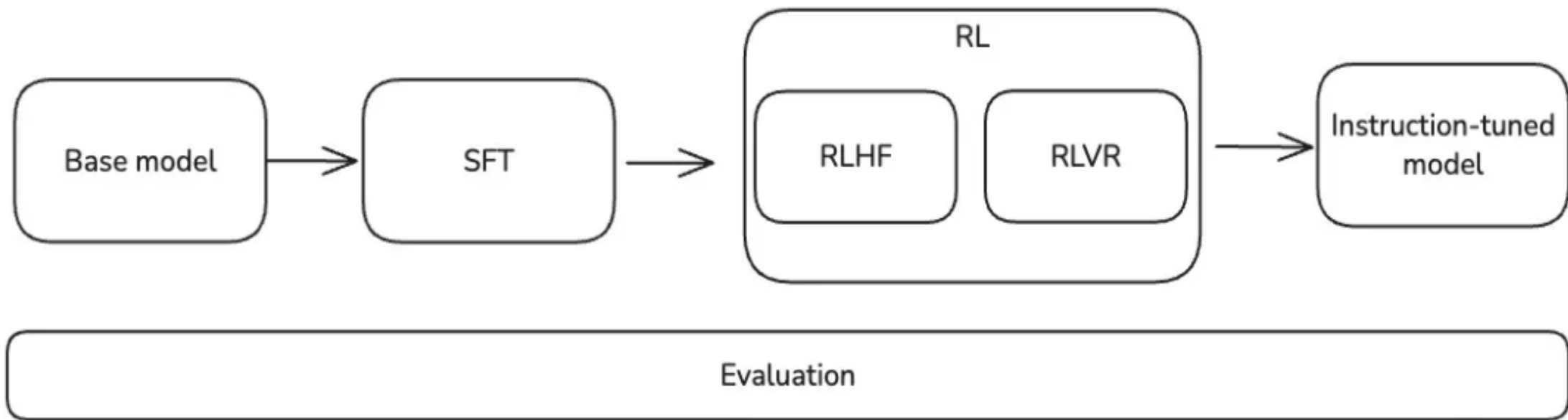
Figure 1. The journey from pre-training to an instruct-tuned model.

□ **预训练**: 让模型学习预测在给定前文序列的情况下，下一个 token 是什么

如何让 LLM 在问答场景中真正发挥作用？

□ **后训练**: 对模型的行为**进行对齐**（如诚实、无害），并进一步强化模型在预训练阶段积累的能力。





□ 监督微调（SFT） & 基于人类反馈的强化学习（RLHF）

□ 基于可验证奖励的强化学习（RLVR）

VR：可验证奖励（也称规则奖励或准确率奖励）



训练范式——监督微调

➤ 什么是监督微调（SFT）？

它将一个在预训练过程中已经获得大量知识的模型，转化为能够遵循用户指令并具有普遍实用性的模型

🔍 DATA EXAMPLES

User prompt:

What is the capital city of U.S.

Response:

The capital city of the United States is **Washington, D.C.**

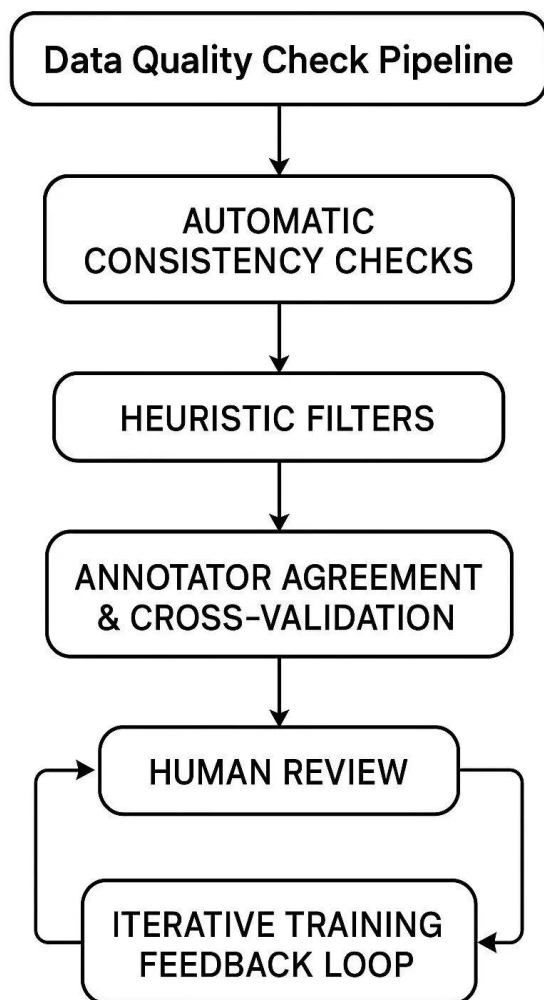
□ 监督学习：

模型在给定前缀的情况下生成后续的一组 token，然后通过目标 token 的交叉熵损失对其进行优化。



训练范式——监督微调

➤ SFT 的效果取决于背后的数据集质量。



❑ SFT 数据集通常规模较小（约 1 万到 10 万个样本），且经过高度筛选。这使得它们对缺陷极为敏感

❑ SFT 数据集中最常见的问题可以分为三类：

- 标签噪声：可能错误、不完整或不一致的答案。
- 分布不匹配：数据集范围过窄时（如只包含数学题或简短回答）
- 伪推理：包含逻辑漏洞、误导性的捷径，甚至是直接复制粘贴的模板

❑ 实践中，最有效的方法是形成一个反馈循环：收集数据、进行质量过滤、评估模型行为，并不断迭代优化数据集，直到 SFT 阶段能够为后续的偏好优化奠定坚实基础。



训练范式——监督微调

➤ SFT 损失函数——负对数似然函数

□ SFT 本质上就是一个多分类问题 (number of classes = vocab_size)

$$\mathcal{L}_{SFT}(\theta) = -\mathbb{E}_{(x,y) \sim \mathcal{D}} \sum_{t=1}^T \log p_{\theta}(y_t | x, y_{<t})$$

- θ : 模型参数
- y_t : 目标回答中的第 t 个 token
- $y_{<t}$: 前缀 (即之前的 token)

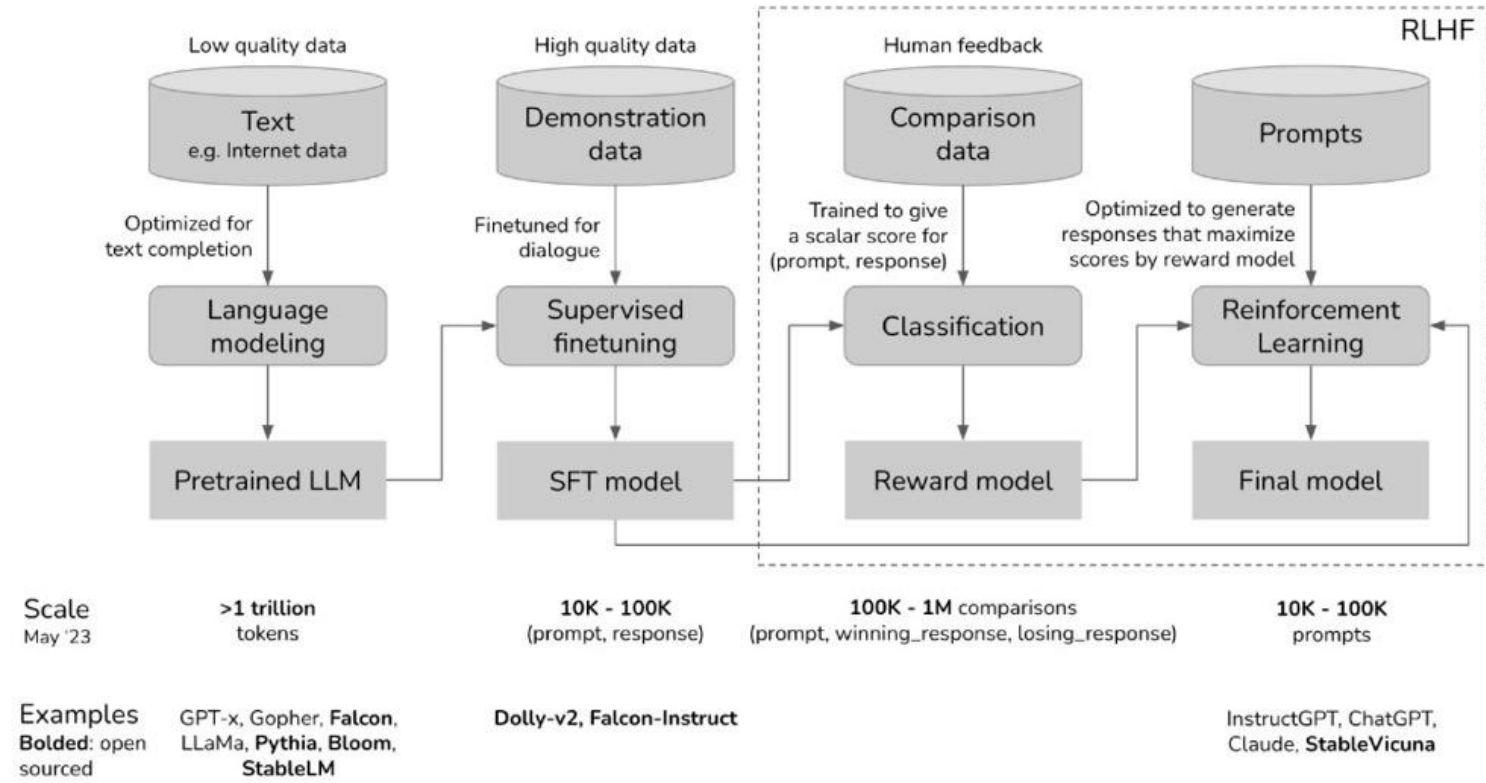
□ 实践中, 通过交叉熵函数来实现

$$\mathcal{L}_{SFT}(\theta) = -\frac{1}{T} \sum_{t=1}^T \left(z_{t,y_t^*} - m - \log \sum_{v=1}^V \exp(z_{t,v} - m) \right)$$

- v : 词表大小
- z_t : 序列中第 t 个 token 在最后一层的 logits



训练范式——强化学习后训练



GPT的开发过程

□SFT的局限性:

- 高质量的 SFT 数据集构建成本高昂;
- 人类的偏好是复杂且主观的, 很难用唯一的 “正确答案” 来概括。

当人们拥有易于产生但难以形式化和自动化的复杂直觉 (complex intuitions) 时, 与其他技术相比, 人类反馈 (HF) 预计将具有最大的比较优势。

——Anthropic

PG (policy gradient) —> TRPO—> PPO—> DPO—> GRPO

□ 最大的区别在于策略数据采样的在线或离线的差异。



训练范式——强化学习后训练

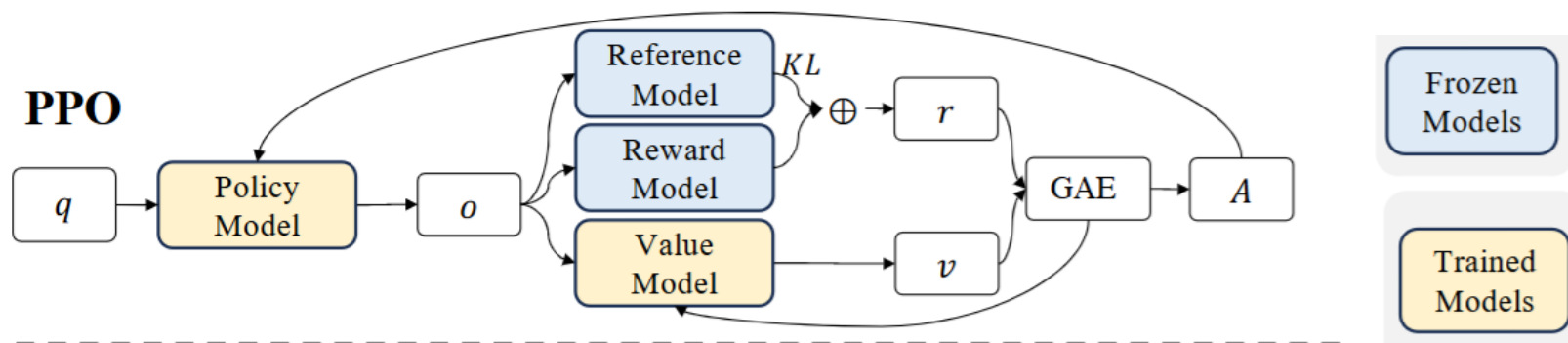
□RLHF的优化目标

$$\max_{\pi} \mathbb{E}_{x \sim D, y \sim \pi} [r(x, y)] - \beta D_{KL} [\pi(y|x) \parallel \pi_{\text{ref}}(y|x)]$$

- RL的优化目标是最大化奖励，最小化KL散度
- 左半部分就是尽可能得到高奖励
- 右半部分防止模型偏离基准模型分布远，保证生成多样性
- X是给定提示指令，y是响应，r(x, y)就是评估质量
- 基准模型是参考模型，一般是SFT之后的模型；
- 训练模型是要进一步RLHF后的模型



训练范式——PPO



□ PPO组成部分:

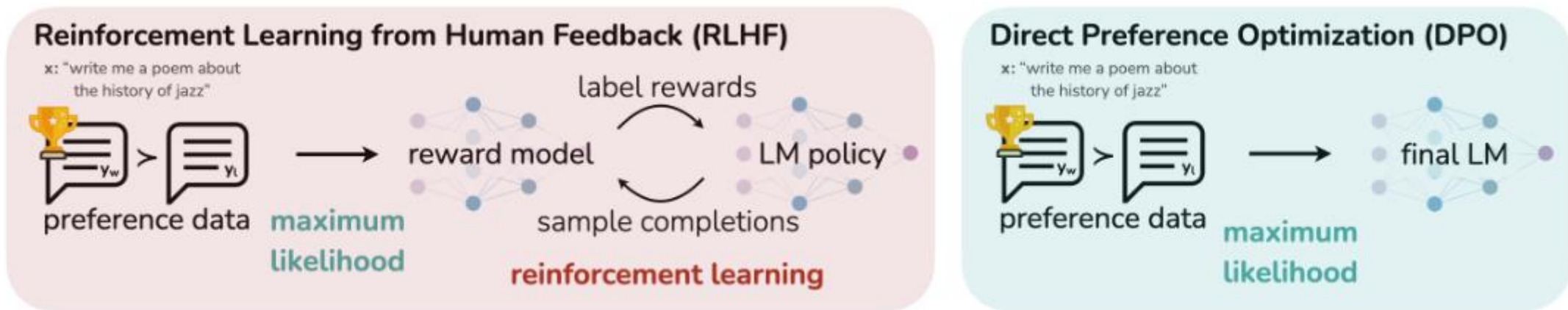
1. Policy: 已预先训练/SFT 的 LLM;
2. Reward model: 一个经过训练和冻结的网络, 在对提示做出完全响应的情况下提供标量奖励;
3. Critic: 也称为值函数, 一个可学习的网络

□ 具体工作流程:

1. Generate responses: LLM 为给定的prompt生成多个response;
2. Score responses: reward model 给每个 response 分配 reward;
3. Compute advantages: 使用 GAE 计算 advantages (it' s used for training the LLM);
4. Optimise policy: 通过优化总目标来更新 LLM;
5. Update critic: 训练 value function以更好地预测给定部分响应的奖励。



训练范式——DPO

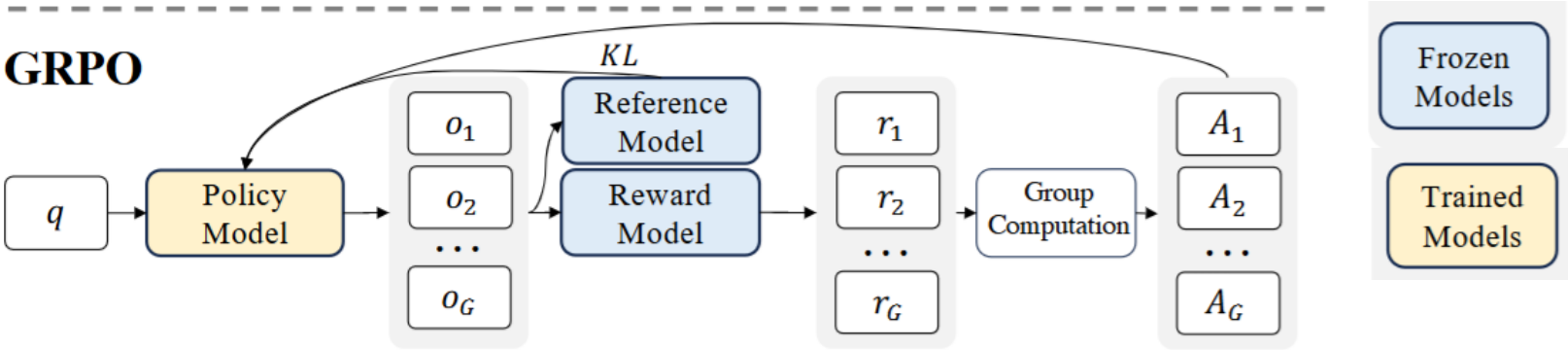


RLHF VS DPO

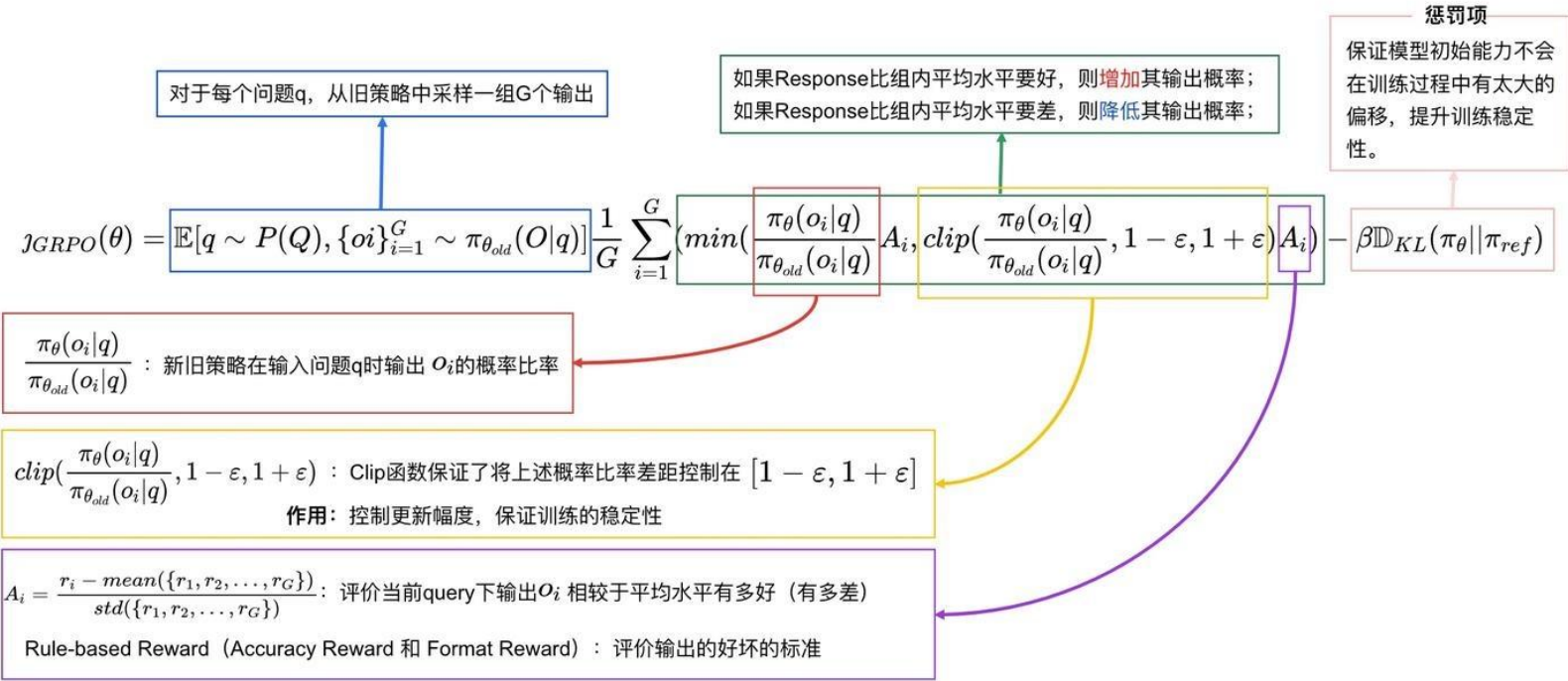
- ❑ 通过引入人类偏好数据，将在线策略优化，修改为通过二元交叉熵直接拟合人类偏好数据的离线策略。 $r(x, y) = \beta \ln \frac{\pi(x, y)}{\pi_{ref}(x, y)} + \beta \ln Z(x)$, $Z(x)$ 是归一化常数
 - 训练流程短：根据预先给定的偏好数据直接进行学习，属于离线策略，不需要进行在线数据采样
 - 稳定性高：DPO 属于有监督学习（通过概率匹配直接优化策略），摆脱了强化学习由于高方差带来的不稳定（由于奖励稀疏 or 噪声造成）
- ❑ 不足：
 - 容易过拟合：缺少 reward model 的泛化，因此容易直接拟合人类偏好数据
 - 需求更大标注数据量：效果表现更依赖标注数据量



训练范式——GRPO



通过最大化GRPO目标函数，让策略模型（此处即LLM）逐渐提升好的Response的概率，降低生成不好Response的概率。



GRPO相较于PPO:

- 消除了对价值模型的需求

为每个查询生成多个响应。然后，在计算advantage时，它将 value 函数替换为样本的奖励，该奖励由同一查询的所有响应的 mean 和 std 标准化。此外，它还将 KL 惩罚移动到损失函数中（RLHF 通常将 KL 惩罚添加到奖励中），从而简化了优势的计算

- 基于相对优势

GRPO 关注的是组内样本之间的相对优势，而非绝对的奖励值。





05

自动驾驶VLA数据集制作

数据集制作的一般过程

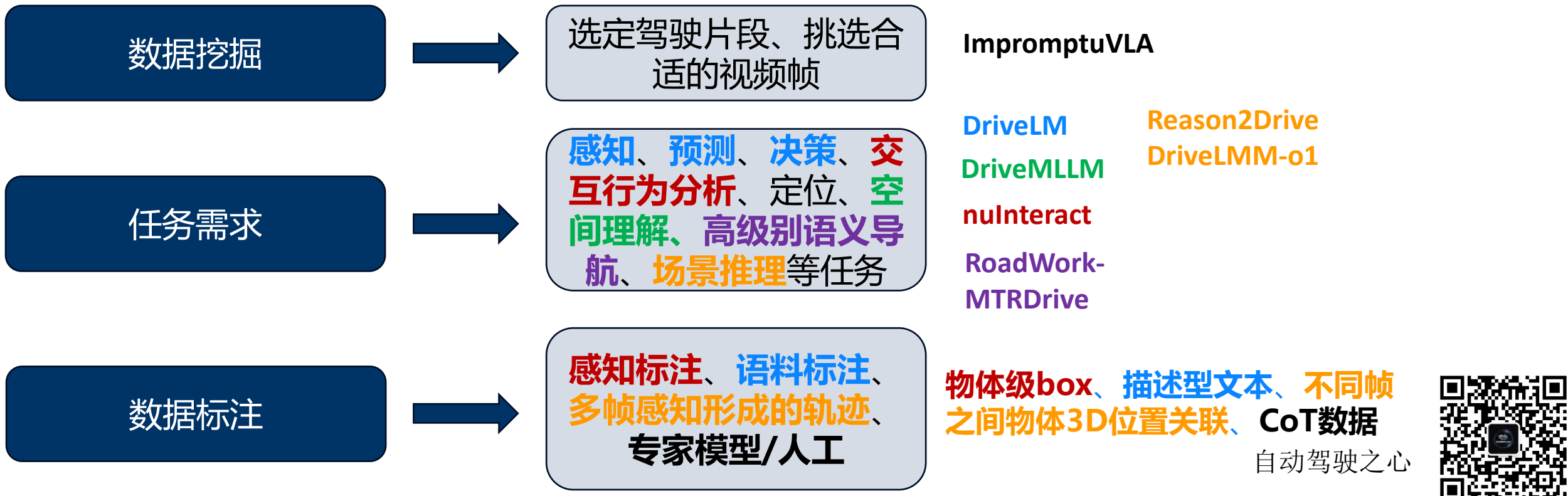
- 制作数据本质上是需要模型的输入和输出信息
- 图片分类任务——输入：图片，输出：类别标签；
- 时间序列预测任务——输入：某个时间片段的数据，输出：未来一定时间片段的数据
- 自动驾驶目标检测任务——输入：图片/点云/相机参数等，输出：场景内物体的3D框坐标等
- 自动驾驶地图检测任务——输入：图片/点云/相机参数等，输出：场景内地图元素的实例点集合
- 自动驾驶占据检测任务——输入：图片/点云/相机参数等，输出：场景内占据网格的体素场/BEV场
- 自动驾驶轨迹预测任务——输入：图片/点云/相机参数/历史轨迹等，输出：场景内实例的未来轨迹点坐标

VLA的数据怎么制作？



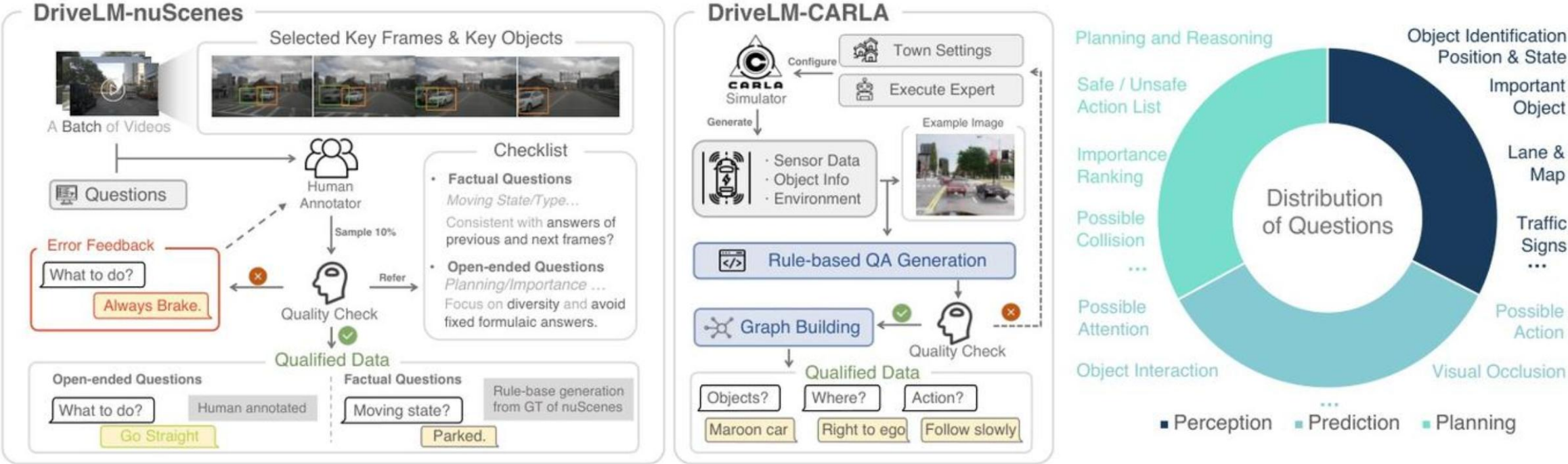
自动驾驶VLA数据集制作——如何制作大模型需要的语料？

- ❑ 制作大模型的语料任务就是要设计一系列（图片）问题-答案对；输入：图片/指令；输出：答案/动作
- ❑ 考虑到VLA系统的输入和输出，对于视觉，怎么选特定的数据片段？对于语言，怎么设计问题模板？
对于语言输出和动作输出，怎么制作合适的自然语言回答和对应的轨迹或者控制信号？
- ❑ 破题，怎么选数据对应的是**数据挖掘**问题，对于问题模板的设计是**任务需求**的问题，对于怎么得到合适的回答，是**数据标注**的问题。



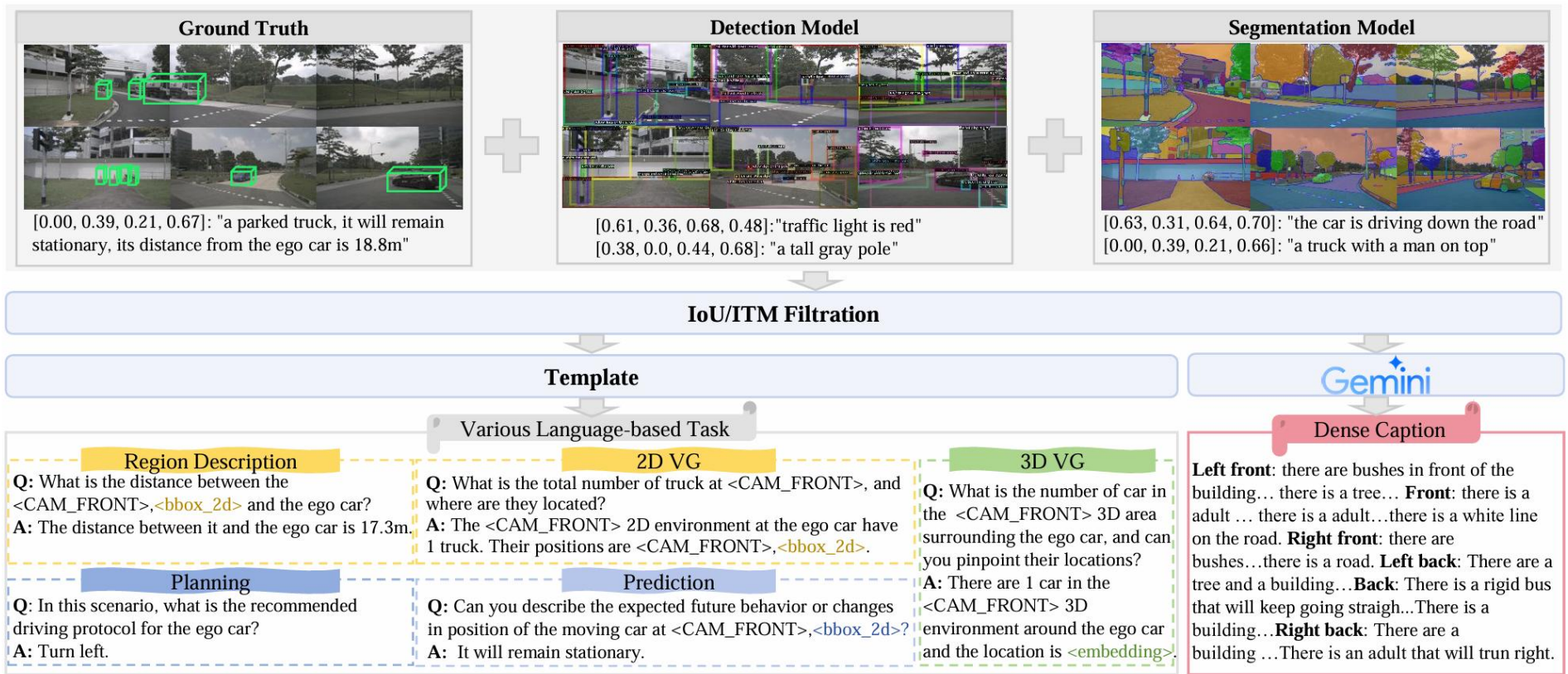
自动驾驶VLA数据集制作——QA Scene Understanding

- ❑ 回顾第一章中VLA开源Benchmark介绍，QA的场景理解可以包含哪些任务？
- ❑ DriveLM数据集是怎么制作的？视频序列选一些**关键帧（数据挖掘）**，确定**问题模板**（DriveLM把问题模板分为感知-预测-决策-行为-运动），然后通过**感知自动标注**和**其他人工标注**实现QA的制作（数据标注）。



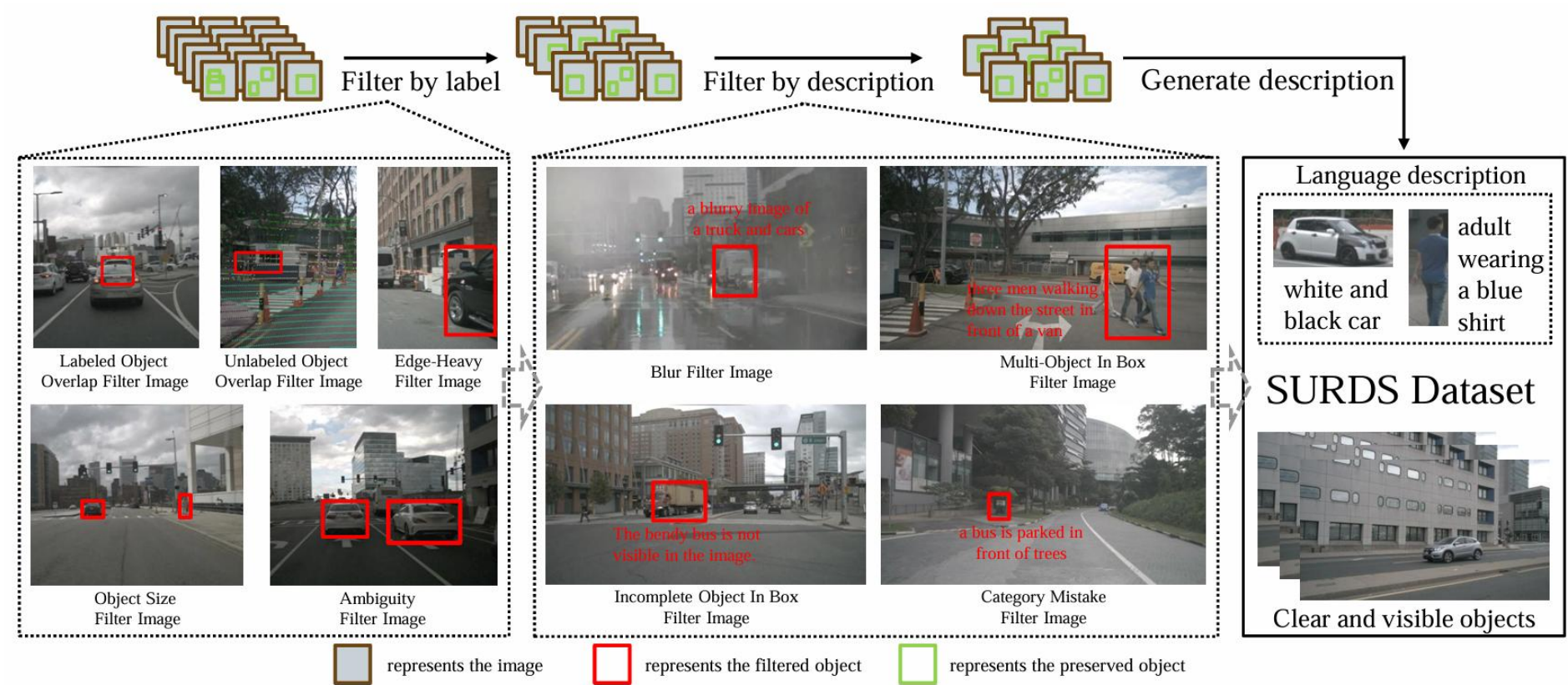
自动驾驶VLA数据集制作——QA Scene Understanding

- ❑ 回顾第一章中VLA开源Benchmark介绍，QA的场景理解可以包含哪些任务？
- ❑ 以场景交互理解为代表的nuInteract是如何制作的？数据挖掘-任务需求-数据标注思考
- ❑ 通过nuscenegs**感知标注**+开放**词汇检测感知标注**+语言**分割模型**+Gemini描述，得到**选出的视**
频频帧中的密集场景描述，其次设定**5种问题模板**，通过**密集场景描述**来匹配这些问题的答案；



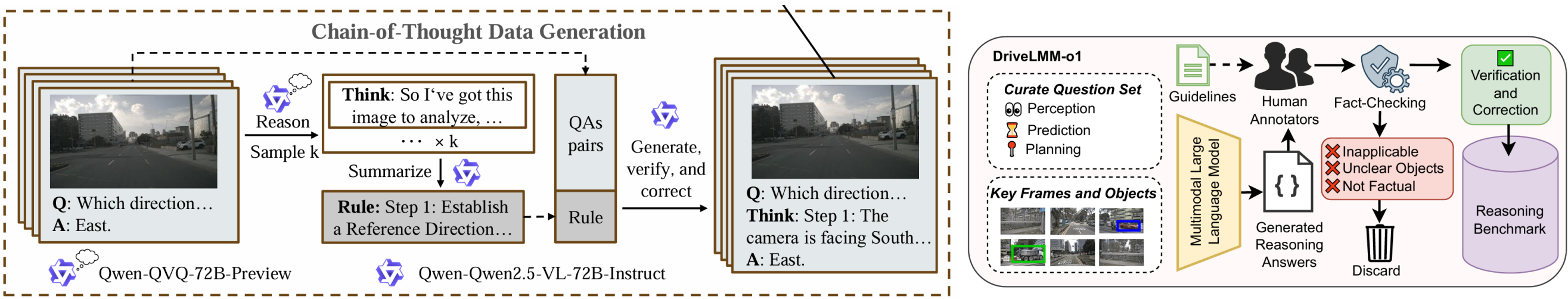
自动驾驶VLA数据集制作——Grounding

- ❑ 回顾第一章中VLA开源Benchmark介绍，QA的场景理解可以包含哪些任务？
- ❑ DriveMLLM空间推理的任务；
- ❑ 考考你，这个数据集可以怎么制作？



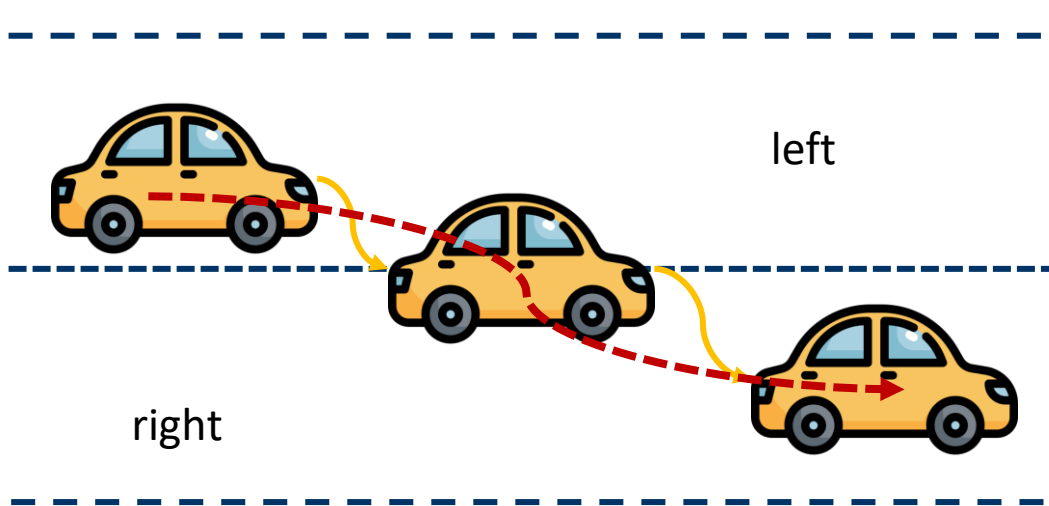
自动驾驶VLA数据集制作——CoT

- ❑ CoT数据的表现形式有哪些？根据前面章节的知识点，CoT可以表征为一系列步骤组成的语料；
- ❑ 最常用的方式就是先选定关键帧（图），然后设计问题模板（文），然后用专家模型去进行云端标注，用另外一个专用的智能体去修饰过滤前面生成的云端标注内容，最终人工抽检；
- ❑ DriveMLLM和DriveLMM-o1就是通过这样的方式，你发现了吗？

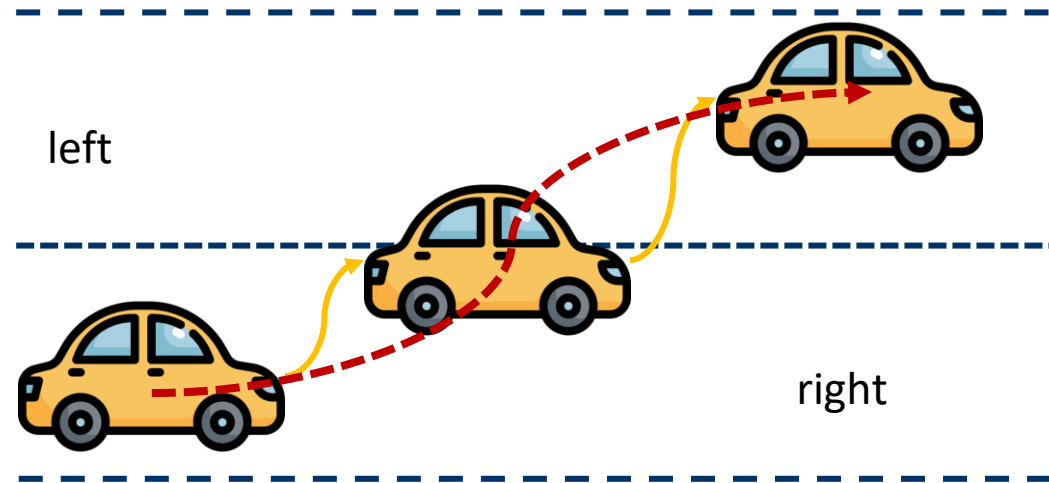


自动驾驶VLA数据集制作——Planning

- Planning数据的表现形式有哪些？从前面动作解码器的部分，Planning可以表征为语言形式的高级别语义导航、轨迹点以及数值形式的轨迹点或者动作控制信号；
- 通过自车未来的位置和当前帧位置的差值，即可得到，自车的轨迹信息，通过自车的轨迹信息，也可以得到自车在这个时间段内的高级别语义导航指令，因此这里需要设定问题模板，然后通过前面得到的轨迹或者高级别语义导航指令，就能制作这个数据集；



自车位置变换得到轨迹
高级别语义导航信息是向右边变道（道路上面部分为左边）



自车位置变换得到轨迹
高级别语义导航信息是向左边变道（道路上面部分为左边）



自动驾驶VLA数据集制作——Sce2DriveX

➤ 考考你，下面论文中制作数据的流程是怎么样的？

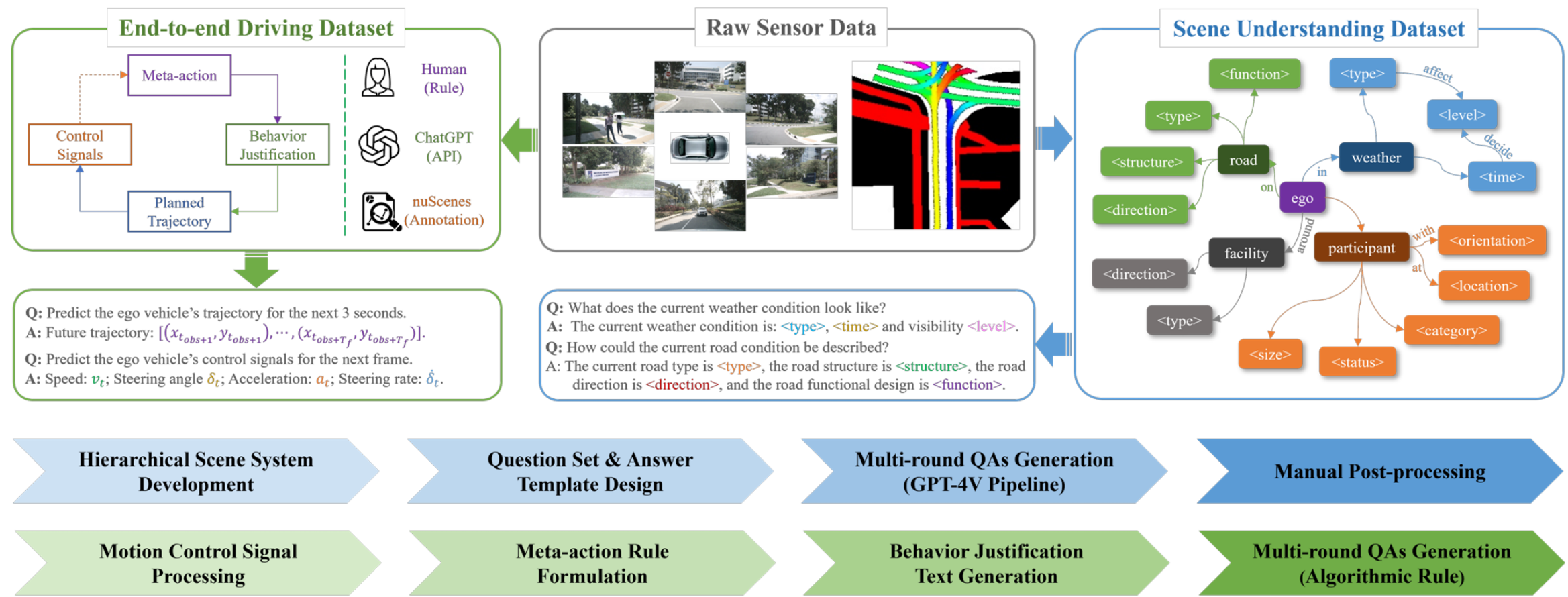
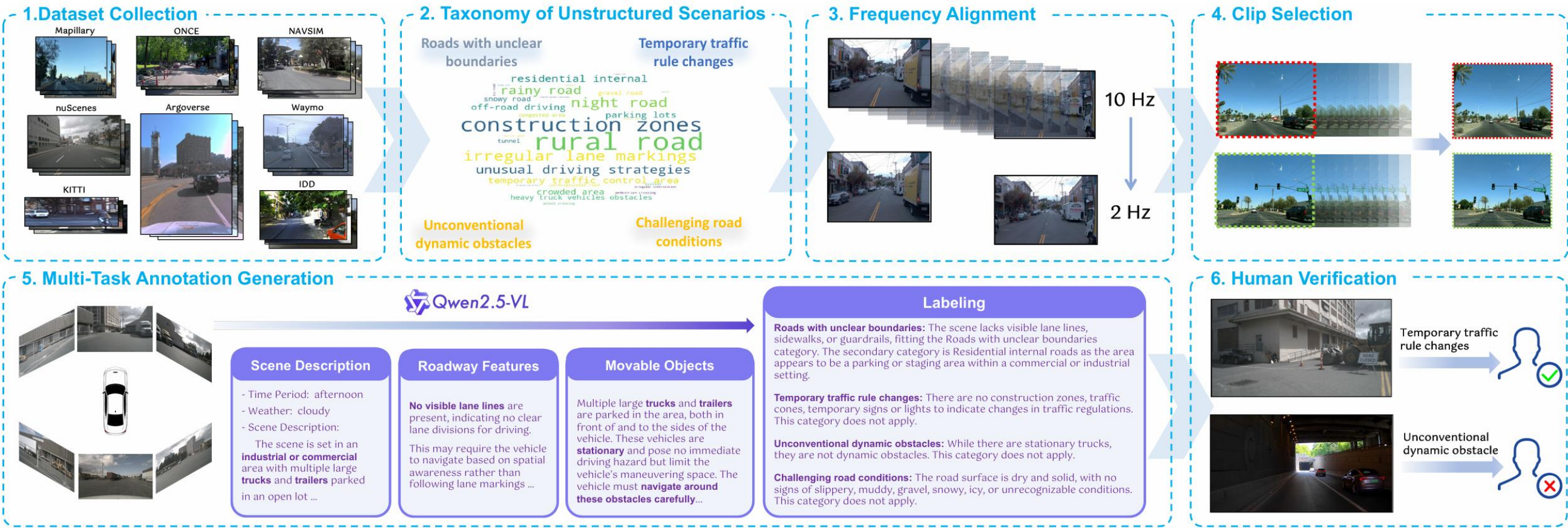


Figure 3. **Dataset construction.** Comprehensive VQA driving instruction dataset consists of two subsets: 1) Hierarchical Scene Understanding Dataset; 2) Interpretable End-to-End Driving Dataset.



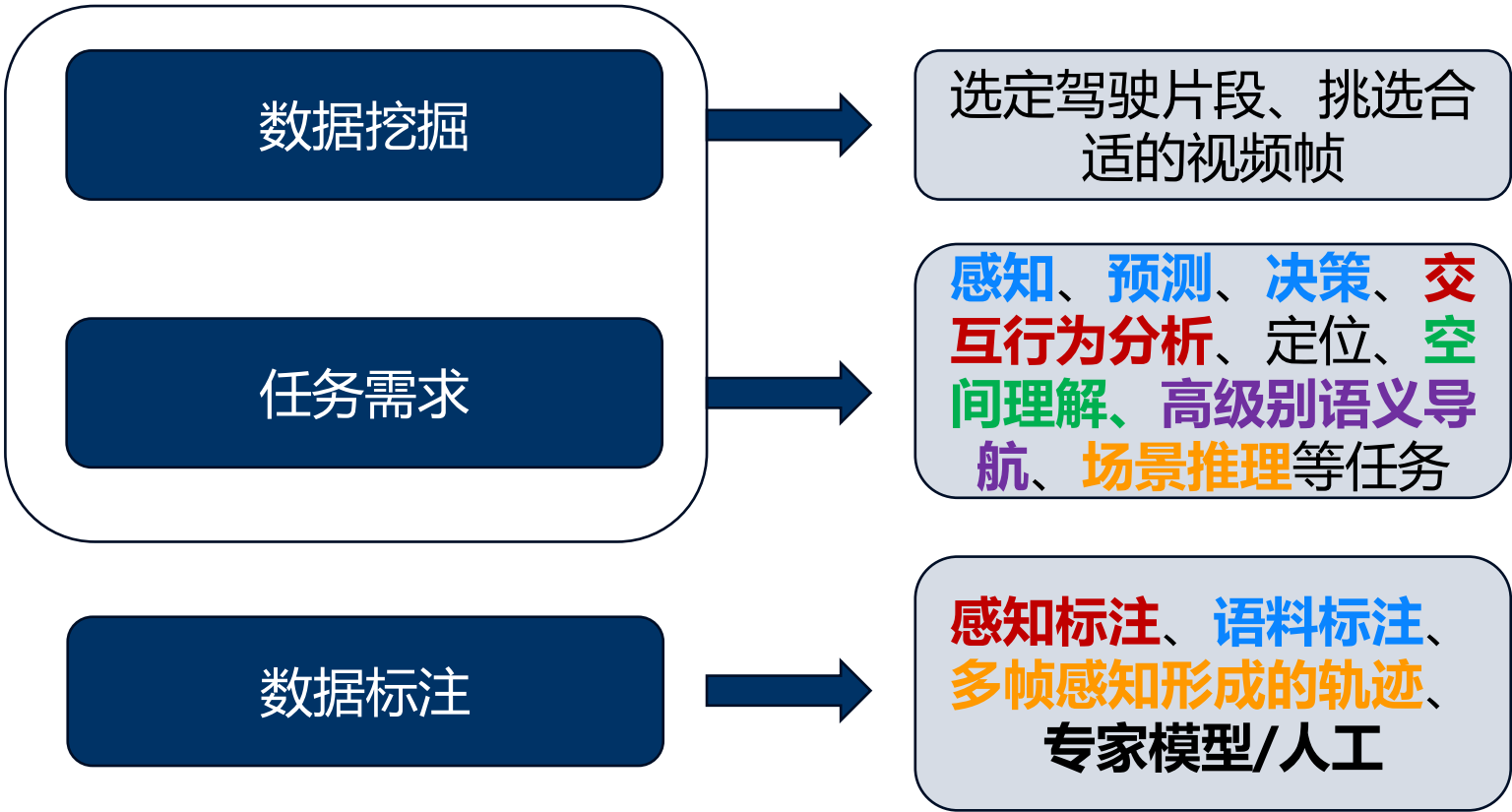
自动驾驶VLA数据集制作——CoT

❑ 复杂的VLA数据汇总：ImpromptuVLA, 先挖掘非结构化场景，然后选关键帧，接着再由Qwen模型生成密集描述，**场景分类与信息提取**，最后设计问题模板进行多任务问答生成



VLA数据制作总结

- 输入:
- 视觉、语言
- 输出:
- 语言、动作, 也有一些可以输出目标检测等自动驾驶的信息



□ 设定输入和输出的映射关系

- DriveLM
- nuInteract
- Surds(DriveMLLM)
- DriveLMM-o1
- Sce2DriveX
- ImpromptuVLA



1. 主要和观众互动，回答问题、相互讨论！
2. 欢迎大家提问xxxxx！
3. 也可以在PPT上展示领域常见的一些问题引发讨论xxx！



公众号



自动驾驶与计算机视觉
日常干货分享

知识星球



加入自动驾驶之心知识星球，
获取更多硬核干货

自动驾驶与AI全栈技术学习+领域大咖交流+职位内推！